

Dedications

To my family, my unwavering foundation.

To my parents, **Younes Tlili** and **Imen Tounsi**. Your belief in me has always been the steady light guiding my way. Mom, thank you for your endless encouragement and for teaching me compassion. Dad, thank you for your quiet strength and for showing me the value of perseverance. Your sacrifices and love are the bedrock upon which this achievement is built. I hope I have made you proud.

To my brothers, **Amine Tlili** and **Aziz Tlili**. Thank you for the laughter, the support, and for always being a source of motivation for me to aim high and reach higher places.

To my beloved sisters, **Yassmine Tlili** and **Mayar Tlili**, They say brothers provide competition, but sisters provide perspective. Your wisdom, empathy, and unwavering belief in me have been a quiet but constant force throughout this journey. You've shaped the man I am, and therefore, every page of this work.

TLILI Mohamed Moamen

Acknowledgments

*First and foremost, praise and thanks to **Allah** the Almighty, the most gracious and the most merciful, for His blessings and grace.*

*I would like to extend my sincere gratitude to my academic supervisor, Dr. **RAIK AISSAOUI**. Your guidance was a masterclass in critical thinking and scholarly inquiry. Thank you for your immense patience, for reading through my drafts, and for the insightful feedback.*

*I was extremely fortunate to carry out the practical phase of this research at the **Association for Scientific Research and Innovation in Computer Science**, an opportunity that would not have been possible without the support of my industry supervisor, Dr. **NOURCHENE OUERHANI**. Your mentorship was instrumental in bridging the gap between academic theory and real-world engineering.*

Contents

General Introduction	1
1 General context	3
1.1 Context & Background	4
1.1.1 The Role of Communication in Modern Workplaces	4
1.1.2 Challenges of Unmoderated CSNs	4
1.1.3 The Case for AI-Driven Moderation	5
1.2 Host Organization	5
1.2.1 Its Services	5
1.3 Motivation and Problem Statement	6
1.3.1 Motivation	6
1.3.2 Problem Statement	6
1.4 Objectives	7
1.5 Existing Solutions and Related Work	8
1.5.1 Overview of Prominent Corporate Social Network Platforms	8
1.5.2 Critique and Analysis of Content Moderation in Existing Platforms	9
1.5.2.1 Microsoft Viva Engage:	9
1.5.2.2 SLACK:	10
1.5.2.3 Jive:	10
1.5.3 Advancements in Hate Speech Detection and Associated Technologies	11
1.5.3.1 Rule-Based Approach	11
1.5.3.2 Limitations of The Rule-Based Approach	11
1.5.3.3 Machine Learning-Based Approach	12
1.5.3.4 Limitations of Machine Learning-Based Approach	13
1.5.3.5 Deep Learning-Based Approach	13
1.5.3.6 Comparative Study of Hate Speech Detection Methods	17
1.5.4 WorkSpere As a Solution	18
1.5.4.1 WorkSpere vs. Viva Engage	19
1.5.4.2 WorkSpere vs. Slack	20
1.6 Work methodology	21
1.6.1 Agile methods	21

1.6.1.1	Extreme Programming (XP)	21
1.6.1.2	Feature Driven Development (FDD)	21
1.6.1.3	Adaptive software development (ASD)	22
1.6.1.4	Scrum	22
1.6.2	The chosen methodology	22
1.7	Gantt Diagram	23
2	Sprint 0: Project Initiation	24
2.1	Needs Framing	25
2.1.1	Actors Identification	25
2.1.2	Needs Analysis	25
2.1.2.1	Functional requirements	25
2.1.2.2	Non-functional requirements	27
2.1.3	Overall Use Case Diagram	27
2.2	Product backlog	28
2.3	Overall Application Architecture	30
2.4	Working environment	31
2.4.1	Hardware environment	31
2.4.2	Development environment	32
2.4.3	Software environment	33
2.4.4	DataBase Service	33
3	Sprint 1: Authentication, Feed and Notifications	34
3.1	Sprint 1 Backlog	35
3.2	Functional specification	36
3.2.1	Refined Use Case Diagram « Sprint 1 »	37
3.2.2	Description of refined use case « Authentication »	38
3.2.3	Description of refined use case « Create Content »	40
3.2.4	Description of refined use case « Like Content »	41
3.2.5	Description of refined use case « Edit Content »	42
3.2.6	Description of refined use case « Delete Content »	43
3.2.7	Description of refined use case « Notifications System »	44
3.2.8	Logical Flow and Structure« Feed »	46
3.2.9	Logical Flow and Structure«Notifications»	47
3.3	Design	48
3.3.1	Sequence diagram « Authentication »	48
3.3.1.1	Sequence diagram « Sign up »	48
3.3.1.2	Sequence diagram « Login »	49
3.3.2	Sequence diagram « Content Creation »	50

3.3.3	Sequence diagram « Like Content »	51
3.3.4	Sequence diagram « Notifications »	52
3.3.5	Class Diagram	53
3.4	Implementation	54
4	Sprint 2: Profile Page, Edit Account and Admin Dashboard	58
4.1	Sprint 2 Backlog	59
4.2	Functional specification	60
4.2.1	Refined use case diagram « Sprint 2 »	60
4.2.1.1	Use case description « View Profile »	62
4.2.1.2	Use case description « Edit Profile »	62
4.2.1.3	Use case description « View Users List »	63
4.2.1.4	Use case description « Delete Users »	65
4.2.1.5	Use case description « Update User Role »	66
4.3	Design	67
4.3.1	Scenario	67
4.3.1.1	Sequence diagram « View Profile »	67
4.3.1.2	Sequence diagram « Edit Profile »	68
4.3.1.3	Sequence diagram « Toggle User Roles (Admin) »	68
4.3.2	Class Diagram	69
4.4	Implementation	71
5	Sprint 3: Implementing Manual Moderation, and Dataset Preparation.	75
5.1	Sprint 3 Backlog	76
5.2	Functional specification	76
5.2.1	Refined Use Case Diagram « Sprint 3 »	77
5.2.2	Description of refined use case « Sprint3»	77
5.2.2.1	Description of the use case « Report Content »	77
5.2.2.2	Use Case Description « View Reported Content »	78
5.2.2.3	Use Case Description « Delete Reported Content »	79
5.2.2.4	Use Case Description « Allow Reported Content »	80
5.3	Design	81
5.3.0.1	Sequence diagram « Report Content »	81
5.3.0.2	Sequence diagram « Delete Reported Content »	81
5.3.1	Classe Diagram	82
5.4	Dataset Preparation	84
5.4.1	Challenges	84
5.4.2	Datasets Curation	85
5.4.3	Data Split	87

5.4.4	Data Balancing, and Augmentation Techniques	87
5.5	Implementation	89
6	Sprint 4. AI Model Development and Integration.	92
6.1	Sprint Overview	94
6.1.1	Sprint 4 backlog	94
6.1.2	Functional specification	95
6.1.2.1	Refined use case diagram « Sprint 3 »	96
6.1.2.2	Refined use case description « Sprint 4 »	97
6.1.2.2.1	Use case description « Analyze Content for Hate Speech »	97
6.2	Design	98
6.2.1	Scenario	98
6.2.1.1	Sequence diagram « Analyze Content for Hate Speech »	98
6.2.2	Class Diagram	99
6.2.3	Description of the classes	100
6.3	Multilingual Hate Speech Detection	100
6.3.1	Challenges in Multilingual Hate Speech Detection	101
6.3.2	Our proposed Multilingual Hate Speech Detection Solution	102
6.4	Model Development and Integration	103
6.4.1	Hate Speech Detection As Binary Classification	103
6.4.2	Model Selection and Fine-tuning	103
6.4.3	Transformer models used	103
6.4.4	Fine-Tuning Approaches	107
6.5	Model Evaluation	108
6.5.1	Chosen Evaluation Metrics	108
6.5.2	Evaluation Results and Model Selection	109
6.6	Model Deployment & API Integration	113
6.6.1	Model Deployment Architecture	113
6.6.2	Model Integration	113
6.7	Implementaion	115
	General conclusion	117
	Bibliography	120

List of Figures

1.1	Generalization problem of rules. Logical rules, while easy to explain, are inherently fragile to the nuances of natural language [17].	12
1.2	The Transformer - model architecture [35].	15
1.3	SCRUM Life Cycle	22
1.4	Gantt Diagram	23
2.1	General Use Case Diagram	28
2.2	Overall Application Architecture	31
3.1	Use case diagram "Sprint 1"	37
3.2	Flow Chart "Feed"	46
3.3	Flow Chart "Notifications"	47
3.4	Sequence diagram "Signup"	48
3.5	Sequence diagram "Login"	49
3.6	Sequence diagram "Post Creation"	50
3.7	Sequence diagram "Comment Creation"	51
3.8	Sequence diagram "Like Content"	52
3.9	Sequence diagram "Notifications"	52
3.10	Class diagram for Sprint 1	53
3.11	Sign Up Interface	54
3.12	Login Page	55
3.13	Feed Page	55
3.14	notifications display	56
3.15	notifications display	56
4.1	Refined use case diagram for Sprint 2	61
4.2	Sequence diagram "View Profile"	67
4.3	Sequence diagram "Edit Profile"	68
4.4	Sequence diagram "Toggle User Role"	69
4.5	Class diagram for Sprint 2	70
4.6	interface "Profile Page"	71
4.7	interface "Edit Profile-1"	72

4.8	interface "Edit Profile-2"	72
4.9	Interface User management	73
5.1	Refined use case diagram "Sprint 3"	77
5.2	Sequence diagram "Report Content"	81
5.3	Sequence diagram "Delete Reported Content"	82
5.4	Class diagram for Sprint 1	83
5.5	Reporting Content, Step 1	89
5.6	Reporting Content, Step 2	89
5.7	Reporting Content, Step 3	90
5.8	Reported Comments, Step 4	90
6.1	Refined use case diagram of "sprint 4"	96
6.2	Sequence diagram "Check Content for Hate Speech"	98
6.3	Class diagram of Sprint 4	99
6.4	Process flow of multilingual hate speech detection [25].	103
6.5	Architecture of the Language Agnostic BERT (LaBSE) model, illustrating its dual-encoder structure with shared parameters [22].	105
6.6	WorkSphere System Architecture with Integrated Hate Speech Detection .	114
6.7	AI model in action	115
6.8	AI logs and Activity table	116

List of Tables

1.1	Comparative Analysis of Hate Speech Detection Approaches	18
1.2	WorkSphere vs VivaEngage	20
1.3	WorkSphere vs Slack	21
2.1	Project backlog	30
3.1	Sprint 1 Backlog	36
3.2	Use Case Description "Authentication"	40
3.3	Use Case Description "Create Content"	41
3.4	Use Case Description "Like Content"	42
3.5	Use Case Description "EditContent"	43
3.6	Use Case Description "Delete Content"	44
3.7	Use Case Description "Notifications System"	45
3.8	Participating Classes in Sprint 1	54
4.1	Sprint 2 backlog	60
4.2	Use case description related to "View Profile"	62
4.3	Use case description related to "Edit Profile."	63
4.4	Use case descriptions related to " View Users List "	64
4.5	Use case descriptions related to " Delete Users "	65
4.6	Use case descriptions related to " Update User Role "	66
4.7	Description of the participating classes in the sprint	71
5.1	Sprint 3 Backlog	76
5.2	Use case description "Report Content"	78
5.3	Use case description "View Reported Content"	79
5.4	Use case description "Delete Reported Content"	80
5.5	Use case description "Allow Reported Content"	80
5.6	Description of the participating classes in the first sprint	84
5.7	Multiple hate speech datasets per language, showing number of samples and sources	86
5.8	Class Distribution in the Multilingual Hate Speech Corpus	87

6.1	Sprint 1 Backlog	95
6.2	Features and Specifications	96
6.3	Use case description "Analyze Content for Hate Speech"	98
6.4	Description of the classes participating in sprint 4	100
6.5	Performance Summary of Base Models (Basic)	110
6.7	Overview of Experiments On The Selected LaBSE Model	112
6.8	Evaluation Results for The Different Experiments	112

General Introduction

In today's fast-paced corporate world, effective communication and collaboration are the cornerstones of a productive and innovative workplace. Companies strive to create an environment where employees can exchange ideas, share knowledge, and build meaningful professional relationships. With the rise of digital transformation, organizations have increasingly turned to Corporate Social Networks (CSNs), private social platforms designed to facilitate internal communication, encourage teamwork, and strengthen company culture. These platforms aim to enhance employee engagement, foster a sense of belonging, and ultimately contribute to the overall well-being and efficiency of the workforce.

However, as with any online community, maintaining a respectful and professional discourse remains a challenge. Workplace discussions can sometimes escalate into unprofessional behavior, with instances of hate speech, discrimination, and offensive language emerging, whether intentionally or inadvertently. Such toxic interactions can undermine workplace harmony, decrease employee morale, and even lead to legal and ethical concerns for organizations. This highlights the growing need for intelligent and automated content moderation systems capable of maintaining a safe and inclusive digital space without hindering open communication.

To address this challenge, this project introduces WorkSphere—an AI-powered corporate social hub that integrates multilingual transformer-based hate speech detection models to ensure a respectful and engaging digital workplace. Unlike traditional moderation techniques that rely on simple keyword filtering, WorkSphere leverages advanced Natural Language Processing (NLP) models, including LaBSE, mBERT, and XLM-RoBERTa, to detect and mitigate toxic content in multiple languages. Furthermore, by combining social networking features with AI-driven moderation, WorkSphere enables companies to foster healthy workplace discussions while safeguarding employees from harmful interactions.

This thesis explores the design, implementation, and evaluation of WorkSphere, detailing the development of both the MERN-based web application and the multilingual hate speech detection model. It also discusses the integration of deep learning-based content moderation within a corporate social network, emphasizing the technical, ethical, and practical considerations involved in deploying AI-powered moderation at scale.

The following chapters will present an in-depth analysis of the problem, the system architecture, the model training process, the integration of AI with the web application, and the evaluation of the final system.

This report is organized into 6 chapters:

- The first chapter provides an overview of the project and the context and background of AI-driven moderation in corporate social networks. Additionally, it highlights and outlines the host organization's role, clarifies our objectives, motivations, and methodology.
- The second chapter introduces Sprint 0, the initiation phase of our Scrum project, covering requirements gathering, product backlog elaboration, and architectural concepts.
- The third chapter documents Sprint 1, in which we developed the authentication, feed page, and notifications systems. It presents the design, specification, and implementation as well as functional specifications, use case diagrams, logical flows, and relevant sequence diagrams.
- The fourth chapter presents Sprint 2's development of the profile page, account editing, and admin dashboard features, including their functional specifications, sequence diagrams, and class designs. Furthermore, it details key use cases like profile viewing/editing and admin actions.
- The fifth chapter presents Sprint 3's implementation of the manual moderation feature that allows users to flag hateful or offensive content. Additionally, it showcases the process of data collection and dataset preparation, laying the groundwork for the next chapter.
- The sixth chapter presents Sprint 4 in which we developed and evaluated the AI model for hate speech detection and then detailed the process of its integration into our website.

Chapter 1

General context

Plan

1.1	Context & Background	4
1.1.1	The Role of Communication in Modern Workplaces	4
1.1.2	Challenges of Unmoderated CSNs	4
1.1.3	The Case for AI-Driven Moderation	5
1.2	Host Organization	5
1.2.1	Its Services	5
1.3	Motivation and Problem Statement	6
1.3.1	Motivation	6
1.3.2	Problem Statement	6
1.4	Objectives	7
1.5	Existing Solutions and Related Work	8
1.5.1	Overview of Prominent Corporate Social Network Platforms	8
1.5.2	Critique and Analysis of Content Moderation in Existing Platforms	9
1.5.3	Advancements in Hate Speech Detection and Associated Technologies	11
1.5.4	WorkSpere As a Solution	18
1.6	Work methodology	21
1.6.1	Agile methods	21
1.6.2	The chosen methodology	22
1.7	Gantt Diagram	23

1.1 Context & Background

1.1.1 The Role of Communication in Modern Workplaces

Research indicates that companies fostering open dialogue and transparent communication see up to 43% lower employee turnover [24] and a 20–25% increase in productivity [34]. Organizations prioritizing psychological safety and regular feedback also report higher employee engagement, satisfaction, and innovation. Beyond task coordination, communication fosters trust, empathy, and psychological safety, key drivers of innovation and job satisfaction. According to a Harvard Business Review study, employees in high-trust and communication workplaces are 76% more engaged and 29% more satisfied with their lives [38].

Corporate Social Networks (CSNs) like Microsoft Viva Engage or Workplace by Meta have emerged as tools for sustaining these values in distributed teams. By replicating social media-style interactions within a professional context, CSNs enable employees to share ideas, celebrate milestones, and build rapport, bridging geographical and cultural divides.

1.1.2 Challenges of Unmoderated CSNs

Despite their potential, unmoderated CSNs risk becoming breeding grounds for harmful content. EVERF, a private education technology company that provides training and resources on workplace compliance, reported in a study that 25% of employees experienced increased gender-based harassment while working remotely, and 10% of employees reported an uptick in race or ethnicity-based harassment during remote work [21]. This harassment, unchecked, would often lead to decreased morale and increased turnover.

the research done by the EVERFI organization also provided insight into the form of harassment in online work spheres, and amongst the examples cited, we mention:

- Insults in a chat feed.
- offensive comments/jokes.
- Commenting negatively on pictures or religious/cultural items in a video background.

This sort of toxic interactions—whether explicit slurs or subtle biases—can erode trust, isolate individuals, and undermine the very purpose of workplace social networks.

Traditional moderation tools, such as keyword-based filters used in applications like Slack or Workplace by Facebook, fail to address these challenges. They lack contextual awareness (e.g., sarcasm, cultural nuances) and struggle with multilingual content, leaving

organizations vulnerable to escalating conflicts. For global companies, this gap is particularly acute: a phrase deemed harmless in one language might carry offensive connotations in another, highlighting the need for adaptive, intelligent moderation systems.

1.1.3 The Case for AI-Driven Moderation

To address the limitations of traditional moderation systems, AI-powered solutions leveraging Natural Language Processing (NLP) and deep learning have emerged as transformative tools in communication. Transformer-based models like BERT and LaBSE excel at understanding context, sentiment, and cross-lingual semantics, allowing for precise detection of toxic and harmful content in real-time. These models can process complex expressions, such as sarcasm or cultural nuances, that traditional filters often fail to catch.

One of the major advantages of AI-driven moderation is its ability to adapt to evolving language. Unlike rule-based systems, which rely on fixed keywords, AI models can continuously learn and adjust to new forms of hate speech or offensive behavior, providing a dynamic solution to an ever-changing environment. Moreover, these models offer cross-lingual capabilities, allowing for the management of communication across a global workforce while maintaining a consistent, respectful atmosphere.

WorkSphere leverages these cutting-edge technologies to provide a level of moderation that traditional tools cannot match.

1.2 Host Organization

The ARSII (Association for Research and Innovation in Computer Science) is a scientific, non-political, and non-profit organization established in May 2016 by a group of young computer science researchers and university lecturers in Tunisia. Dedicated to advancing scientific research and innovation in the field of computer science, ARSII actively organizes workshops and engages in research and development partnerships with the aim of providing its members with valuable opportunities for knowledge sharing, skill development, and collaboration with experts and institutions in the field.

1.2.1 Its Services

The service offered by ARSII includes: Organizing national and international scientific events.

- Supporting scientific research and innovation in computer science.
- Initiating, mentoring, and overseeing innovative projects.
- Promoting cooperation with socio-economic partners at both national and international levels.
- Bridging the gap between academic research and the industrial sector.
- Facilitating the integration of students into the professional world.

1.3 Motivation and Problem Statement

1.3.1 Motivation

This project is motivated and propelled by the increasing reliance on digital communication tools in professional environments and the need for advanced and modern state-of-the-art AI-driven moderation solutions. WorkSphere aims to create a corporate social hub with AI-powered content moderation, fostering a safe, inclusive, and engaging digital space for employees. It aims to employ a reliable and advanced AI model that leverages natural language processing (NLP) and deep learning by integrating state-of-the-art multilingual and domain specific transformer models for hate speech detection, as well as a website that provides dynamic social networking features with a minimalist and stylistic design

1.3.2 Problem Statement

The advent of corporate social networks (CSNs) has transformed workplace communication, yet many organizations struggle with low employee engagement on these platforms. Moreover, the risk of having workplace toxicity and harassment carry over to the social network is still ever present, thereby always undermining the benefits of CSNs.

This project proposes a corporate social hub application with AI-powered moderation that addresses the following challenges:

—AI Moderation Limitations:

- Contextual Ambiguity: Detecting and distinguishing harmful content reliably.
- Multilingual Sensitivity: Detecting hate speech in the relevant languages (Arabic, English, and French) without misinterpreting cultural nuances.

—Web Development Complexities:

- Design: Designing a minimalist, visually appealing design that focuses on simplicity, clarity, and functionality
- Functionalities: Incorporate key functionalities from popular social media platforms—tailored for professional, secure, and productivity-focused use.
- Real-Time Integration: Ensuring AI moderation operates seamlessly within user workflows (e.g., posting, commenting) with low latency
- Admin Analytics: Generating actionable insights through real-time toxicity reports, employee behavior trends, and moderation performance metrics.

—Data Transparency:

- Visualization Challenges: Designing intuitive dashboards to display toxicity levels, flagged content trends, and model accuracy across departments or languages.

—Employee engagement:

- No private chats: Encouraging inclusive and public discussions that involve all employees.

1.4 Objectives

WorkSphere aims to ensure that corporate discussions are empowered and that productive, professional, and engaging relationships can be formed and nurtured between employees and teams, ultimately fostering a positive and collaborative work culture. Therefore, the primary objectives can be summarized as follows:

- Develop a private social network for companies.
- Enhance employee engagement through features like employee feeds, posts, and topic-based discussions.
- Implement hate speech detection capabilities to automatically flag offensive content.
- Provide administrators with tools to review flagged content and monitor workplace toxicity.

1.5 Existing Solutions and Related Work

The evolution of workplace culture, management theories, and technological advancements has always been closely tied to healthy communication and interactions. This connection dates back to the early Industrial Era (late 19th – early 20th century), notably with the emergence of Frederick Taylor’s Principles of Scientific Management, which emphasized four key principles:

- Science, not rule of thumb.
- Harmony, not discord.
- Cooperation, not individualism.
- Development of each and every person to their greatest efficiency.

In parallel, Elton Mayo’s studies highlighted the significant impact of social and psychological factors, such as communication and recognition, on employee productivity. Since then, the recognition of the importance of social interaction and communication in the workplace has only continued to grow in both relevance and prominence, alongside the development of approaches and technologies aimed at fostering it, and thus the birth of Corporate Social Networks (CSNs) as essential tools for fostering communication, collaboration, and knowledge sharing within organizations.

1.5.1 Overview of Prominent Corporate Social Network Platforms

Amongst Corporate Social Networks, the most prominent and relevant to showcase are:

- Microsoft Viva Engage (formerly Yammer): an enterprise social networking service that is part of the Microsoft 365 family of products. It lets employees post updates, ask questions, share knowledge, and join communities within the organization. Additionally, it promotes engagement with leadership through features like AMAs (Ask Me Anything) and helps celebrate achievements.
- Slack: a cloud-based team communication platform developed by Slack Technologies that offers a user-friendly interface, supports real-time public and private messaging, file sharing, and integration with various third-party applications. slack is used

primarily by businesses, remote teams, and organizations to improve internal communication and collaboration. Slack provides workspace administrators with tools to manage and moderate content in their organizations. Admins can set rules for what is acceptable behavior, manage user permissions, and ensure that inappropriate content is addressed. Additionally, Admins can delete or edit messages that violate workplace policies, including hate speech or offensive language.

- Jive: a provider of communication and collaboration software for business that offers a comprehensive suite for internal collaboration, including discussion forums, blogs, and document sharing.

1.5.2 Critique and Analysis of Content Moderation in Existing Platforms

1.5.2.1 Microsoft Viva Engage:

Although it offers robust enterprise features and flaunts deep integration with the Microsoft 365 ecosystem, Viva Engage exhibits several weaknesses when evaluated against the goal of creating a truly safe, engaging, and culturally insightful corporate social hub.

First, the most significant limitation of Viva Engage is its moderation strategy, which is largely generic and reactive. It relies heavily on users reporting inappropriate content, placing the moderation burden on employees, and meaning that harm has already occurred and been witnessed before action is taken.

Second, this platform is embedded within Microsoft Teams, this can lead to a fragmented user experience and "app fatigue.", as it becomes just another tab or button within the already cluttered, work oriented Teams interface, and therefore It struggles to establish itself as a distinct "destination" for community and culture, often feeling like an auxiliary function rather than a central hub.

Thirdly, Viva Engage's analytics are primarily focused on adoption and engagement metrics: number of active users, posts, likes, and top contributors. While useful for measuring platform ROI, these metrics are superficial. They do not provide deep insights into the health of the corporate culture. They can tell you if people are talking, but not how they are talking.

Finally, while Viva Engage offers AI-driven models, they are trained on massive generalized datasets meant to be a "one-size-fits-all" solution. They are not, and cannot be, fine-tuned to the specific cultural context, jargon, and subtle forms of toxicity within a single company. Moreover, they are designed to create a queue of alerts for human

reviewers, so they're not entirely automatic and self-reliant.

1.5.2.2 SLACK:

Slack has established itself as an indispensable tool for real-time collaboration, but its very design presents fundamental limitations when considered as a platform for fostering a lasting corporate culture.

While it excels as a communication tool for teams, Slack is not designed as a full-fledged social platform with advanced moderation capabilities. As a result, it lacks comprehensive content analysis features and automated AI-driven safety mechanisms. The current Slack approach to moderating toxicity and hate speech is predominantly manual, relying on user reports and administrative oversight. When users encounter harmful behavior or inappropriate language, they are encouraged to report it to admins or workplace moderators, who then review the content and take appropriate action. While this system offers a degree of flexibility and human judgment, it is inherently reactive. Without AI-powered real-time moderation, harmful content can persist in channels for extended periods before action is taken. Although admins can delete messages, block users, and remove inappropriate content manually, these interventions often come too late to prevent harm. Additionally, its core paradigm of synchronous, ephemeral channels creates a fast-moving stream of information where valuable discussions and cultural moments are quickly buried, leading to a form of 'digital amnesia' that undermines the creation of a shared institutional memory. Consequently, Slack functions more as a digital office floor—optimized for tactical, project-first-based communication, than as a 'company town square' designed for community building and for fostering relations and a sense of belonging. This creates a distinct gap for a dedicated social hub architected not for transient operational chat, but for the persistent, safe, and meaningful interactions that cultivate a positive and enduring workplace culture.

1.5.2.3 Jive:

Although it emphasizes knowledge management and employee engagement, Jive lacks sophisticated, automated content moderation features, instead depending on conventional administrative controls, which we've already showcased how such an approach can be limiting and inefficient. Furthermore, Jive's attempt to unify blogs, wikis, forums, and social features into a single hub often results in a cumbersome and feature-bloated user experience. Its legacy architecture not only presents a steeper learning curve for users accustomed to the intuitive interfaces of contemporary social media

1.5.3 Advancements in Hate Speech Detection and Associated Technologies

In this section, we will go through four major categories of hate speech detection methods: rule-based, machine learning-based, deep learning-based, and transformer-based approaches, stating the positives as well as the limitations of each to finally justify our choice of approach.

1.5.3.1 Rule-Based Approach

The rule-based approach to hate speech detection is one of the earliest methods developed for this task, relying on predefined linguistic rules, patterns, or lexicons to identify offensive, discriminatory, or harmful speech content. Unlike machine learning-based methods, which learn from labeled data, rule-based systems do not require training on large datasets. Instead, they function by explicitly encoding linguistic and semantic rules that define what constitutes hate speech. These rules are typically defined by the programmer or domain expert and often involve keyword matching, regular expressions, or syntactic structures that highlight potentially hateful language[36]. This approach is interpretable and straightforward to implement, making it an attractive choice in settings where computational resources are limited or explainability is critical, or when there is limited labeled data.

1.5.3.2 Limitations of The Rule-Based Approach

One of the biggest limitations of this approach is how it's inherently static, requiring constant updates to remain effective as the language of hate speech evolves. Additionally, they struggle to account for linguistic context, such as sarcasm, metaphorical usage, or indirect expressions, which often mask hate speech in subtle ways.

These systems are also prone to high false-positive rates, as the presence of certain keywords does not always indicate harmful intent. For example, a rule-based system might misclassify a discussion about combating hate speech as containing hate speech itself due to keyword overlaps.

Moreover, as illustrated in Figure 1, rules are inherently fragile due to their static nature and lack of linguistic coverage. For example, a simple rule such as :

$$R_1 : \text{If contains ('hate' } \vee \text{ 'loathe' } \wedge \text{ 'women')}$$

Where:

- $\vee$ represents logical OR.
- $\wedge$ represents logical AND.

is effective in detecting some forms of hateful speech (e.g., "I hate women"), but it fails to address more complex expressions (e.g., "I despise females" or "I loathe people who hate women"), leading to either false positives or false negatives. This issue underscores the brittleness of rule-based systems when applied to the dynamic and context-rich nature of human language. These limitations become even more pronounced when viewed

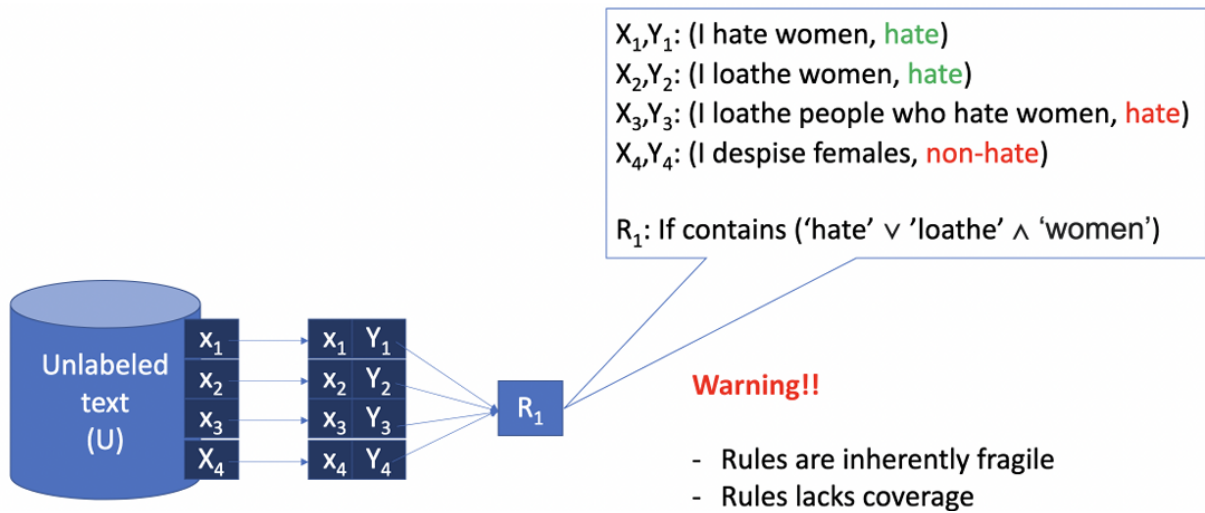


Figure 1.1: Generalization problem of rules. Logical rules, while easy to explain, are inherently fragile to the nuances of natural language [17].

through the lens of multilingual hate speech detection and the nuances of handling multiple languages. Addressing multilingual contexts through a rules-based system would require comprehensive rules for each language and their interactions, adding significant complexity that makes this approach inadequate.

1.5.3.3 Machine Learning-Based Approach

The machine learning (ML) paradigm has marked a significant advancement in the field of hate speech detection. Unlike rule-based approaches, which rely on predefined linguistic rules, machine learning-based methods define rules through examples. These methods utilize complex algorithms that learn from large datasets to identify patterns indicative of harmful or discriminatory language. The primary strength of this approach lies in its ability to continuously learn and adapt based on new data. It is dynamic, evolving its understanding and improving its decision-making process with fresh information. Moreover,

it can discern patterns in vast and diverse datasets, making it more effective in handling the complexities and ambiguities inherent in hate speech detection. machine learning approaches treat hate speech detection as a text classification problem, where the goal is to categorize text into predefined labels (e.g., hateful, neutral), and they operate within a supervised learning framework, training on labeled datasets that include examples of both. Unlike their rule-based predecessors, machine learning models do not require explicit rule definitions, allowing them to generalize to unseen instances of hate speech and are effective in detecting implicit and nuanced forms of offensive language, which elude rule-based systems. However, their performance is contingent on the availability of extensive and well-annotated training data.

1.5.3.4 Limitations of Machine Learning-Based Approach

Traditional ML models often rely on surface-level features, like TF-IDF (term frequency–inverse document frequency) or Bag-of-words models or N-gram frequency counts. These representations don’t capture context or meaning; they just see words as isolated tokens or short phrases. That makes models incapable of understanding social or cultural subtext or intent, and heavily dependent on the presence or absence of specific keywords seen during training, which causes huge bias towards specific keywords, ignoring semantics miss nuanced hate speech (e.g., coded language, indirect hate). for example, imagine you train a model on tweets like:

- “You are such a [slur]!”
- “Go back to your country!”

The model learns that these exact phrases or slurs are signals for hate speech. So now it assumes that: Anytime it sees that slur, it’s hate. If it doesn’t see such words, it’s not hate. Additionally, if this approach is employed for hate speech detection, users may be able to avoid detection by using coded language or irony, or seemingly innocent phrases with underlying hate.

1.5.3.5 Deep Learning-Based Approach

The advent of deep learning has enabled models to automatically learn hierarchical and contextually rich representations of text data. Unlike traditional machine learning approaches that rely on handcrafted features, deep learning methods extract meaningful patterns directly from raw data through multiple layers of neural networks, significantly

reducing the need for manual intervention. These models are capable of capturing complex relationships and addressing the variability and subtleties of human language, including implicit or context-dependent expressions of harmful speech. In particular, recent transformer-based models such as BERT and GPT have become the state-of-the-art in hate speech detection. They utilize contextual embeddings to capture word meaning in context and demonstrate superior generalization capabilities, especially due to their use of large-scale pretraining combined with domain-specific fine-tuning (e.g., on platforms like Twitter or Reddit). Transformers typically follow a semi-supervised learning paradigm: they are first pretrained in an unsupervised fashion on massive unlabeled corpora and then fine-tuned through supervised training for specific tasks. These models operate using a sequence-to-sequence framework in which input tokens are processed in parallel. Unlike traditional recurrent models that rely on sequential computation, transformers leverage an attention mechanism to model the dependencies between tokens.

Transformers introduced several key innovative technologies, such as:

- **Self-attention mechanism** The transformer architecture relies entirely on self-attention to capture relationships within sequences. In this mechanism, each token generates query, key, and value vectors, and attention scores are computed between all tokens to determine how much each token should influence the others. Unlike traditional attention, which typically focuses on inter-sequence relationships (e.g., between source and target sequences), self-attention focuses on intra-sequence relationships, enabling the model to process all tokens simultaneously. This ability allows the model to handle long-range dependencies, support parallel processing, and derive contextually accurate representations of meaning. For instance, consider the two sentences: For example, consider the sentences:

- "Server, can I have the check?"
- "Looks like I just crashed the server."

In the first sentence, "server" refers to a person; in the second, it refers to a computer. Self-attention enables the model to distinguish these meanings by considering the surrounding context..

- **Multi-Head Attention** One of the most significant innovations introduced by the transformer architecture is the multi-head attention mechanism. This approach extends the idea of attention by enabling the model to attend to different parts of the input sequence simultaneously, allowing it to capture a variety of relationships and dependencies from multiple perspectives. Instead of performing a single attention operation, multi-head attention runs several attention mechanisms in parallel,

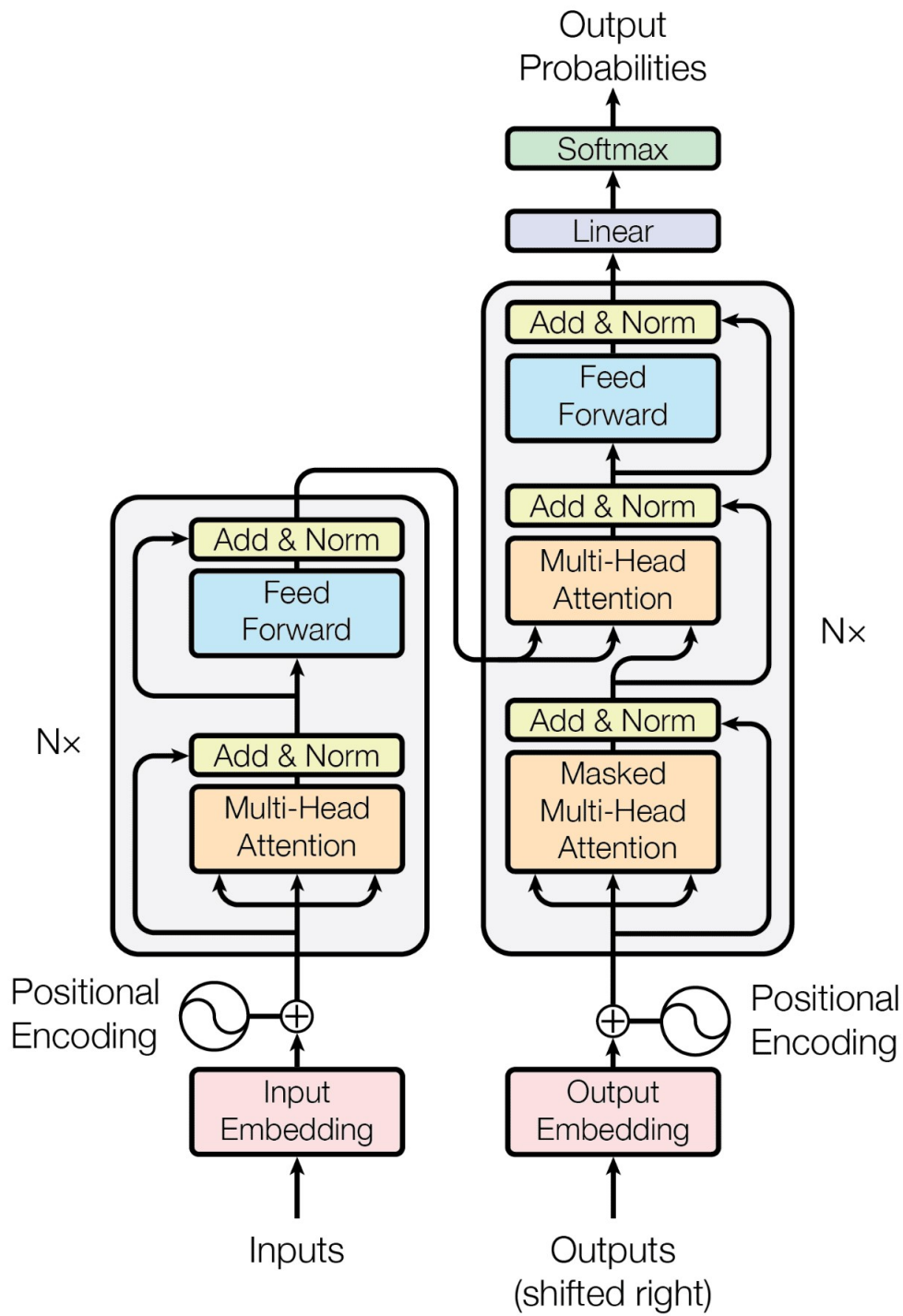


Figure 1.2: The Transformer - model architecture [35].

each with its own set of learnable parameters. Each "head" processes the input independently, enabling the model to learn different types of interactions between tokens within the sequence. By combining the outputs of all attention heads, the transformer generates a richer and more nuanced representation of the input. This enhances the model's ability to capture complex patterns and long-range depen-

dencies, while also improving its understanding of different linguistic structures and contextual nuances. For example, in the sentence:

- "The cat chased the dog,"

one attention head might focus on the relationship between "cat" and "chased," while another focuses on "chased" and "dog." This allows the model to simultaneously capture subject-verb and verb-object relationships, leading to a deeper understanding of sentence structure. Such capabilities are especially valuable in tasks like machine translation, sentiment analysis, and text generation.

- **Scaled Dot-Product Attention** Scaled dot-product attention is a core component of the transformer architecture and a major innovation that distinguishes it from earlier sequence models like RNNs and LSTMs. Instead of processing tokens sequentially, as in traditional recurrent networks, this mechanism allows the model to compute attention scores in parallel, making it highly efficient and scalable. In this mechanism, the model calculates attention by taking the dot product of query and key vectors for each token pair in the sequence, which measures how much focus one token should place on another. The result is then scaled by the square root of the dimension of the key vectors to mitigate the risk of large values causing vanishing gradients or unstable softmax outputs during training. This is followed by a softmax function that converts the scores into a probability distribution used to weight the value vectors. The use of dot-product attention, along with scaling, enables transformers to capture complex dependencies across the entire sequence more effectively and efficiently than previous architectures. It laid the foundation for the fully parallel processing capability of transformers, which has been a critical factor in their widespread success. This innovation was first introduced in the seminal paper "Attention is All You Need" by Vaswani et al. (2017) [35].
- **Pretraining and Transfer Learning** Although pretraining and transfer learning techniques existed prior to transformers, they were significantly popularized and advanced by models such as BERT and GPT. These techniques have become foundational in modern natural language processing (NLP), enabling models to leverage vast amounts of unlabeled data to develop a broad understanding of language, which can then be fine-tuned for specific downstream tasks.

Pretraining involves training a model on a large, general-purpose text corpus in an unsupervised manner, allowing it to learn fundamental language properties such as syntax, grammar, and contextual word relationships. Transfer learning then takes the pretrained model and fine-tunes it on a smaller, task-specific, labeled dataset.

This approach dramatically improves performance, especially in tasks with limited annotated data — a common challenge in real-world NLP applications, including hate speech detection.

By separating general language learning from task-specific learning, pretraining and transfer learning significantly reduce the need for extensive labeled data, lower computational costs, and improve generalization across diverse NLP tasks.

1.5.3.6 Comparative Study of Hate Speech Detection Methods

The table below 1.1 provides a comparative analysis of the different hate speech detection approaches discussed in the previous sections.

Approach	Key Features	Advantages	Limitations	Refs
Rule-Based	Uses predefined linguistic rules, lexicons, and patterns to identify hate speech.	• High interpretability and explainability. • Does not require training data. • Simple and fast to implement.	• High false positives • Cannot generalize well • Requires frequent updates to adapt to evolving language	[36], [23]
Machine Learning	Learns patterns from labeled datasets using algorithms like SVM, Naive Bayes, and Random Forest.	• Handles implicit hate speech better than rule-based • Dynamic and adaptable • Can generalize to unseen data	• Requires large labeled datasets • Class imbalance issues • Limited multilingual capabilities	[19], [37], [23]
Deep Learning	Employs neural networks (e.g., CNNs, LSTMs) to automatically learn hierarchical patterns in text data.	• Captures complex and contextual relationships • No need for manual feature engineering • Scalable for large datasets	• High computational cost • Prone to overfitting • Requires extensive labeled data	[15], [39], [29]

Trans- former- Based	Utilizes attention mechanisms and pretraining techniques to process contextual embeddings. Furthermore, it supports Parallel Processing and an Encoder-Decoder Architecture with Positional Awareness.	• State-of-the-art accuracy and performance • Handles subtle and implicit hate speech better than any other approach • Can adapt across languages and domains	• Extremely resource-intensive • Dependency on pretraining corpora	[20], [28], [18], [33]
-------------------------------------	--	---	---	------------------------

Table 1.1: Comparative Analysis of Hate Speech Detection Approaches

1.5.4 WorkSpere As a Solution

WorkSphere is designed as an intuitive yet sophisticated private social media platform tailored for workplace environments. It combines a clean, minimalistic aesthetic with a user-friendly interface and the core functionalities expected in a modern social application. Beyond usability, WorkSphere differentiates itself by integrating advanced AI-driven moderation systems aimed at ensuring safe, respectful, and inclusive digital interactions among employees. The platform addresses key challenges in workplace communication by proactively detecting and mitigating toxic behavior, harassment, and bullying. Through real-time, context-aware AI models, WorkSphere fosters a culture of healthy communication, mutual respect, and collaboration. By automating moderation in a reliable and scalable way, the application not only enhances user safety but also reduces the burden on human moderators, making it a robust solution for modern, diverse organizational settings.

Furthermore, WorkSphere aims to set an industry standard of Transparency & Trust through haring information about the data sources used to train the models, the performance and evaluation results as well as the steps taken to avoid biases therefore providing organizations with the ability to understand how content is being flagged or moderated and increasing trust in the system. This is a feature unavailable in any other application providing a similar service.

1.5.4.1 WorkSpere vs. Viva Engage

Aspect	Microsoft Viva Engage	WorkSphere
Core Philosophy & Purpose	Integrated Employee Experience. A social layer embedded within the broader Microsoft 365 productivity suite. Its primary goal is to add community features to an existing workflow.	Dedicated Corporate Social Hub. A standalone, purpose-built platform focused exclusively on fostering community, engagement, and a safe communication environment.
Moderation Approach	Reactive (Post-Facto). Content is published first, then scanned by Microsoft Purview policies. Harmful content may be seen before it is flagged and removed by an administrator.	Proactive (Pre-emptive). Content is analyzed by a dedicated middleware before being published. Harmful content is blocked in real-time at the API level.
AI Model Used	General-Purpose. Uses Microsoft's large-scale, standardized classifiers for threats, harassment, and profanity across all its customers. Lacks company-specific nuance and is not completely self-reliant.	Specialized. Employs a custom, multilingual transformer-based model for hate speech detection. Allows for fine-tuning to a specific company's culture and context.
User Feedback on Violations	Delayed & Opaque. A user's post is removed by an administrator, often hours or days later, with little to no automated feedback on the specific violation.	Instantaneous & Transparent. The ai service immediately rejects the post and can return a clear reason, providing an immediate learning opportunity.
Administrator's Role	Manual Reviewer. Admins must manually review a queue of flagged content in the separate Purview portal, creating a significant operational burden.	Strategic Overseer. The system automates the blocking of clear violations. Admins use a dedicated dashboard to monitor and review logs.

Aspect	Microsoft Viva Engage	WorkSphere
Data Accessibility	Siloed in Compliance Tools. Analytics are part of the broad M365 admin center or Purview, designed for IT and legal compliance, not for direct cultural insight by HR or leadership.	Centralized & Purpose-Built. All key metrics, including user management, reported content, and AI detection logs, are consolidated into a single, accessible Admin dashboard.

Table 1.2: WorkSphere vs VivaEngage

1.5.4.2 WorkSpere vs. Slack

Aspect	Microsoft Viva Engage	WorkSphere
Core Philosophy & Purpose	Collaboration & Tactical Communication. Designed for fast-paced conversations in channels to get work done.	Community Building & Cultural Engagement. Designed for thoughtful communication and discussions that build a sense of belonging and preserve institutional knowledge.
Moderation Approach	Limited & Reactive. Relies on user reporting and basic keyword alerts for administrators. There is no built-in, proactive, AI-driven moderation for all content by default.	Proactive (Pre-emptive) & AI powered. Uses a specialized AI model to analyze and block inappropriate content before it is published, providing instant user feedback.
Administrator's Role	Manual Investigator. Admins react to user reports or keyword alerts.	Strategic Overseer. The system automates the blocking of clear violations. Admins use a dedicated dashboard to monitor and review logs.

Aspect	Microsoft Viva Engage	WorkSphere
Community Features	Tactical Social Elements. Features like custom emojis, status updates, and Huddles add social flair to a work-focused tool. They are a secondary function.	Strategic Community Infrastructure. The entire platform is built around community features: rich user profiles, a like/comment/reply system, and a dedicated space that encourages healthy communication and fosters a sense of belonging.

Table 1.3: WorkSphere vs Slack

1.6 Work methodology

1.6.1 Agile methods

There are numerous agile methods available, including the most popular methods:

1.6.1.1 Extreme Programming (XP)

Extreme Programming, or XP, is an agile project management method particularly well-suited for software development projects. The basic principle of the XP method is to closely involve all project participants and to opt for very short development iterations. Task planning is always highly flexible, and workload estimation is simplified through short-term forecasting. Consequently, the alignment between customer expectations and deliverables is ensured. Features are regularly provided for testing and verification via operational prototypes.

1.6.1.2 Feature Driven Development (FDD)

Feature-Driven Development (FDD) is a project management methodology that is based on risk management. Development occurs in short iterations focused on user-testable features. As a result, participating in development enables tracking of the project's progress and functional verification. It is not advisable to consider it solely as a programming method; instead, it emphasizes a function-driven approach.

1.6.1.3 Adaptive software development (ASD)

ASD, or Agile Software Development, is a rapid application development methodology. The principle involves automating and industrializing as many processes as possible. It utilizes modeling tools and establishes a software factory to automatically generate a maximum amount of computer code. Then, the software engineering workshop (SEW) allows developers to modify the resulting applications. Finally, the delivery factory automates the entire deployment process.

1.6.1.4 Scrum

The agile method Scrum [7], particularly dedicated to IT project management, borrows its name from rugby. The principle of Scrum is to be able to modify the project direction as progress unfolds. It is characterized by iterations (called sprints) and minimal formalism: roles (Product Owner, Scrum Master, team), timeboxes (daily scrum, sprint review, release planning, sprint planning, release burndown/burnup, sprint burndown/burnup). The figure 1.3 below illustrates the lifecycle of the Agile Scrum methodology.

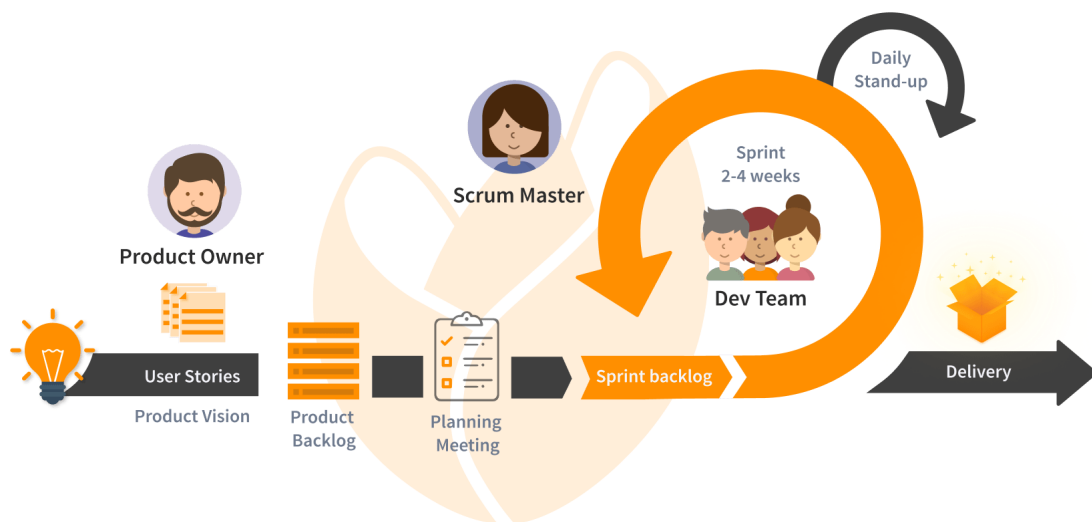


Figure 1.3: SCRUM Life Cycle

1.6.2 The chosen methodology

In our project, we are trying to adapt Scrum to our needs. Therefore, we will be using all of the following best practices

- Extraction of functions to develop after completing the product task list;
- Divide the project into a series of sprints ;

- Break down each sprint into a set of tasks ;

1.7 Gantt Diagram

The theoretical Gantt chart defines the period of each phase:

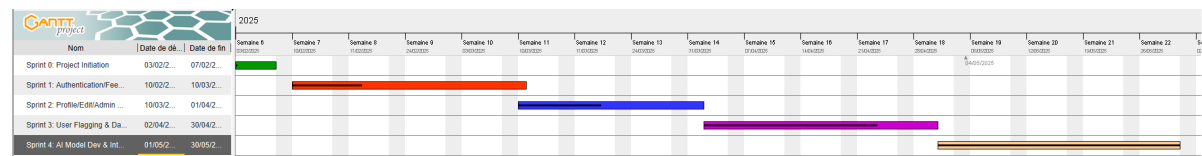


Figure 1.4: Gantt Diagram

Conclusion

This chapter has systematically established the foundational context and justification for the WorkSphere project. It began by highlighting the critical role of effective communication in modern corporate environments and the significant challenges posed by unmoderated Corporate Social Networks (CSNs), which can foster toxicity and hinder productivity. This laid the groundwork for the central problem statement: the need for a corporate communication platform that is both engaging and proactively safe. A critical analysis of existing solutions, including prominent platforms like Microsoft Viva Engage and Slack, revealed a significant gap in the market. While these tools excel at facilitating communication, their content moderation capabilities are often reactive, manual, or lack the sophistication required to handle the nuances of multilingual and context-dependent harmful content. The exploration of hate speech detection technologies further underscored the limitations of traditional rule-based and machine learning methods, pointing towards advanced Deep Learning models as the most promising path forward. In response to this identified gap, WorkSphere is proposed as an innovative solution. It distinguishes itself by integrating a state-of-the-art, AI-driven multilingual hate speech detection model at its core, aiming to create a digital workplace that is not only collaborative but also inherently respectful and secure.

Finally, the chapter outlined the strategic approach for bringing this vision to life.

Sprint 0: Project Initiation

Plan

2.1	Needs Framing	25
2.1.1	Actors Identification	25
2.1.2	Needs Analysis	25
2.1.3	Overall Use Case Diagram	27
2.2	Product backlog	28
2.3	Overall Application Architecture	30
2.4	Working environment	31
2.4.1	Hardware environment	31
2.4.2	Development environment	32
2.4.3	Software environment	33
2.4.4	DataBase Service	33

Introduction

This chapter outlines the initial phase of our implementation process, referred to as Sprint 0. We begin by identifying key user roles (actors) who will interact with the WorkSphere platform, followed by the functional and non-functional requirements, and then a definition of the main use case. The chapter then details the product backlog and sprint planning. We also describe the design of the application's architecture to ensure it meets the identified needs. Finally, we provide an overview of the core technologies, programming languages, and hardware components used to establish the development environment.

2.1 Needs Framing

2.1.1 Actors Identification

- **An Employee (Standard User):** The main user of the platform. The functional needs of this actor are:
 - Create/edit/delete posts.
 - Comment, reply, and react to content.
 - Receive notifications.
 - Edit profile information.
- **Administrator:** IT or platform administrator who manages access, user accounts, and configurations and is also responsible for moderation when needed. The functional needs of this actor are:
 - Dashboard for managing users, roles, and settings.
 - View flagged content and either remove or allow it.
 - Monitor the AI system's actions and logs.

2.1.2 Needs Analysis

2.1.2.1 Functional requirements

- **User Management:**
 - Register/login.

- Edit profile.
- Search for other users.
- Admins can delete users or change roles (user/admin).
- **Posts & Feed:**
 - Create, edit, and delete posts.
 - View posts in a feed.
 - Like/unlike posts.
- **Comments & Replies:**
 - Add or delete comments on posts.
 - Reply to comments (nested replies).
 - Like/unlike comments and replies.
- **Notifications:**
 - Get alerts for likes, comments, replies
 - Mark notifications as read
- **Reporting & flagging content:**
 - Users can report posts/comments/replies for hate speech or offensive content.
 - Admins can see reported content and can delete or allow it.
- **AI-Powered Hate Speech Detection:**
 - The AI model will automatically flag hate speech, offensive comments, and workplace harassment, and prevent it.
 - AI model will generate reports and provide valuable data to the admin.
- **Admin Dashboard:**
 - View all users, delete users.
 - View reported content and take action.
 - See stats (active users, reports, etc.)
 - Monitor actions taken by the AI model as well as its generated reports.

2.1.2.2 Non-functional requirements

- **Performance:**

- The platform should handle a large number of concurrent users with minimal latency.
- The AI model should process and flag content in real-time.

- **Security:**

- Implement secure authentication and authorization mechanisms.
- Ensure data privacy and security for all users.

- **Usability:**

- Simple and appealing UI.
- Light/dark mode.

2.1.3 Overall Use Case Diagram

We started the implementation of the project by constructing the overall use case diagram that would define and showcase the system's functionality for the length of this project. Figure 2.1 illustrates all the basic use cases of the application.

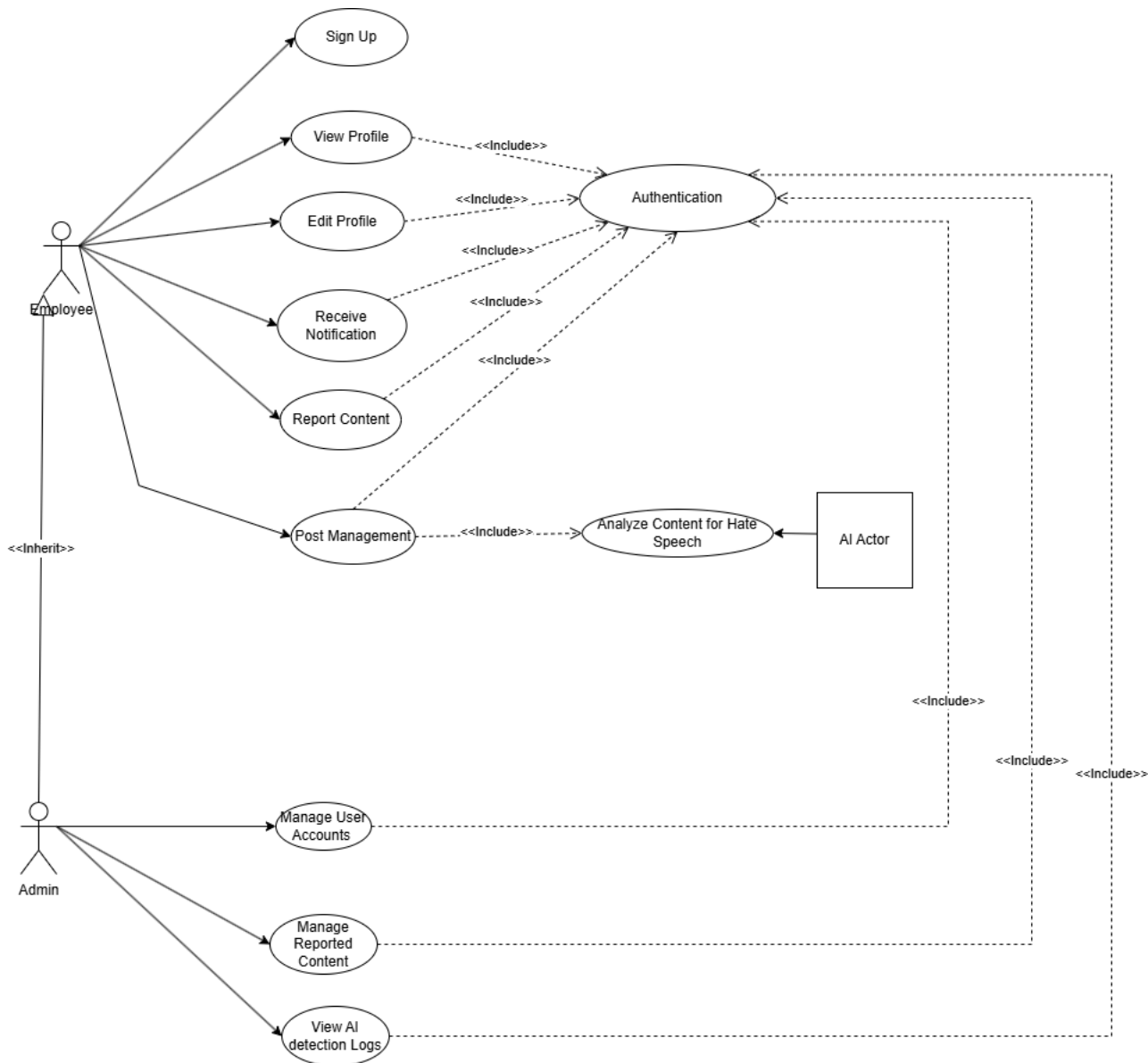


Figure 2.1: General Use Case Diagram

2.2 Product backlog

At the start of each sprint, the product backlog was employed as a dynamic, ordered list of work items derived from the project’s vision, goals, and requirements, to serve as the single source of truth for all features and improvements that may be added to the product, and to ensure that development is aligned with user and business needs.

The table 2.1 presents the initial product backlog and identifies the various « User Stories »

- **ID Sprint:** The table presents the number of each sprint ;

- **User Story:** that describes the functions of the application and the user of the company ;
- **Priority:** that mentions the priority level of development for « User Stories » according to user needs.

ID Sprint	Theme	Actor	User Story	Priority
N°1	Authentication & Security	Employee	As a user, I want to register and log in with my credentials securely, and I want to maintain secure sessions.	High
	Post Management	Employee	As an employee, I want a social feed feature where I can create, view, and interact with posts, so that I can better share information and engage with my colleagues across the company.	High
	Notifications	Employee	As an employee, I want to receive notifications when someone interacts with my posts and I want to see a notification badge so I know when I have new alerts.	High
N°2	Profile Management	Employee	As an employee, I want to be able to manage my profile and to view colleagues' profiles.	Medium
	Admin Dashboard	Admin	As an admin, I need a suite of governance tools to manage content moderation, user accounts, and AI performance.	High
N°3	Content Flagging	Employee	As an employee, I want to be able to report inappropriate content if encountered so that the platform remains free of toxicity and hate speech.	High

	Data Curation	Developer	As the developer, I want to collect data to make a robust multilingual corpus to be used to fine-tune and train the multilingual hate speech detection model.	High
N°4	AI Model Testing	Developer	As the developer, I want to evaluate and validate the model on sample test posts so we ensure real-world accuracy.	High
	Multilingual Support	AI System	As a system, I want to detect hate speech in posts posted in English, French, and Arabic and correctly classify them.	High

Table 2.1: Project backlog

2.3 Overall Application Architecture

The figure 2.2 depicts the overall architecture of the application, which employs a three-tier MERN stack architecture:

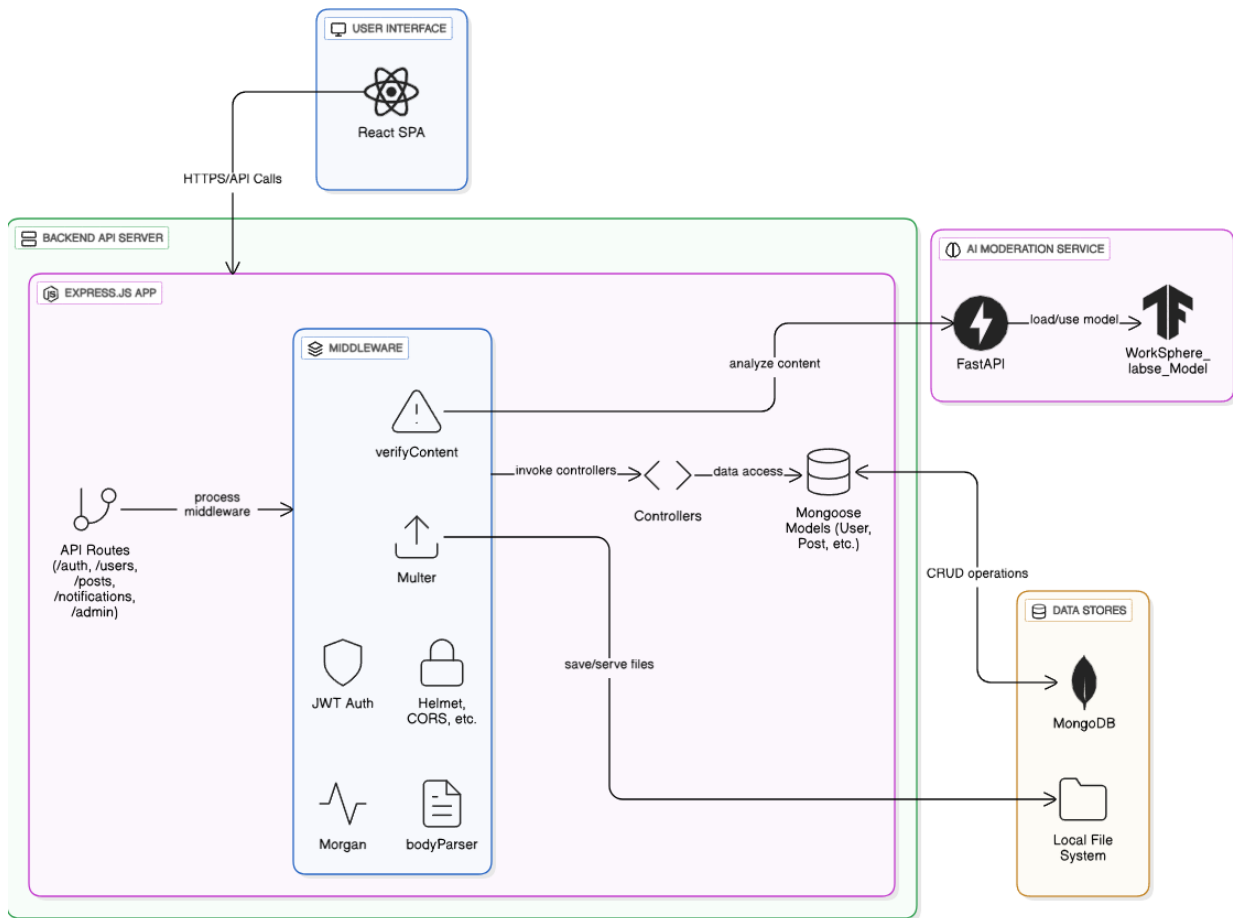


Figure 2.2: Overall Application Architecture

- The client-side Single Page Application (SPA), built with React, directly interfaces with the backend Express.js server via HTTPS/API calls.
- The backend processes requests through a defined middleware chain—including authentication, file handling, and AI-powered content verification, before invoking controllers for business logic execution.
- Data persistence is managed through MongoDB and a local file system.
- The integrated AI Moderation Service, a FastAPI application, analyzes content for policy adherence by leveraging a pre-trained Transformer model.

2.4 Working environment

2.4.1 Hardware environment

To carry out this project, we used a laptop with the following configuration:

- ASUS: Intel Core i5 @ 2.50 GHz, Ram: 8,00 Go ;

— Operating system: Windows 10.

for the AI model development, we used Google Colab Pro's subscription resources.

2.4.2 Development environment

—  **Node.JS** : Node.js is an open-source, cross-platform JavaScript runtime environment that executes JavaScript code outside of a web browser. Built on Chrome's V8 JavaScript engine, Node.js enables developers to build scalable and efficient network applications using a non-blocking, event-driven architecture. It is particularly well-suited for developing data-intensive real-time applications across distributed devices. [13].

—  **Express.JS** : Express.js is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications. It simplifies the process of building server-side applications by offering powerful middleware and routing capabilities, enabling developers to create RESTful APIs and dynamic web applications efficiently.[10].

—  **Postman** : Postman is a popular API client that facilitates testing and development of APIs. It provides a user-friendly interface for sending requests, inspecting responses, and debugging APIs during development [6].

—  **React vite** : React + Vite refers to the integration of the React JavaScript library with the Vite build tool to create modern, high-performance web applications. Vite serves as a development server and build tool that offers significantly faster startup times and hot module replacement (HMR) compared to traditional bundlers. When used with React, Vite provides a minimal and efficient setup by leveraging native ES modules in the browser and optimizing the build process with Rollup. [14].

—  **Google Colab Pro** : Google Colab Pro is a cloud-based development environment that provides users with enhanced computational resources (like faster GPUs, longer runtimes, and more memory) compared to the free version of Google Colab. It allows users to write and execute Python code in a Jupyter Notebook interface, making it ideal for prototyping, developing, and testing models, especially those involving large datasets or neural networks. [11].

2.4.3 Software environment

—  **Visual Studio Code:** It is a source code editor developed by Microsoft for Windows, macOS, and Linux. It includes built-in support for JavaScript, TypeScript, and Node.js. Used for the Frontend part [8].

—  **Overleaf:** "It is a free online platform for editing text in LATEX without the need for any application download. Additionally, it offers the possibility to write documents collaboratively and submit them directly to various publishers (IEEE Journal, Springer, etc.) [5].

2.4.4 DataBase Service

—  **MongoDB Atlas:** MongoDB Atlas is a fully managed cloud database service provided by MongoDB Inc. It offers an automated, scalable, and secure environment for deploying MongoDB databases on major cloud providers like AWS, Google Cloud, and Azure. With built-in tools for performance monitoring, data backup, and global distribution, Atlas simplifies database operations, allowing developers to focus on building applications rather than managing infrastructure.[12].

Conclusion

In summary, Sprint 0 has successfully laid the essential groundwork for the WorkSphere project. Through the identification of key user roles and a detailed analysis of requirements, a clear vision for the platform was established. This vision has been translated into an actionable product backlog and a scalable MERN and AI-based system architecture. With the development environment fully configured, all preparatory tasks are complete. The project now transitions from planning to execution, poised to begin the implementation phase, starting with Sprint 1.

Chapter

3

Sprint 1: Authentication, Feed and Notifications

Plan

3.1	Sprint 1 Backlog	35
3.2	Functional specification	36
3.2.1	Refined Use Case Diagram « Sprint 1 »	37
3.2.2	Description of refined use case « Authentication »	38
3.2.3	Description of refined use case « Create Content»	40
3.2.4	Description of refined use case « Like Content»	41
3.2.5	Description of refined use case « Edit Content»	42
3.2.6	Description of refined use case « Delete Content»	43
3.2.7	Description of refined use case « Notifications System»	44
3.2.8	Logical Flow and Structure« Feed »	46
3.2.9	Logical Flow and Structure«Notifications»	47
3.3	Design	48
3.3.1	Sequence diagram « Authentication »	48
3.3.2	Sequence diagram « Content Creation »	50
3.3.3	Sequence diagram « Like Content »	51
3.3.4	Sequence diagram « Notifications »	52
3.3.5	Class Diagram	53
3.4	Implementation	54

Introduction

In this chapter, we will present the first sprint, which will cover the authentication and security setup, the feed page, and the notifications implementation. The chapter will involve and discuss both the back-end and front-end.

3.1 Sprint 1 Backlog

The table 3.1 The table below details the sprint 1 backlog. Sprint id N1 represents the Authentication and Security, Sprint N2 the Feed and Post Management, and Sprint N3 the notifications implementation.

ID Sprint	Theme	Actor	User Story	Priority
N°1	Authentication	User	As a user, I want to register with email/password so I can access the platform.	High
			As a user, I want to log in with JWT so I can maintain a secure session.	High
		System	As a system, I want to hash passwords so they're never stored in plaintext.	Critical
			As a system, I want to protect routes with JWT middleware so only logged-in users can access them.	High
	Post management	User	As a user, I want to create posts with text and images so I can share updates with colleagues.	High
			As a user, I want to view a feed of posts from all users so I can stay updated.	High
			As a user, I want to edit or delete my own posts so I can manage my content.	Medium
		System	As a system, I want to store posts with user references so content is properly attributed.	High

			As a system, I want to handle nested comments and replies so users can have conversations and back-and-forth discussions.	High
			As a system, I want to track post likes using efficient data structures..	Medium
	Notification	User	As a user, I want to receive notifications when someone interacts with my content.	High
			As a user, I want to see a count of unread notifications so I know when there's new activity.	Medium
			As a user, I want to mark notifications as read so I can manage my alerts.	Medium
			As a user, I want to have simple and quick access to the posts referenced in notifications	Medium
		System	As a system, I want to generate notifications for likes, comments, and replies.	High
			As a system, I want to track notification read status so users know what's new.	Medium
			As a system, I want to fetch notifications efficiently when the dropdown is opened.	Medium

Table 3.1: Sprint 1 Backlog

3.2 Functional specification

In this section, we will present the requirements specification part of our project «**Sprint 1**» to describe the expected outcomes in terms of functionalities.

3.2.1 Refined Use Case Diagram « Sprint 1 »

The primary objective of Sprint 1 was to implement three core components of WorkSphere. First, we established a secure authentication system using JWT tokens and bcrypt password hashing, with role-based access control for admin privileges. Second, we developed the main feed functionality, including post creation with image uploads, a paginated feed view, and interactive features like post liking and nested comment threads. Finally, we implemented a real-time notification system that alerts users about interactions with their content, complete with unread status tracking and mark-as-read functionality.

The figure 3.1 describes the global use case diagram of the first sprint.

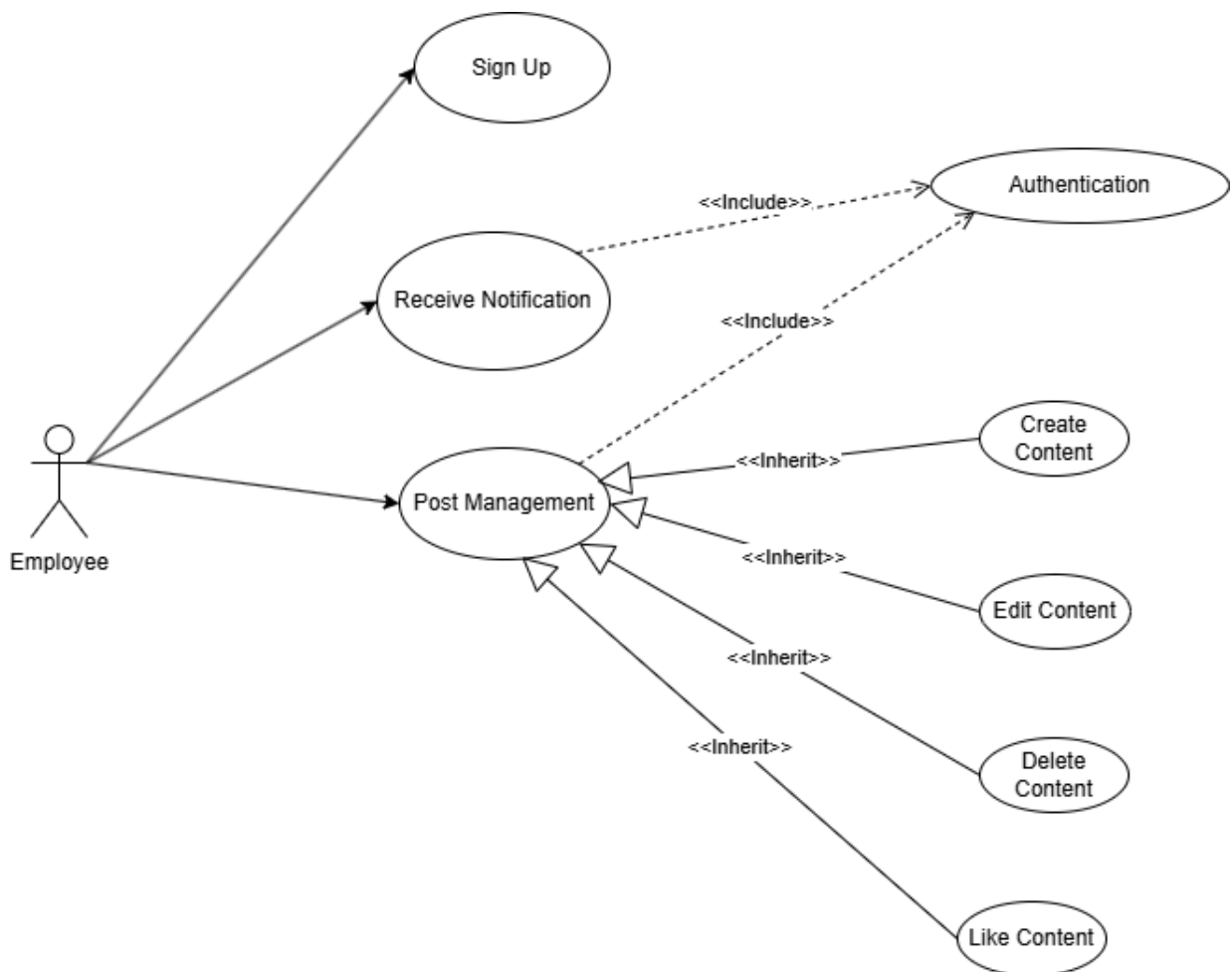


Figure 3.1: Use case diagram "Sprint 1"

— The user model was designed using Mongoose to define the schema for employee accounts in MongoDB to ensure data integrity while allowing flexibility for future enhancements in user profiles and permissions. Key fields included:

- Required fields (firstName, lastName, email, password) with validation.

- Security fields (role with enum ["user", "admin"], isAdmin boolean flag for quick privilege checks).
- Profile fields (picturePath for profile images, location, and department for organizational structure).
- Timestamps to track account creation and updates.

— Admins aren't required to perform any unique authentication steps (such as a special 'Admin Login'). They log in like normal users, and the system checks their role later. The isAdmin flag is just a property in the JWT token.

— The Post model was designed with nested comment/reply structures using Mongoose subdocuments, featuring:

- Multi-media support (text posts with optional image uploads via Multer)
- Interaction tracking.
- User references (preserving creator info even if accounts are deleted).
- Timestamps for chronological sorting and activity monitoring.

— We implemented the notification system as a separate Mongoose model linked to user activities, with:

- Real-time triggers for likes, comments, and replies.
- Status flags (isRead boolean with mark-as-read API endpoint).
- Real-time triggers for likes, comments, and replies.

3.2.2 Description of refined use case « Authentication »

The table 3.2 provides a description of the use case.

Title	Authentication and Security.
Actor	The user (Employee) and The System (Backend).

Summary	Sprint 1 implemented core authentication for employees, enabling secure registration and login using JWT tokens and password hashing (bcrypt). Users (employees) can sign up with profile pictures (via Multer) and log in to receive a token containing their user ID and isAdmin status.
Sequence Description	
Preconditions	• No existing account with same email • Valid email/password format for registration • User exists in database for login • Correct credentials provided for login • Valid JWT provided for token verification
Postconditions	• New user created in database (registration) • Password hashed with bcrypt (registration) • JWT token generated (login) • Token contains userId and isAdmin status (login) • System grants access to protected routes (token verification) • Decoded user data attached to request (token verification)
Nominal scenario	

	• User navigates to registration page. • Enters valid email, password (meeting complexity rules), and optional profile picture. • System creates account, hashes password with bcrypt, and stores user data. • User receives JWT token containing <code>userId</code> and <code>isAdmin</code> status. • User logs in with valid credentials and gains access to protected routes.
Alternative scenario	
	• Registration fails if: – Email already exists (system returns "Email already registered") – Skipping required fields or using an invalid format. • Login fails if: – Email doesn't exist (system returns "User not found") – Password is incorrect (system returns "Invalid credentials") • Token rejection if: – JWT is expired/missing (system returns "Unauthorized")

Table 3.2: Use Case Description "Authentication"

3.2.3 Description of refined use case « Create Content »

The table 3.3 provides a detailed description of this use case.

Title	Create Content.
Actor	The user (Employee).
Summary	Allows users to create content, which includes posts, comments, and replies.
Sequence Description	
Preconditions	• User must be authenticated (valid JWT token). • No hate speech or harmful contents.
Postconditions	• New content appears in feeds. • Like/comment counts update in real-time.
Nominal scenario	
	• User opens the feed page. • System fetches paginated posts (sorted by timestamp). • User creates a post. • System validates content, stores post in DB. • Other users interact with posts. • User edits/deletes their own posts
Alternative scenario	
	• Invalid post content (empty/too long/Hate Speech). • Invalid token.

Table 3.3: Use Case Description "Create Content"

3.2.4 Description of refined use case « Like Content»

The table 3.4 provides a detailed description of this use case.

Title	Like Content.
--------------	---------------

Actor	The user (Employee).
Summary	Allows users to interact with content through liking.
Sequence Description	
Preconditions	• User must be authenticated (valid JWT token). • Present content in feed.
Postconditions	• Like/comment counts update in real-time for the content we interacted with.
Nominal scenario	
	• User opens the feed page. • System fetches paginated posts (sorted by timestamp). • The user likes the content present in the feed.
Alternative scenario	
	• Invalid token.

Table 3.4: Use Case Description "Like Content"

3.2.5 Description of refined use case « Edit Content »

The table 3.5 provides a detailed description of this use case.

Title	Edit Content.
Actor	The user (Employee).
Summary	Allows users to edit their posts.
Sequence Description	
Preconditions	• User must be authenticated (valid JWT token). • Present user post in feed. • No hate speech or harmful contents.

Postconditions	• New edited post register in feed.
Nominal scenario	
	• User opens the feed page. • System fetches paginated posts (sorted by timestamp). • User edits their posts.
Alternative scenario	
	• Invalid token. • The user doesn't have any posts.

Table 3.5: Use Case Description "EditContent"

3.2.6 Description of refined use case « Delete Content»

The table 3.6 provides a detailed description of this use case.

Title	Like Content.
Actor	The user (Employee).
Summary	Allows users to interact with content through liking.
Sequence Description	
Preconditions	• User must be authenticated (valid JWT token). • Present content in feed.
Postconditions	• Like/comment counts update in real-time for the content we interacted with.
Nominal scenario	

	• User opens the feed page. • System fetches paginated posts (sorted by timestamp). • The user likes the content present in the feed.
Alternative scenario	
	• Invalid token.

Table 3.6: Use Case Description "Delete Content"

3.2.7 Description of refined use case « Notifications System»

The table 3.7 describes the use case.

Title	Notifications System.
Actor	The user (Employee).
Summary	The notification system alerts users about interactions with their content (likes/comments/replies) through real-time in-app alerts. Built using a dedicated Mongoose model, it triggers notifications from post controllers and displays them via a dropdown interface with unread status tracking.
Sequence Description	
Preconditions	• User must be logged in (valid JWT). • Notification triggers require a valid postId.
Postconditions	• Notifications persist. • Unread count updates in real-time. • Read state maintained.
Nominal scenario	

	• Trigger Event (e.g., comment on post) causes the system to create a notification. • When the recipient clicks the notifications' bell icon, notifications are fetched. • Viewed notifications are marked as read.
Alternative scenario	
	• In case of no new notifications, the dropdown notifications list displays "No new notifications". • In case of a notification concerning a post that has been deleted, Notifications will still show but link to a "Post not found" message.

Table 3.7: Use Case Description "Notifications System"

3.2.8 Logical Flow and Structure« Feed »

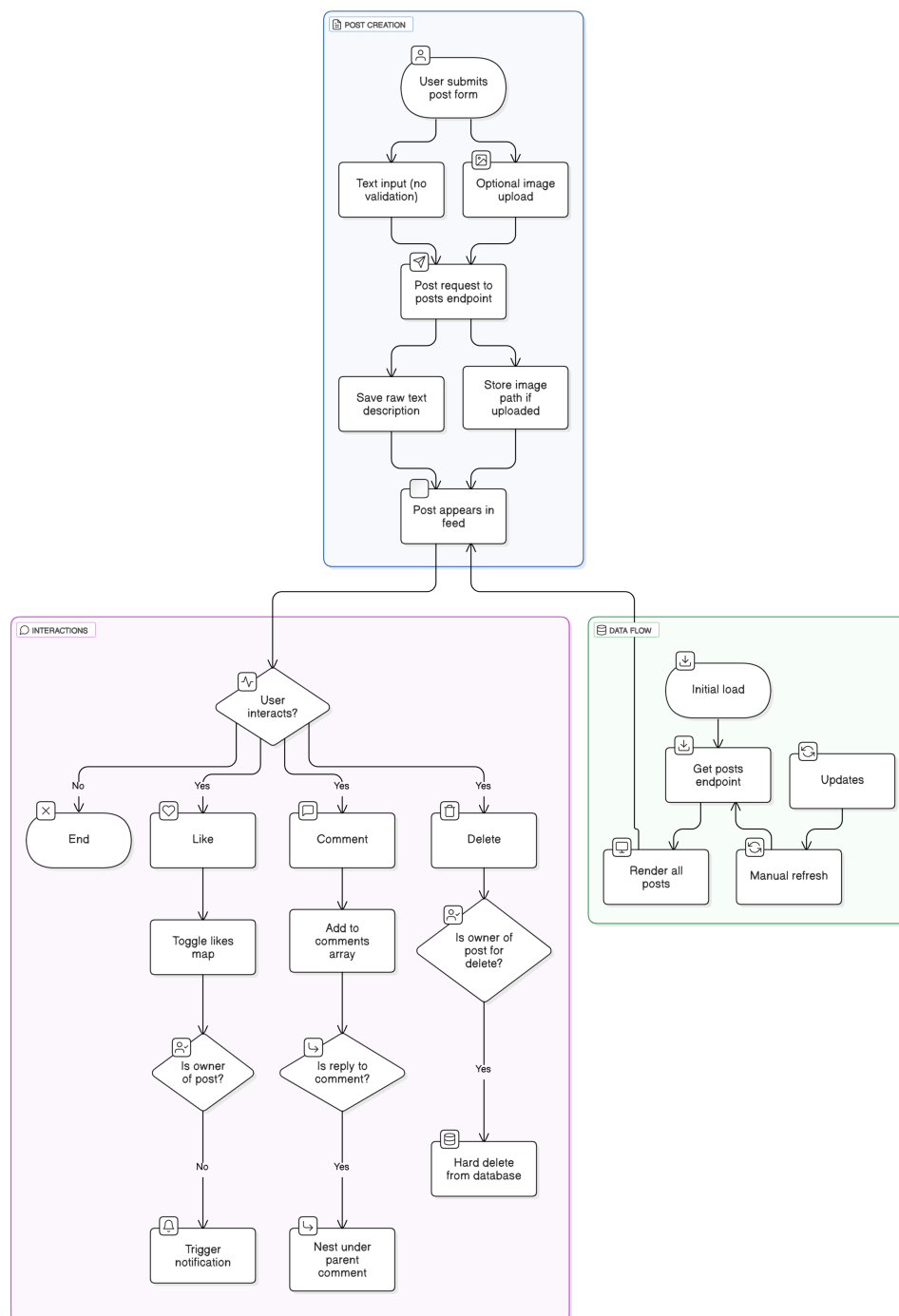


Figure 3.2: Flow Chart "Feed"

- The Feed component (Feed.jsx) fetches posts sorted by date via GET /posts, storing them in React state and re-fetching on updates triggered by a newPostCreated event.
- Each post is rendered via the Post component (Post.jsx), displaying user metadata (name, profile picture, timestamp), content (text/image), and owner-controlled edit /

delete options, alongside like / comment/reply functionalities.

— **F**or creation, the PostForm component (PostForm.jsx) submits text/images to POST /posts using multipart/form-data; successful uploads fire the newPostCreated event to refresh the feed dynamically.

3.2.9 Logical Flow and Structure«Notifications»

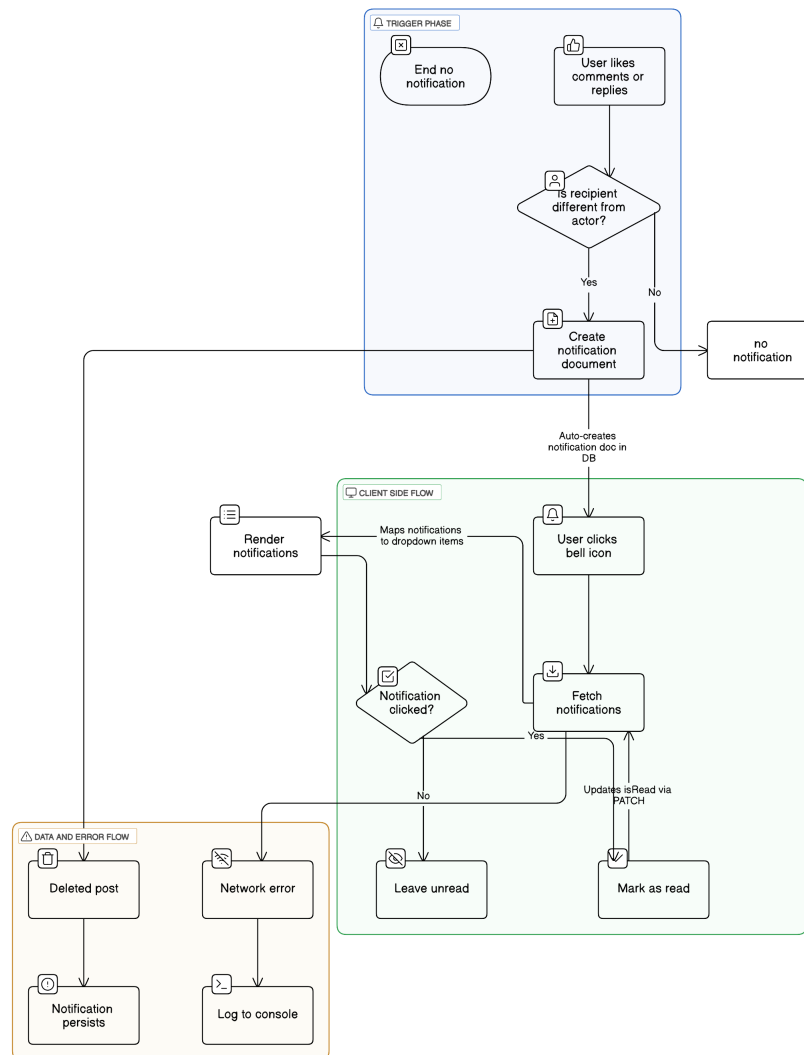


Figure 3.3: Flow Chart "Notifications"

The notifications system is set up to alert users when another user interacts with their posts, comments, or replies. Each notification contains:

- The receiver's user ID.
- The sender's following details: name, profile picture, and ID.

- The post or comment ID it's related to.
- A type specification (like, comment, reply).
- The notification's message that's to be displayed.
- An isRead flag that would help users keep track of already viewed notifications.

3.3 Design

In this section, we will present the design part by visually mapping out how different parts of a system interact over time and showcasing the order of operations between users, screens, backend components, and databases.

3.3.1 Sequence diagram « Authentication »

3.3.1.1 Sequence diagram «Sign up »

The figure 3.4 illustrates the Signup process implemented in Sprint 1.

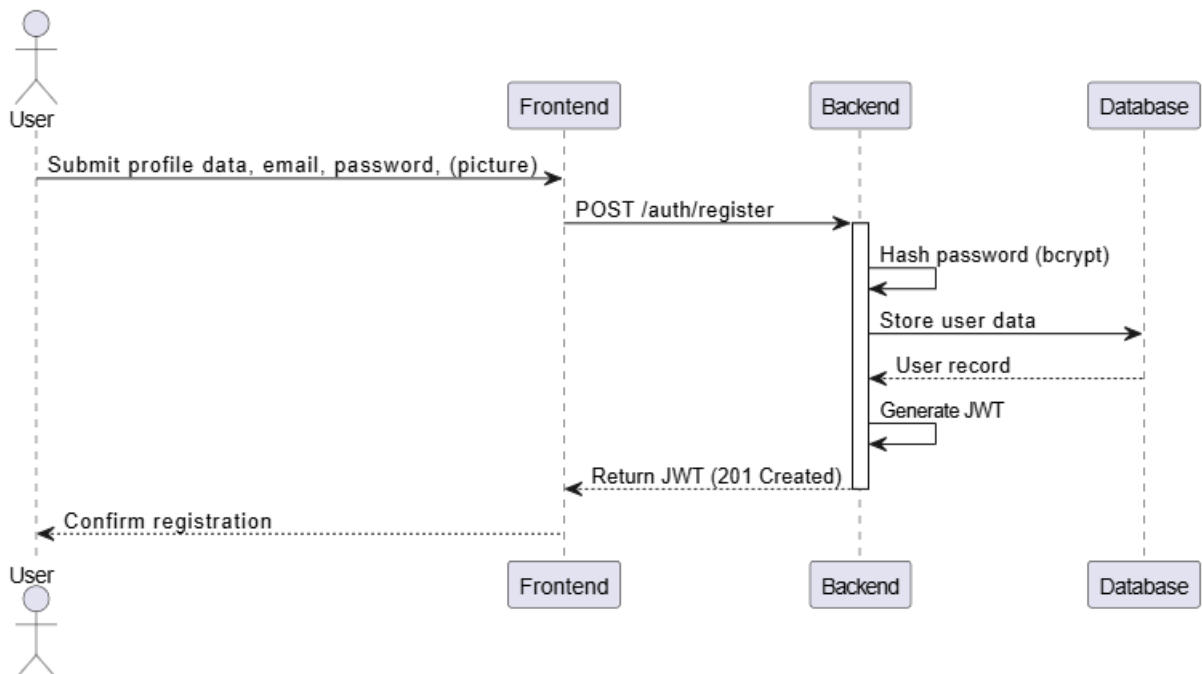


Figure 3.4: Sequence diagram "Signup"

A dual-mode authentication system was implemented to support both email/password and profile picture uploads during registration. Key aspects include:

- Users submit first name, last name, email, password, and other information via a POST request to `/auth/register`.
- Passwords are hashed using `bcrypt` with a 10-round salt, ensuring they are never stored in plaintext, preventing credential leaks and brute-force attacks. Additionally, passwords are never returned in API responses, even in hashed form, and password updates require current password verification before allowing changes.
- Profile pictures (if uploaded) are processed via `Multer` middleware, stored in an `uploads/` directory, and linked via a file path in the database. `Multer` is a middleware for `Express.js` designed specifically for handling multipart/form-data, which is the encoding type used for file uploads in web forms. Unlike the default `Express` body parsers (which only handle JSON or URL-encoded data), `Multer` processes file uploads efficiently and securely.

When the user submits their credential and requests to sign up, this request is sent to the server, where the password is hashed for security, their data is stored in the database, and their JWT token is generated.

3.3.1.2 Sequence diagram «Login »

The figure 3.5 illustrates the Login implemented in Sprint 1.

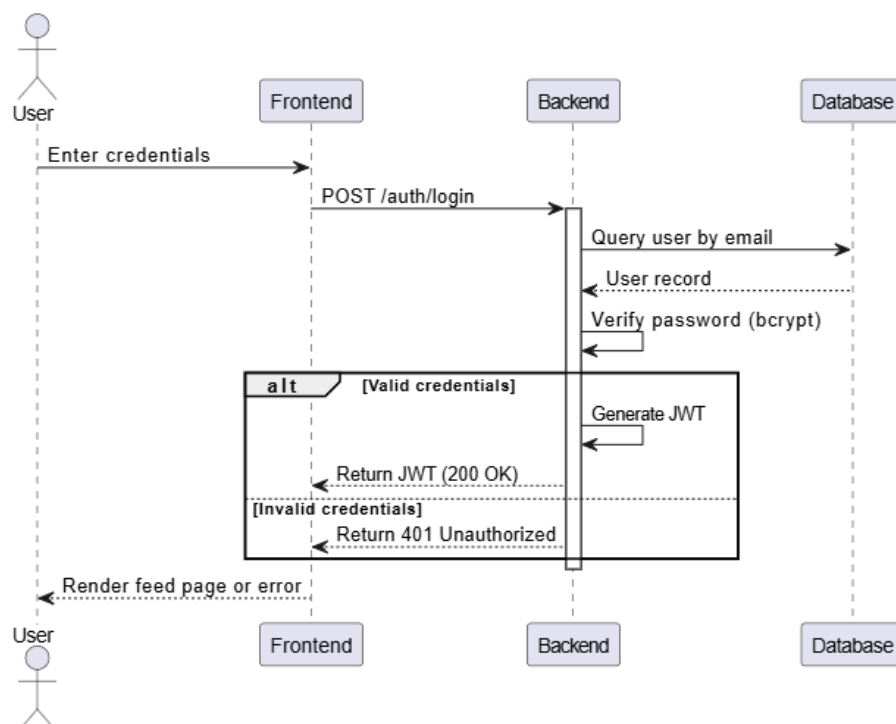


Figure 3.5: Sequence diagram "Login"

when a user logs in, a request is sent to the server, where it will be processed, and their credentials will be checked and verified. If valid, it would return a new JWT token (200 OK); otherwise, it would return 401 unauthorised. The token verification is implemented as a gatekeeper for your API endpoints. It checks if a provided token is authentic, valid, and unexpired, and ensures that only legitimate, authenticated users can access the different routes.

3.3.2 Sequence diagram « Content Creation »

The figure 3.6 illustrates the implementation of post creation in Sprint 1.

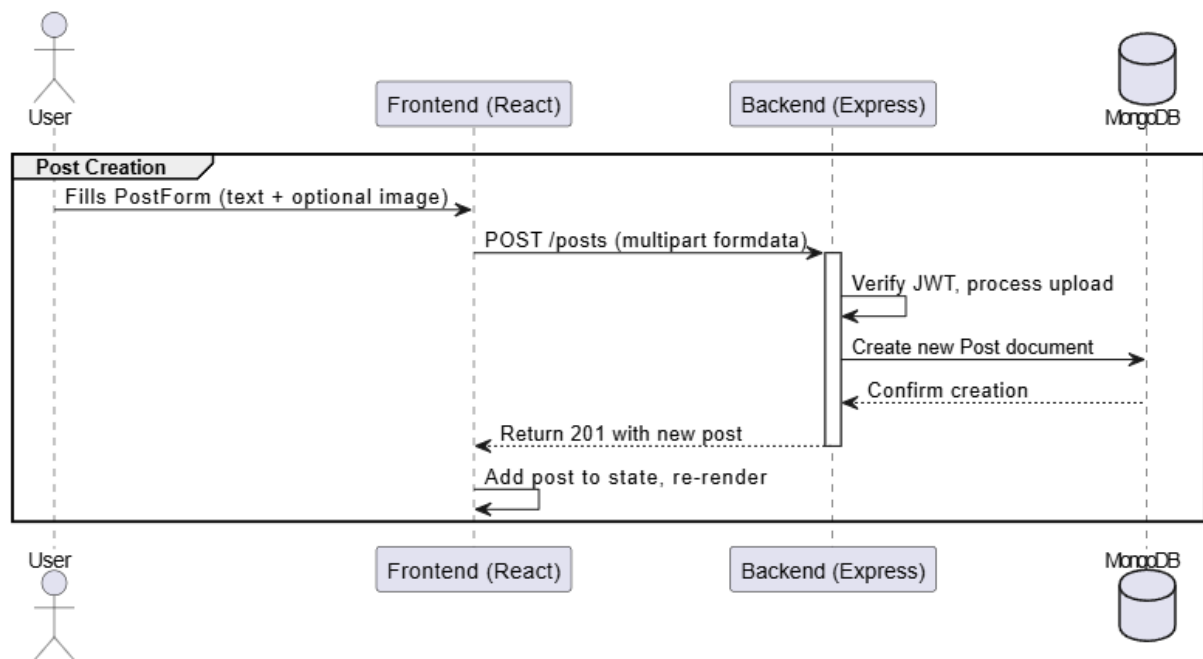


Figure 3.6: Sequence diagram "Post Creation"

The post creation process begins when the user submits a new post through the React frontend, providing text and an optional image. The frontend then sends this information as a POST request (using multipart/form-data for potential image uploads) to the Express backend. The backend first authenticates the user (via JWT) and processes the image upload if present, and then it instructs the MongoDB database to create and store a new post document. Upon success, the backend responds to the frontend with a 201 Created status and the data of the newly created post, and in turn, the front end updates its local state accordingly.

The figure 3.7 illustrates the implementation of comment creation in Sprint 1.

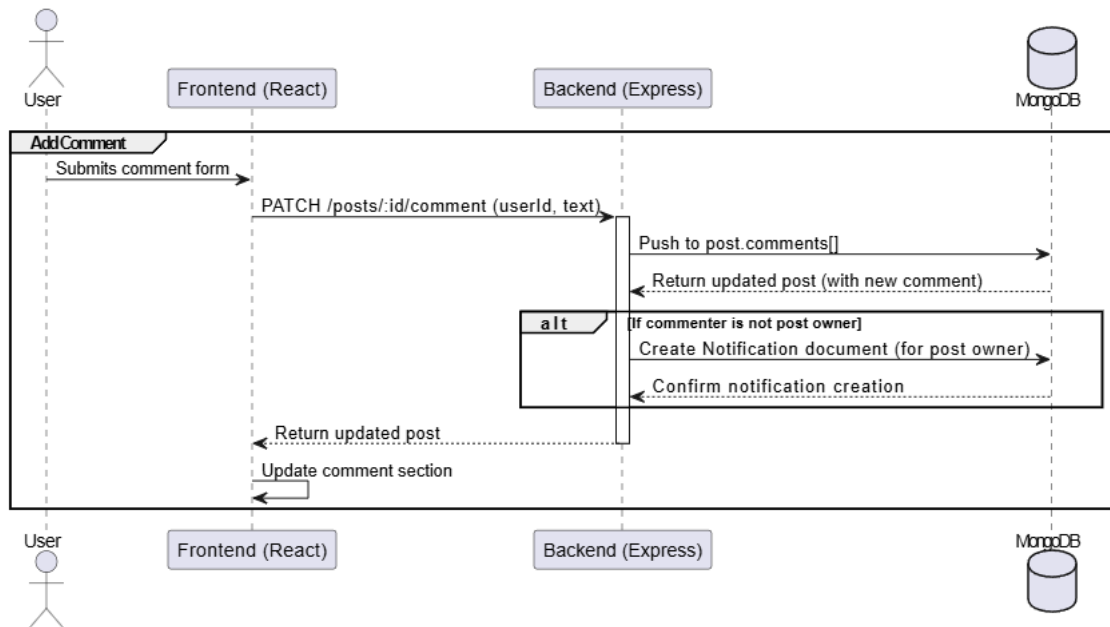


Figure 3.7: Sequence diagram "Comment Creation"

The workflow for adding a comment is initiated by a user submitting said comment via the React frontend, which then dispatches a PATCH request to the backend's `/posts/:id/comment` endpoint, carrying the `userId` and comment text. The Express backend controller then orchestrates two primary database interactions with MongoDB. first, it appends the new comment object to the `comments` array within the target `Post` document and awaits its persistence. Upon successful update of the post, and critically, if the commenter is not the post owner, a secondary operation creates a new `Notification` document in a dedicated collection, recording details like `senderId`, `postId`, and `type: "comment"`. Finally, the backend responds to the frontend with the updated post data

3.3.3 Sequence diagram « Like Content »

The figure 3.8 illustrates the implementation of the liking content functionality in Sprint 1.

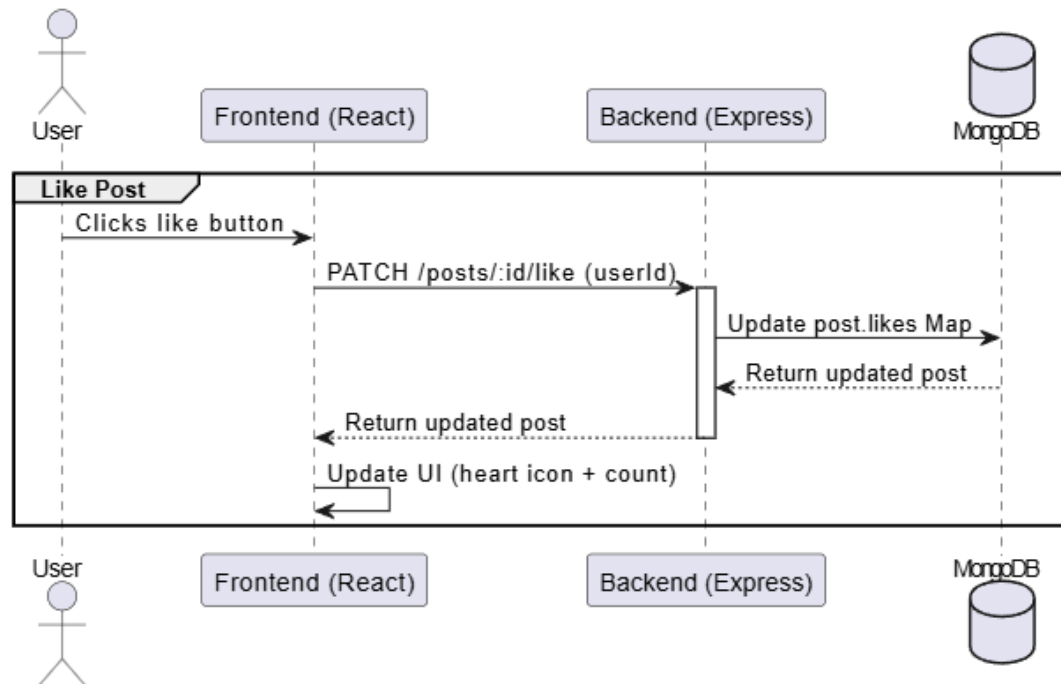


Figure 3.8: Sequence diagram "Like Content"

3.3.4 Sequence diagram « Notifications »

The figure 3.9 illustrates the implementation of notifications in Sprint 1.

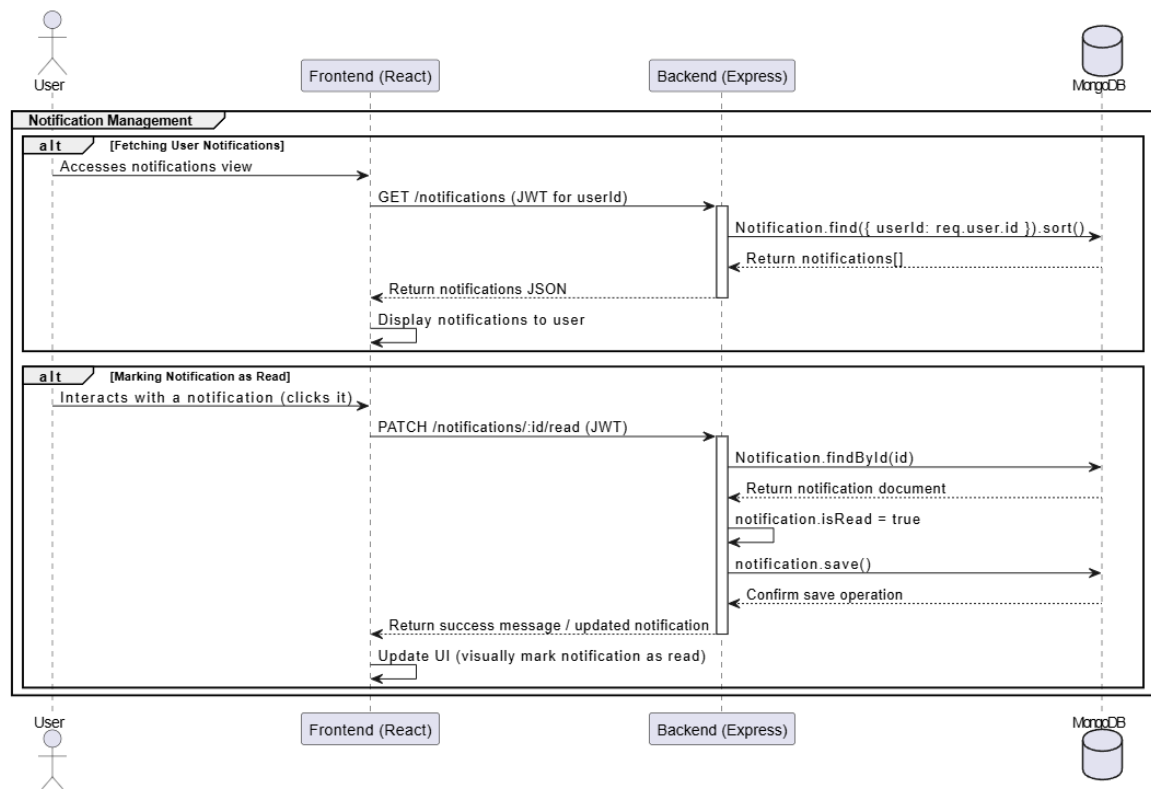


Figure 3.9: Sequence diagram "Notifications"

This diagram captures the application's notification workflow that's set in motion when a user clicks the bell icon on the navbar, therefore triggering an authenticated request to fetch unread items. If a user interacts to mark a specific notification as read, the frontend sends a PATCH request to `/notifications/:id/read`. The `markAsRead` controller retrieves the corresponding Notification document from MongoDB by its ID, sets its `isRead` property to true, and saves the change back to the database.

3.3.5 Class Diagram

The figure 3.10 represents the class diagram for the development of this Sprint.

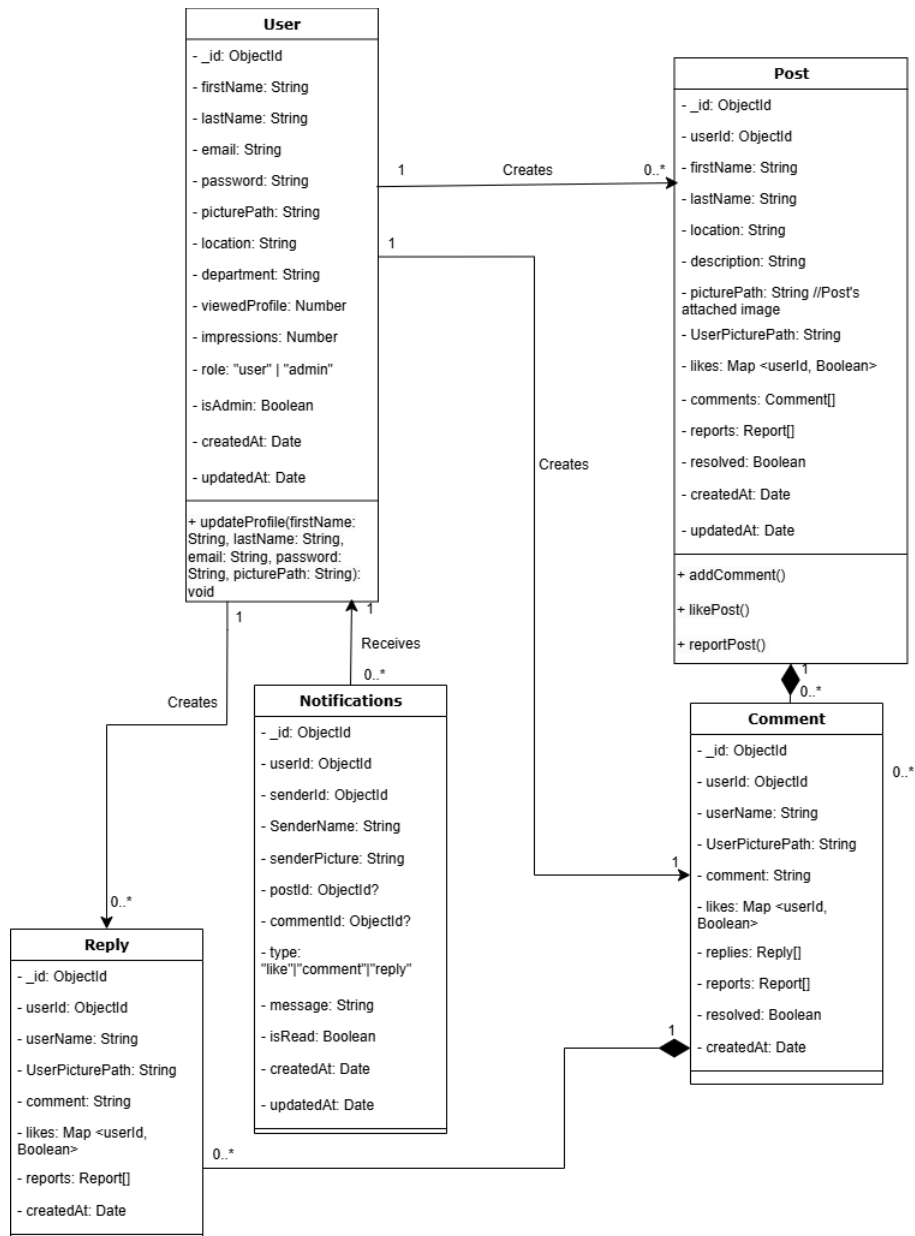


Figure 3.10: Class diagram for Sprint 1

The table 3.8 describes the participating classes in this sprint.

Class	Description
User	Represents the user (Employee) of the application's account, storing their profile details, authentication credentials, and role-based access information.
Post	Core entity representing user-generated content in the feed. It supports creation, modification, and deletion of posts.
Comment	Represents user comments on posts. Includes comment text, author information, likes, and support for nested replies.
Reply	Specialized form of a comment that serves as a direct response to another comment, enabling threaded discussions.
Notification	Tracks user interactions (e.g., posts, comments, replies) and delivers relevant alerts and updates.

Table 3.8: Participating Classes in Sprint 1

3.4 Implementation

In this section, we will present the interfaces related to this sprint.

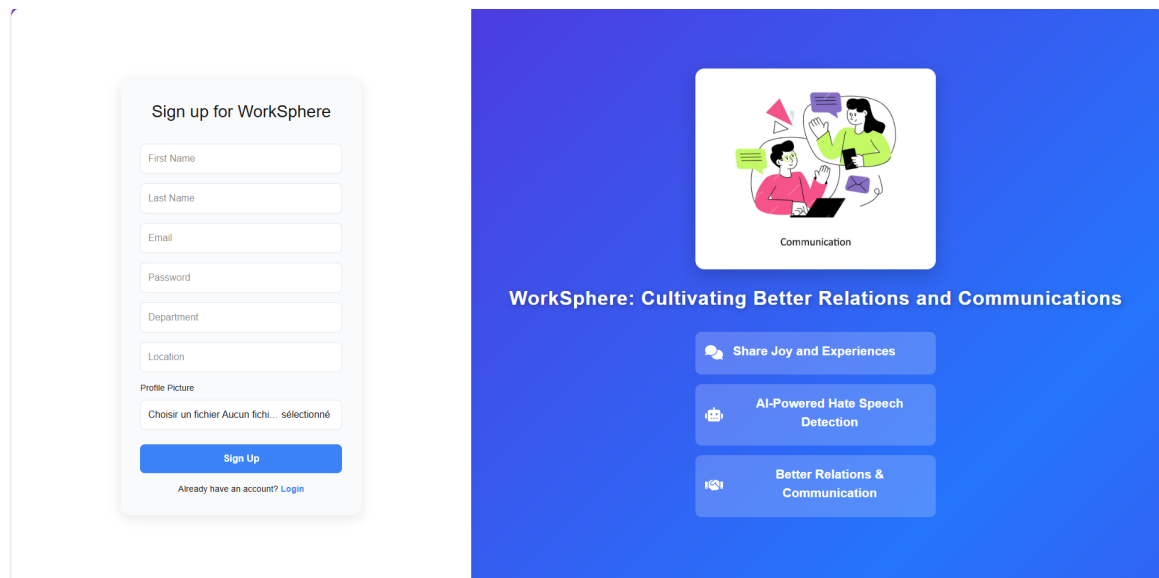


Figure 3.11: Sign Up Interface

We will start by presenting the "Signup" page as presented in the figure 3.11 above. To create a user account, the user needs to fill out the form and press the "Sign Up" button.

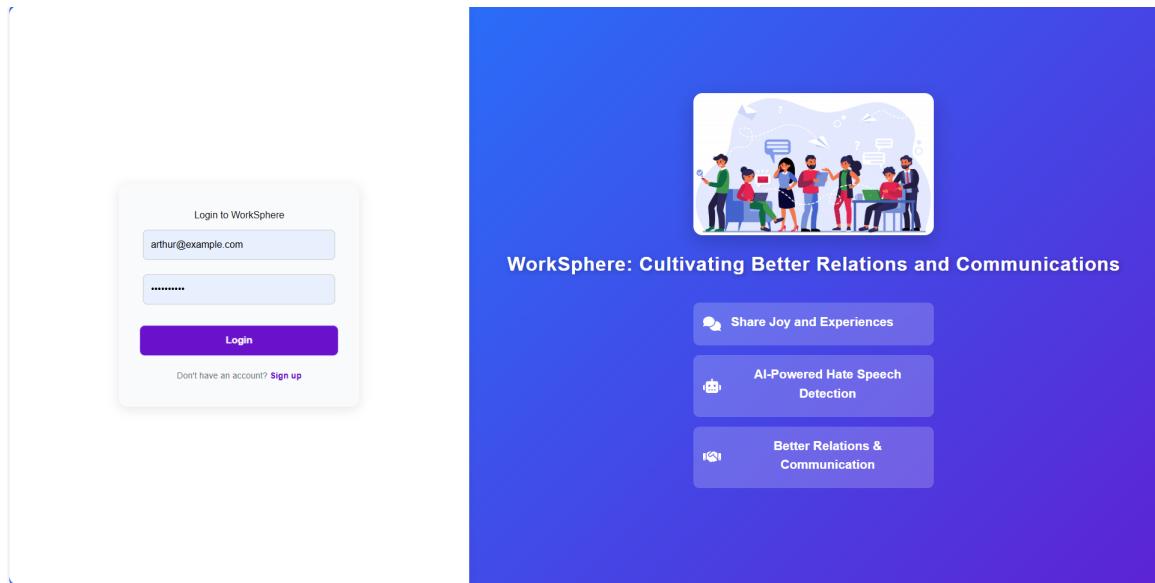


Figure 3.12: Login Page

To log in, the user must enter their email address and their password, then click on 'Log in'. The authentication interface is presented in the figure 3.12 above.

Once the user is authenticated, based on their role, they will be redirected to either the feed page or the admin dashboard page.

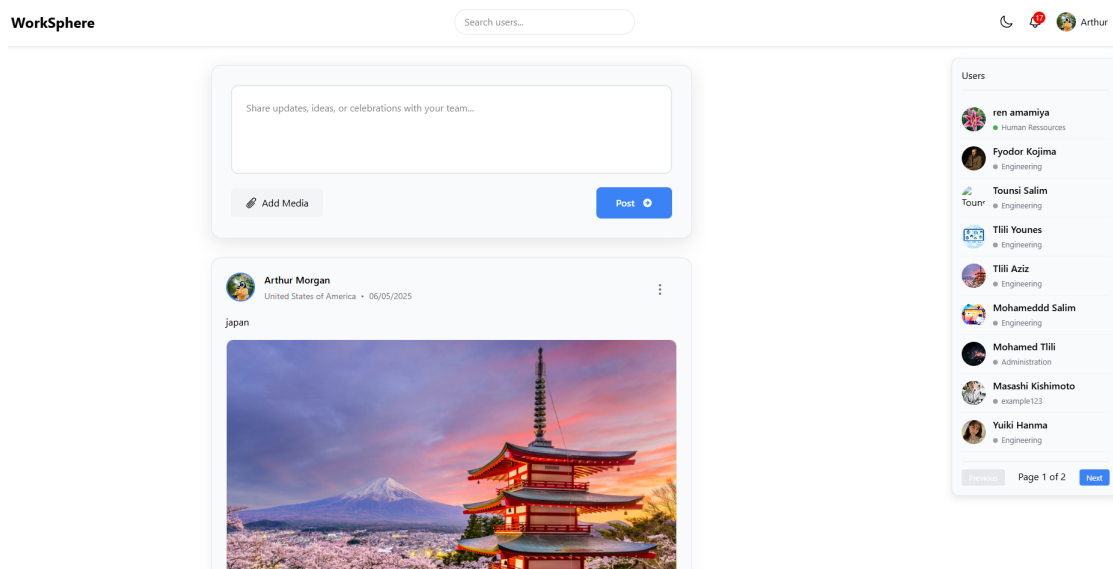


Figure 3.13: Feed Page

The Feed page is the central hub of the application where all employees would get to share their updates and whatever is on their minds, partake in discussions, and interact with each other. You will find the option to post something or view other employees' posts, react to them, comment, or reply to comments. Moreover, we added a sidebar in

the feed page that, using webSocket, provides real-time information concerning the active status of other users.

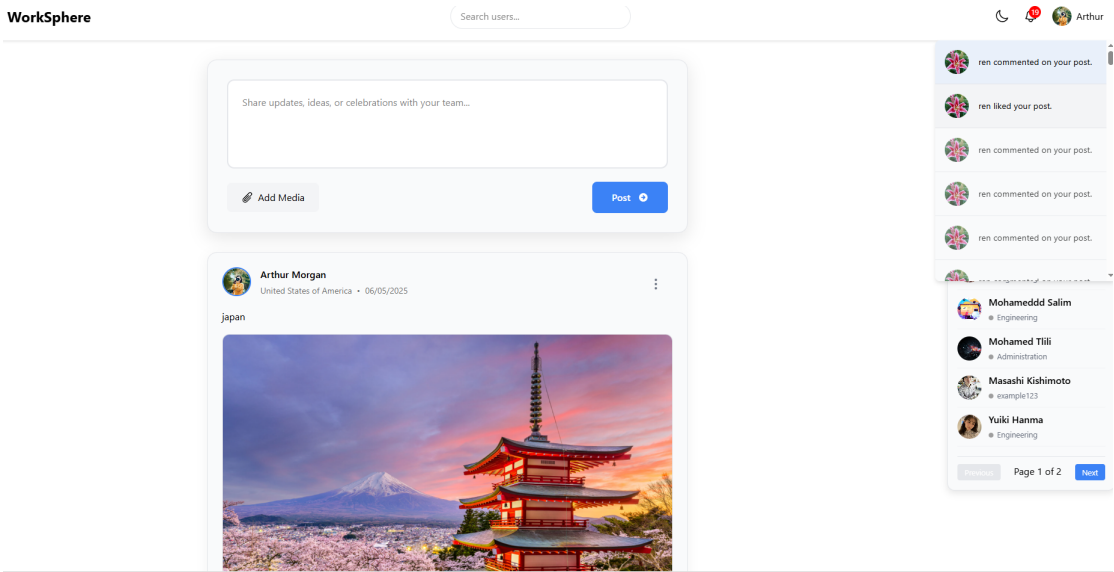


Figure 3.14: notifications display

The bell icon displays the count for unread notifications to alert employees of new interactions and ensure that they don't miss any interactions, and the format of any notification would include the usernames, profile pictures, and context.

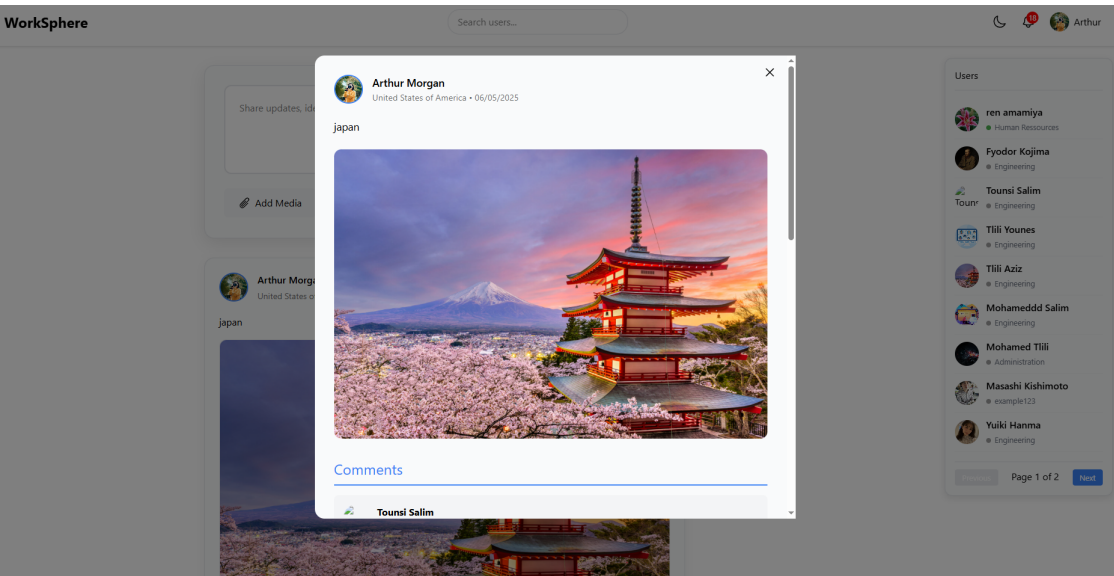


Figure 3.15: notifications display

To guarantee that users can quickly identify and respond when they click a certain notification, would bring a pop-up of the post in question with the comments and replies.

Conclusion

Sprint 1 successfully established the foundational architecture and core functionalities of the WorkSphere platform, transforming the project from a conceptual framework into a tangible, interactive application. We addressed authentication and security. The central content feed was brought to life, empowering users with the core abilities to create, view, edit, and delete posts. This establishes the primary channel for communication and information sharing within the platform. Furthermore, vital social interaction mechanics were integrated. The ability for users to like content introduces the first layer of engagement, while the notification system provides immediate feedback, alerting users to interactions with their content.

Sprint 2: Profile Page, Edit Account and Admin Dashboard

Plan

4.1	Sprint 2 Backlog	59
4.2	Functional specification	60
4.2.1	Refined use case diagram « Sprint 2 »	60
4.3	Design	67
4.3.1	Scenario	67
4.3.2	Class Diagram	69
4.4	Implementation	71

Introduction

In this chapter, we will present the second sprint where we will discuss and showcase the implementation of the Profile page, the Edit Account feature, and the Admin Dashboard. As with the previous sprint, this chapter will be organized into three main parts: Analysis, design, and implementation.

4.1 Sprint 2 Backlog

This sprint focused on enhancing user engagement and administrative control by implementing the profile and account editing pages, as well as the admin dashboard. The profile page and edit profile page allow users to subsequently view their posts and activities as well as other users' activities, and edit their personal information, while the admin dashboard provides administrators with tools to manage users, moderate content, and monitor platform activity. These features were prioritized to improve user autonomy and ensure platform integrity through robust administrative oversight.

The table 4.1 Presents the Sprint 2 backlog

ID Sprint	Theme	Actor	User Story	Priority
N°2	Profile Management	User	As a user, I want to view other users' profiles and activities as well as my profile and posts so that I can manage my presence on the platform.	high
			As a user, I want to edit my profile details (e.g., name, department, picture) so that my information remains up-to-date.	High
			As a user, I want to change my password to maintain account security.	medium

	Admin Dash- board	Admin	As an admin, I want to view all users and their roles to manage account and access levels.	high
			As an admin, I want to delete or suspend users to enforce platform policies.	high
			As an admin, I want to view reported posts/com-ments to moderate inappro- priate content.	high
			As an admin, I want to delete flagged content to maintain a respectful envi- ronment.	high
			As an admin, I want to view analytics (e.g., active users, reports) to monitor platform health.	medium

Table 4.1: Sprint 2 backlog

4.2 Functional specification

In this section, we will outline the functional requirements of our « **Sprint 2** » to describe the expected outcomes in terms of functionalities.

4.2.1 Refined use case diagram « **Sprint 2** »

The figure 4.1 illustrates the refinement of the use case for Sprint 2.

The Sprint 2 use case diagram effectively captures the functional requirements for profile management and administrative controls in the WorkSphere application. The diagram maintains a clear separation between regular user and administrator roles, reflecting the role-based access control implemented in the backend through JWT verification and ad-

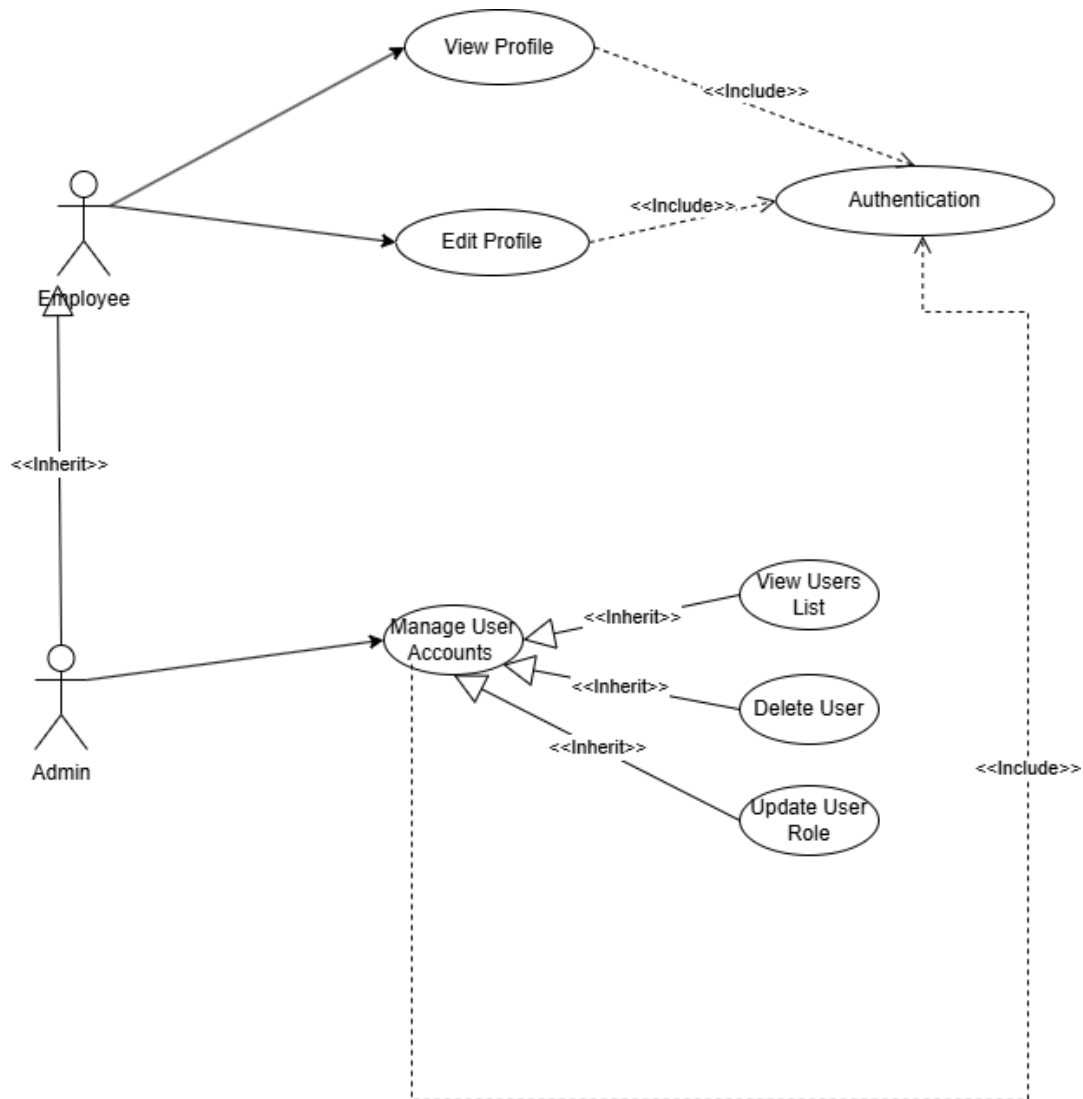


Figure 4.1: Refined use case diagram for Sprint 2

min middleware checks. This architecture ensures that sensitive administrative functions like content moderation and user management remain restricted to authorized personnel only, while still allowing all authenticated users to manage their profiles.

- For employees, the central capability introduced in this sprint is the ability to view their profile page and to edit their profile. The profile page allows users to keep track of their posts and activities on the platform. Furthermore, the edit profile page enables users to update their account's information as well as their security details.
- The administrative role introduced in this sprint allows admins to monitor application statistics and manage users and users' access.

4.2.1.1 Use case description « View Profile »

The table 4.2 provides a description of the use case « **View Profile** »

Title	View Profile
Actor	Authenticated User
Summary	Allows users to view their own and other users' profile information and posted content.
Sequence description	
Pre conditions	• User is logged in (valid JWT exists).
Post conditions	• Profile data is displayed. • User's posts are loaded in chronological order.
Nominal scenario	
	• User navigates to /profile/:userId. • System fetches user data via GET /users/:id (handled by getUser controller). • System fetches user's posts via GET /posts/:userId/posts. • Frontend (Profile.jsx) renders the data.
Alternative scenario	
	• User not found, system returns 404 error, and the Frontend displays "User not found" message.

Table 4.2: Use case description related to "View Profile"

4.2.1.2 Use case description « Edit Profile »

The table 4.3 Provides a description of the use case « **Edit Profile** »

Title	Edit Profile
--------------	--------------

Actor	Authenticated User
Summary	Allows users to modify their profile information, including personal details and security credentials.
Sequence description	
Pre conditions	• User is logged in (valid JWT exists) • User account exists in database.
Post conditions	• Database records are updated. • Changes are reflected across the application.
Nominal scenario	
	• User navigates to /edit-profile. • System loads current profile data via GET /users/:id. • User modifies fields and submits form. • System validates input and uploads image (if changed). • Backend processes PATCH /users/:id request (handled by updateUser controller). • Frontend (EditProfile.jsx) displays success message.
Alternative scenario	
	• Invalid Input, in which case the system rejects the submission and displays field-specific errors. • File Upload Failure, in which case the system maintains old profile picture and notifies user.

Table 4.3: Use case description related to "Edit Profile."

4.2.1.3 Use case description « View Users List »

The table 4.4 Provides a description of the use cases related to « View Users List »

Title	View Users List
Actor	Administrator
Summary	Allow admins to view all website users, admins, and non-admins.
Sequence description	
Pre conditions	• Valid admin JWT exists. • MongoDB collection (User) is accessible.
Post conditions	• Users list is displayed.
Nominal scenario	
	• Admin accesses admin dashboard upon logging in (verified by isAdmin middleware). • Frontend (AdminDashboard.jsx) fetches users via GET /admin/users (getAllUsers controller).
Alternative scenario	
	• Data Fetch Failure, in which case the frontend displays an error.

Table 4.4: Use case descriptions related to " View Users List "

4.2.1.4 Use case description « Delete Users »

The table 4.5 Provides a description of the use cases related to « **Delete Users** »

Title	Delete Users
Actor	Administrator
Summary	Provides administrators with the ability to delete users from the platform.
Sequence description	
Pre conditions	• Valid admin JWT exists. • Users list fetched correctly.
Post conditions	• Database reflects delete actions. • Analytics data is updated to display the new number of users and admins.
Nominal scenario	
	• Admin accesses /admin route (verified by isAdmin middleware). • Frontend (AdminDashboard.jsx) fetches users via GET /admin/users (getAllUsers controller). • Admin performs delete action. • Backend validates and executes via DELETE /admin/posts/:id.
Alternative scenario	
	• Data Fetch Failure, in which case the frontend displays an error instead of user list.

Table 4.5: Use case descriptions related to " Delete Users "

4.2.1.5 Use case description « Update User Role »

The table 4.6 Provides a description of the use cases related to « **Update User Role** »

Title	Update User Role
Actor	Administrator
Summary	Provides admins with the ability to update users roles (admin/regular user).
Sequence description	
Pre conditions	• Valid admin JWT exists. • Users list fetched correctly.
Post conditions	• Database reflects role update action. • Analytics data is updated to display the new number of users and admins.
Nominal scenario	
	• Admin accesses /admin route (verified by isAdmin middleware). • Frontend (AdminDashboard.jsx) fetches users via GET /admin/users (getAllUsers controller). • Admin updates roles. • Backend validates and executes via PATCH /admin/users/:id/role.
Alternative scenario	
	• Data Fetch Failure, in which case the frontend displays an error instead of the user list.

Table 4.6: Use case descriptions related to " Update User Role "

4.3 Design

In this section, we will present the scenarios that we implemented in this sprint 2.

4.3.1 Scenario

4.3.1.1 Sequence diagram « View Profile »

The figure 4.2 illustrates the sequence diagram 'View Profile', which will allow users to access their profile page to keep track of their presence and posts and other users' profile pages to view their activities.

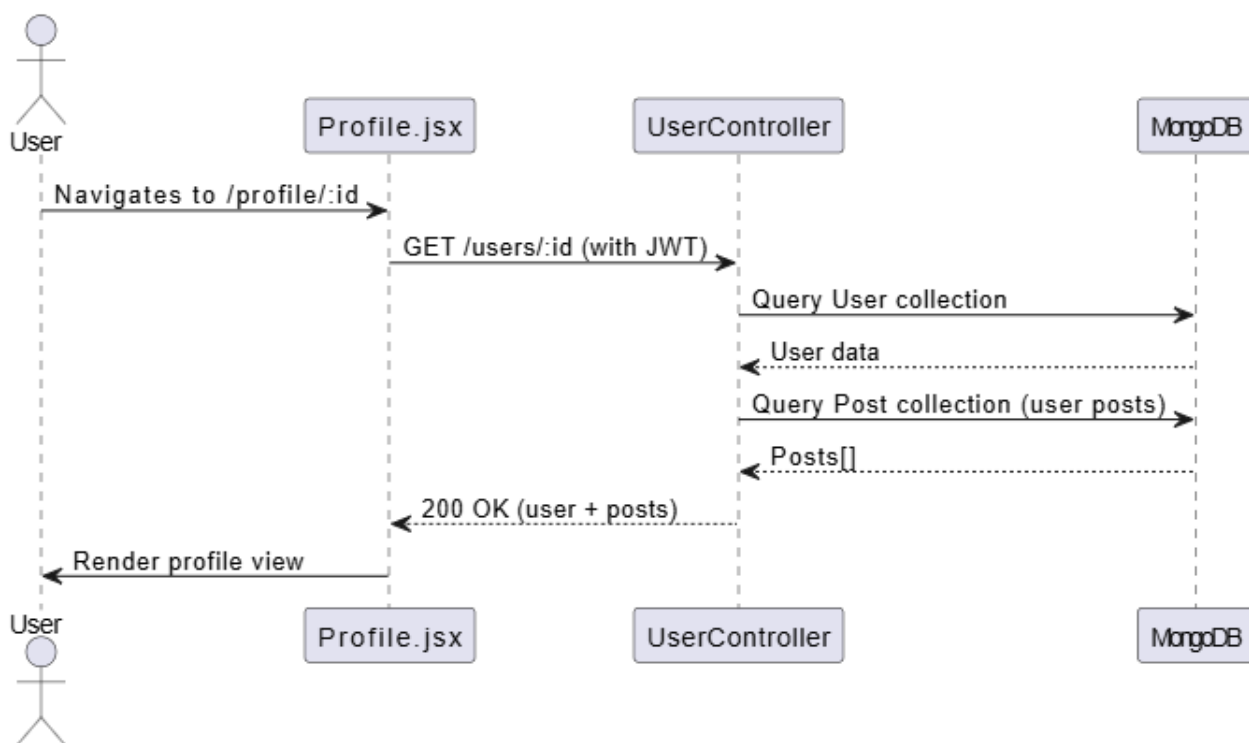


Figure 4.2: Sequence diagram "View Profile"

This sequence diagram accurately reflects the current profile-viewing implementation in WorkSphere. The flow demonstrates our standardized approach to user data retrieval, where all profiles, regardless of whether they belong to the authenticated user or others, return the same set of non-sensitive fields (password excluded via `.select('-password')` in the User model query). The JWT token, while required for authentication, does not currently gate profile visibility or alter the returned dataset, maintaining simplicity in our RESTful API design. Furthermore, the diagram highlights two sequential database queries: (1) user data fetch and (2) post retrieval, which reflects our frontend's compo-

sitional rendering. This separation of concerns was implemented to ensure independent caching and error handling of user metadata and post content.

4.3.1.2 Sequence diagram « Edit Profile »

The figure 4.3 illustrates the sequence diagram for the "Edit Profile" functionality, which will handle the scenario of changing a user's information or security credentials.

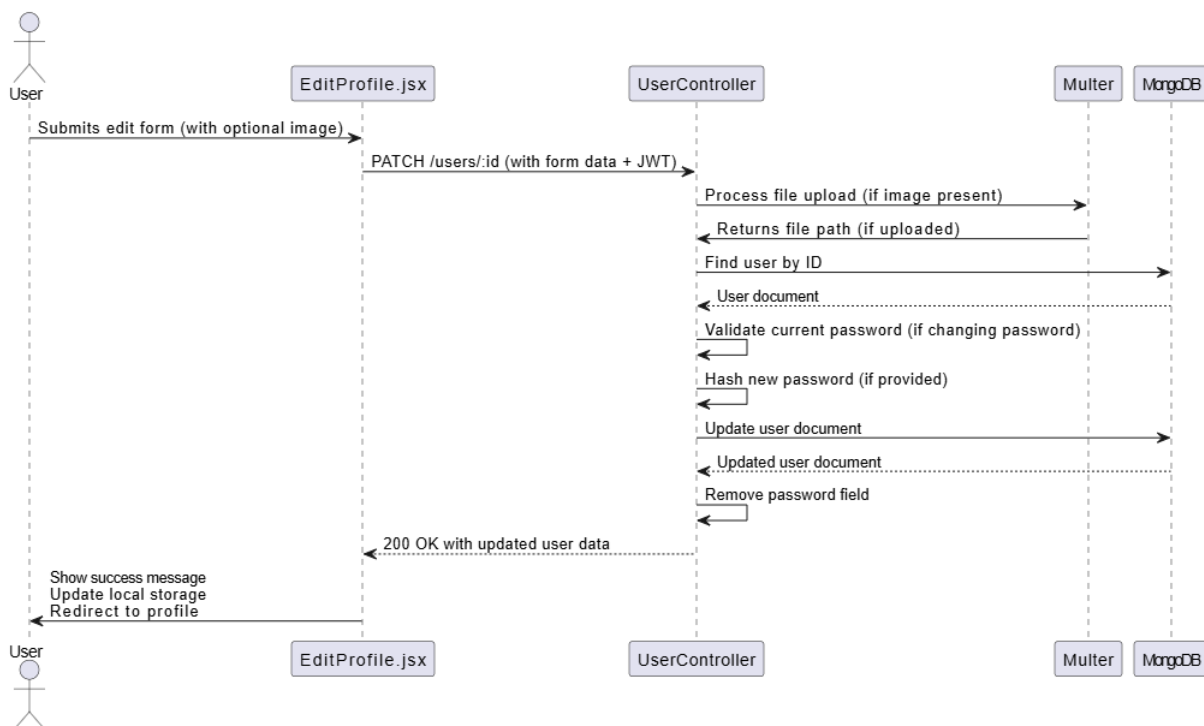


Figure 4.3: Sequence diagram "Edit Profile"

In this diagram, we observe that when a user submits changes (including optional profile picture uploads), the frontend (EditProfile.jsx) sends a PATCH request to the backend with the updated fields, authenticated via JWT.

Multer middleware processes file uploads (if present) before the UserController validates the request, ensuring password changes include verification of the current password.

The backend then hashes any new password, updates the MongoDB document, and returns a sanitized user object (excluding sensitive fields like the password). Finally, the frontend updates the UI and local storage to reflect changes immediately and seamlessly.

4.3.1.3 Sequence diagram « Toggle User Roles (Admin) »

The figure 4.4 illustrates the sequence diagram for the "Toggle User Roles" functionality, which will handle the scenario of updating a user's role from "user" to "admin" or vice

versa.

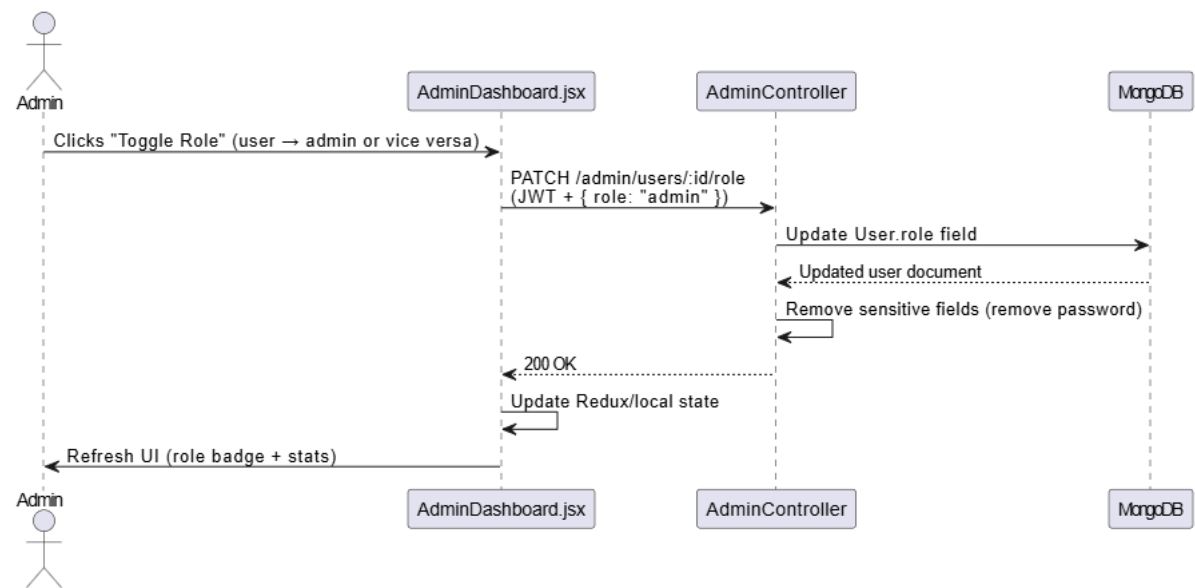


Figure 4.4: Sequence diagram "Toggle User Role"

When an admin selects a new role, the frontend sends a PATCH request to the backend with the updated role (validated via JWT). The AdminController updates the user's role in MongoDB and returns the sanitized user object (excluding sensitive fields like passwords). The frontend then updates both the UI (e.g., role badge) and global state (e.g., Redux or local stats) to reflect the change.

4.3.2 Class Diagram

The figure 4.5 represents the class diagram for the development of Sprint 2.

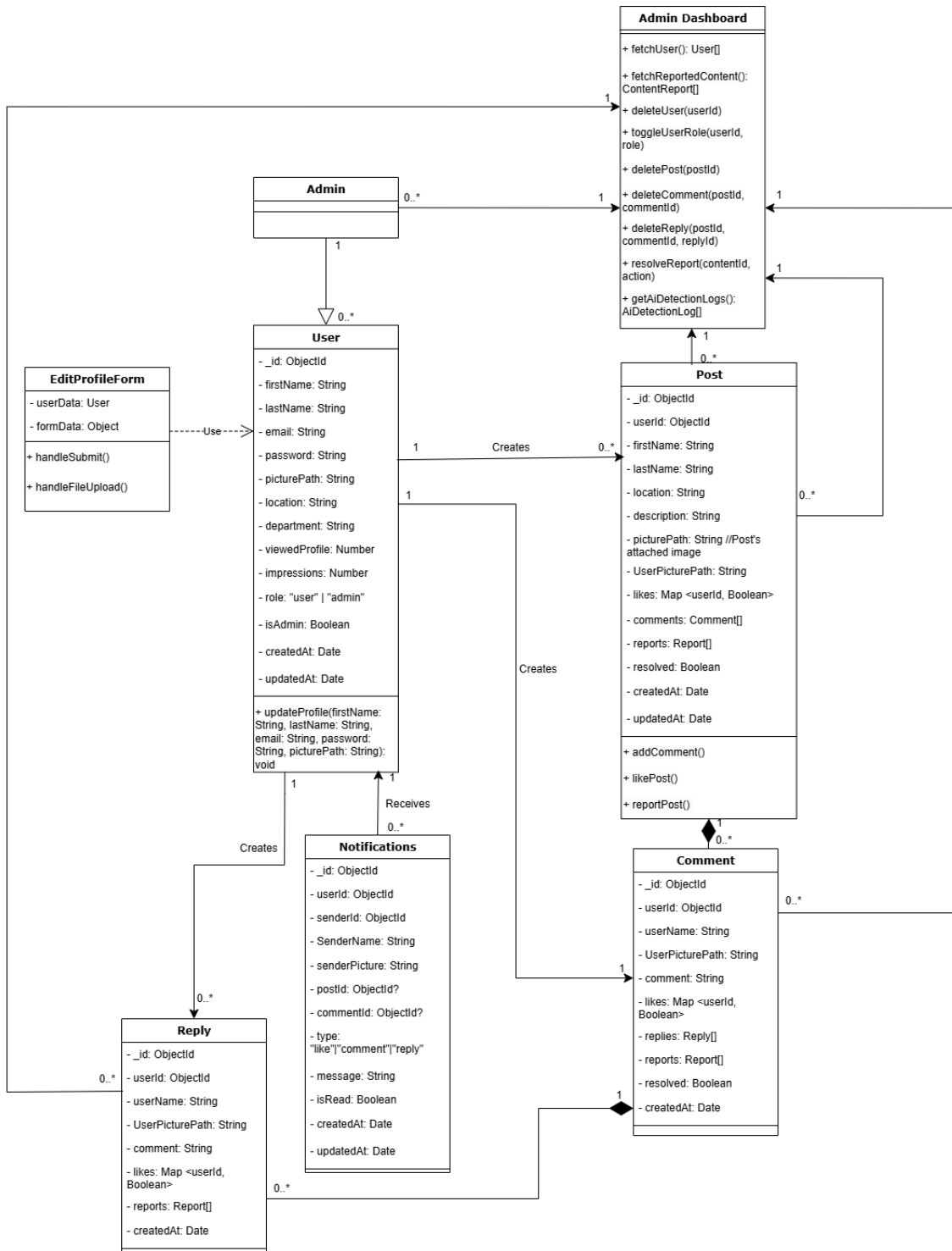


Figure 4.5: Class diagram for Sprint 2

The table 4.7 Description of the participating classes in the second sprint.

Class	Description
-------	-------------

Admin	Represents the admin of the application, which is a specialized type of User that inherits all user properties but is distinguished by elevated system privileges, enabling access to the Admin Dashboard for platform management tasks.
AdminDashboard	Central class for administrative actions. Manages users, moderates reported content (posts/comments/replies), resolves reports, and reviews AI detection logs.
EditProfileForm	Frontend component allowing users to update their profile (name, email, password, profile picture).

Table 4.7: Description of the participating classes in the sprint

4.4 Implementation

This section presents the front-end final implementation of this sprint.

First, the figure 4.6 presents the profile page interface, which serves as a personal space where other users can learn more about you, your information, and your activity on the platform. It also allows you to monitor and manage your presence and posts.

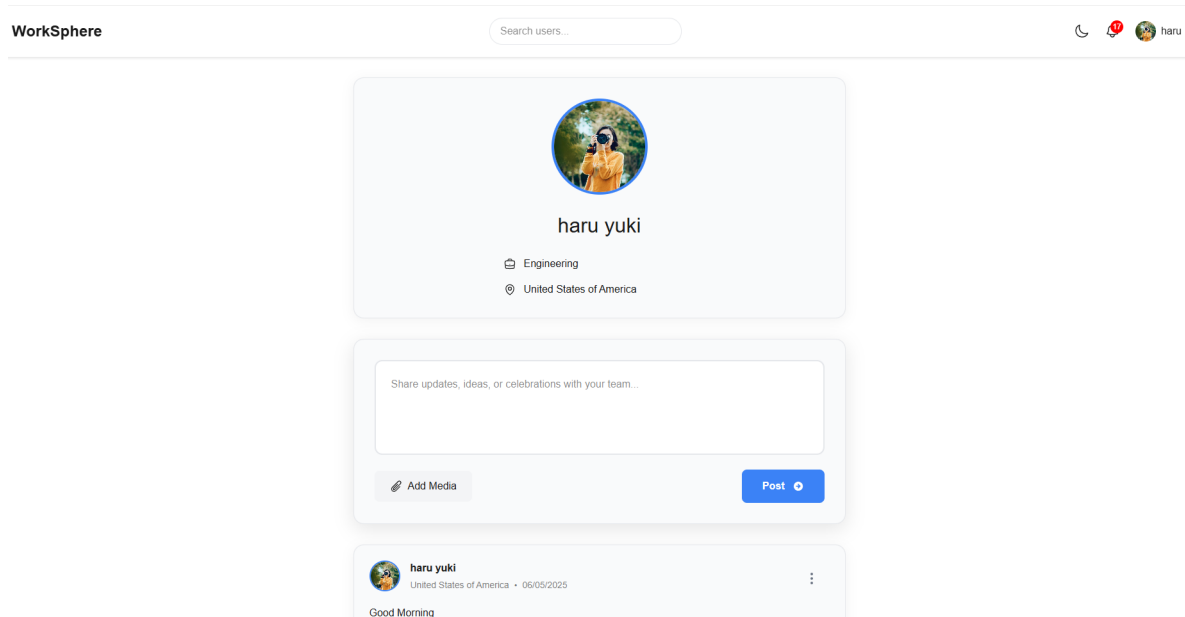


Figure 4.6: interface "Profile Page"

Next, we present in the figures 4.7 and 4.8 the "Edit Profile" interface, where a user can modify their personal information as well as their security credentials.

The screenshot shows the 'Edit Profile-1' interface within the 'WorkSphere' application. At the top left is the 'WorkSphere' logo. At the top center is a search bar labeled 'Search users...'. At the top right are icons for a bell, a red notification dot, and a user profile icon labeled 'haru'. The main content area is a light gray box containing a 'Profile Picture' section with a circular image of a person in a yellow jacket. Below this are several form fields: 'First Name' (containing 'haru'), 'Last Name' (containing 'yuki'), 'Email' (containing 'arthur@example.com'), 'Department' (containing 'Engineering'), and 'Location' (containing 'United States of America').

Figure 4.7: interface "Edit Profile-1"

The screenshot shows the 'Edit Profile-2' interface, specifically the 'Change Password' section. It contains three input fields: 'Current Password' (filled with dots), 'New Password' (containing the text 'Leave blank to keep current password'), and 'Confirm New Password' (also containing the text 'Leave blank to keep current password'). A blue 'Save Changes' button is located at the bottom of the form.

Figure 4.8: interface "Edit Profile-2"

Finally, the figures below showcase the admin dashboard where we have general relevant statistics about the website, along with a dropdown menu that allows us to toggle between different elements of the dashboard. The default element is the "User Management" table, as shown in Figure 4.9.

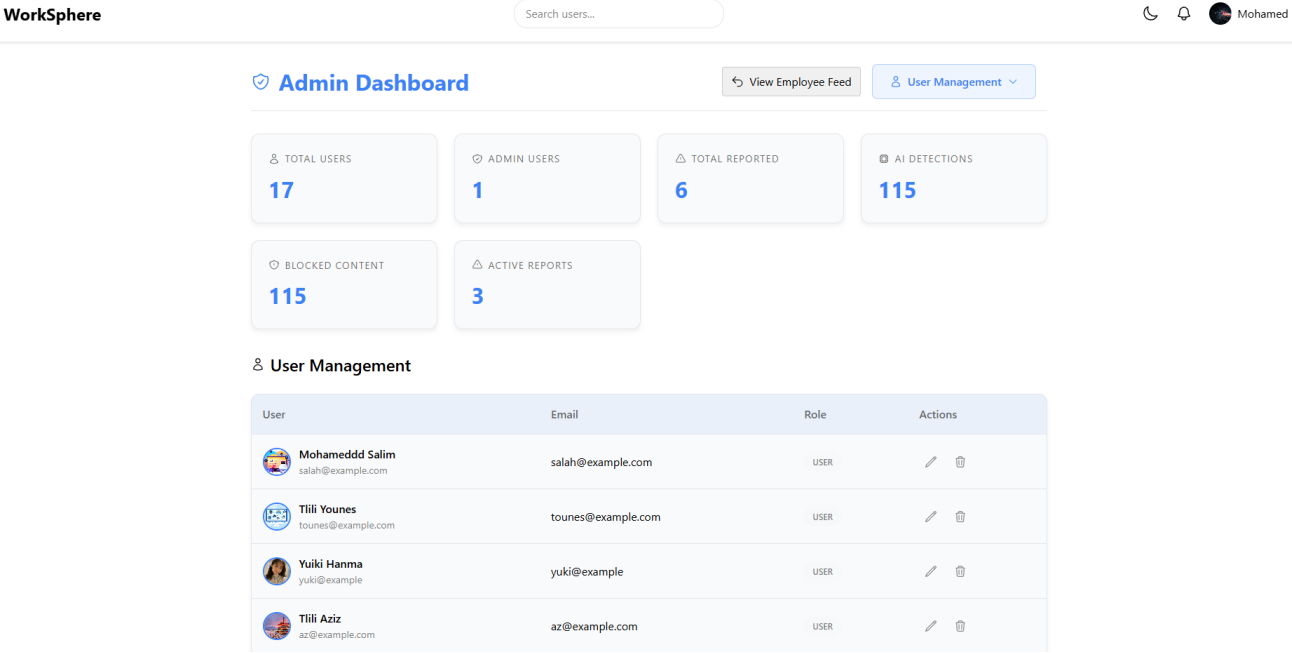


Figure 4.9: Interface User management

Conclusion

Building upon the core functionalities established in the first sprint, Sprint 2 significantly expanded the WorkSphere platform by introducing comprehensive user management features and crucial administrative oversight. This sprint successfully transitioned the application from a simple social feed into a more structured and governable corporate environment, fulfilling all objectives outlined in the sprint backlog. The primary achievements of this sprint can be viewed in two key areas. First, on the user-facing side, the introduction of a dedicated Profile Page provides employees with a personalized space within the network, while the Edit Profile functionality gives them control over their personal information. These features are essential for fostering a sense of identity and community within the platform. Second, and most critically, this sprint saw the development of the Admin Dashboard, a powerful and centralized control panel, as well as a new set of protected API endpoints exclusively for administrative action. The implementation of this dashboard marks a pivotal moment in the project, equipping administrators with the necessary tools for platform governance. As detailed in the use case and sequence diagrams, administrators can now view a complete list of users, delete user accounts, and manage user roles by toggling between standard user and administrator privileges. The class diagram was also evolved to incorporate the role and isAdmin attributes, reflecting these new capabilities at the data model level. In conclusion, Sprint 2 has successfully added a vital layer of personalization and administration to the WorkSphere application.

With robust user profile management and a functional administrative dashboard in place, the platform is now equipped for effective management and moderation. This sets the stage perfectly for the next phase of development, which will focus on implementing the manual content reporting system and preparing for the integration of the AI-powered moderation engine.

Sprint 3: Implementing Manual Moderation, and Dataset Preparation.

Plan

5.1	Sprint 3 Backlog	76
5.2	Functional specification	76
5.2.1	Refined Use Case Diagram « Sprint 3 »	77
5.2.2	Description of refined use case « Sprint3»	77
5.3	Design	81
5.3.1	Classe Diagram	82
5.4	Dataset Preparation	84
5.4.1	Challenges	84
5.4.2	Datasets Curation	85
5.4.3	Data Split	87
5.4.4	Data Balancing, and Augmentation Techniques	87
5.5	Implementation	89

Introduction

In this chapter, we will present the third sprint that covers the implementation of a manual flagging system to allow users to report hateful content that might have passed over the AI model undetected. Additionally, we will showcase the data curation and preparation process for building the dataset that we'd use in the next sprint.

5.1 Sprint 3 Backlog

The table 5.1 The table below details the sprint 3 backlog.

ID Sprint	Theme	Actor	User Story	Priority
N°1	Manual User Flagging	User	As a user, I want to report posts/comments that violate guidelines, so that admins can review them.	High
		Admin	As an admin, I want to view reported content in a dashboard, so I can take action (delete/allow).	High
	Dataset Preparation	Developer	As a developer, I need to collect and preprocess multilingual hate speech datasets, so the AI model can be trained.	High
			As a developer, I need to clean and balance the dataset (remove bias, handle class imbalance), so the model performs well and fairly across languages.	High

Table 5.1: Sprint 3 Backlog

5.2 Functional specification

In this section, we will present the requirements specification part of our project «**Sprint 3**».

5.2.1 Refined Use Case Diagram « Sprint 3 »

In this sprint, we provide the user with the ability to report hateful content that violates guidelines. Furthermore, we collect and preprocess data to create a quality dataset to be used later for the AI model training and evaluation.

The figure 5.1 describes the global use case diagram of this sprint.

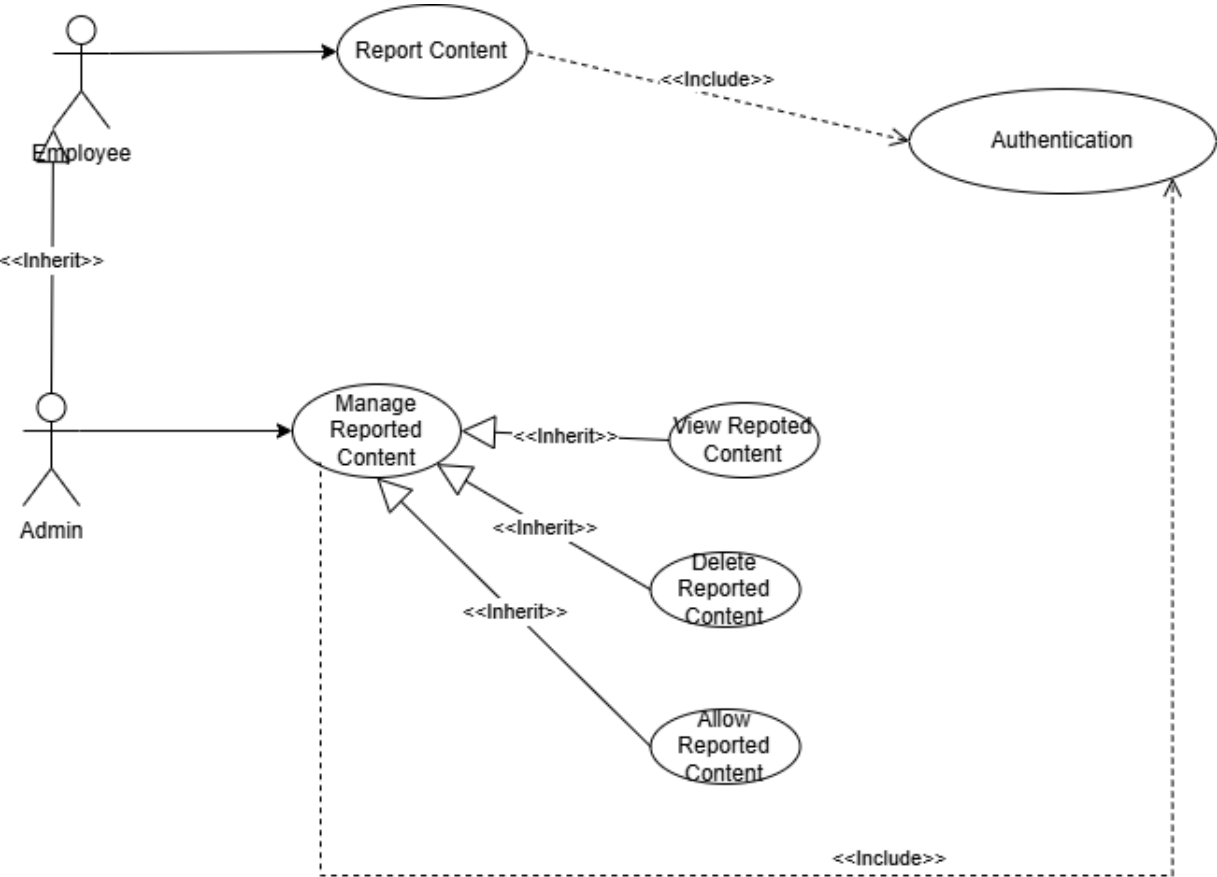


Figure 5.1: Refined use case diagram "Sprint 3"

— The "Report Content" use case provides the user with the ability to report posts, comments, or replies. The admin is notified of the reported content and is prompted to take action against it, whether by deleting or allowing this content.

5.2.2 Description of refined use case « Sprint3»

5.2.2.1 Description of the use case « Report Content »

The table 5.2 provides a description of the use case « Report Content »

Title	Flag Content for Review
Actor	User
Summary	A user (Employee) manually flags a post, comment, or reply they find inappropriate, offensive, or harmful. The flagged content is sent to the admin dashboard for review.
Sequence description	
Pre conditions	• User is authenticated and content exists on the platform.
Post conditions	• A report entry is stored, and the content becomes visible to moderators.
Normal scenario	
	• User sees inappropriate content that passed the AI model's check. • User clicks the "Report" button, selects a reason, and submits a report. • The report is saved to the database and the administrators can review it on the admin dashboard
Alternative scenario	
	• No report is submitted.

Table 5.2: Use case description "Report Content"

5.2.2.2 Use Case Description « View Reported Content »

The table 5.3 Provides a description of the use case« **View Reported Content** »

Title	View Reported Content
Actor	Admin
Summary	Admin can reviews content flagged reported by users as well as who posted them.

Sequence description	
Pre conditions	• Admin is authenticated and flagged content exists in the system.
Post conditions	• Administrator can view these content and who posted them as well as who reported it .
Normal scenario	
	• On the admin dashboard, upon selecting the "Reported" option of the scroll down menu, the admin can view in a table the list of reported content and any other necessary details concerning said reports.
Alternative scenario	
	• No reported content.

Table 5.3: Use case description "View Reported Content"

5.2.2.3 Use Case Description « Delete Reported Content »

The table 5.4 Provides a description of the use case« **Delete Reported Content** »

Title	Delete Reported Content
Actor	Admin
Summary	Admin reviews reported by users and can take action to delete them.
Sequence description	
Pre conditions	• Admin is authenticated and flagged content exists in the system.
Post conditions	• Reported content is deleted.
Normal scenario	

	• Admin can view reported content. • Admin may choose to delete reported content.
Alternative scenario	
	• No reported content or reported content contains no guideline violations.

Table 5.4: Use case description "Delete Reported Content"

5.2.2.4 Use Case Description « Allow Reported Content »

The table 5.5 Provides a description of the use case« **Allow Reported Content** »

Title	Allow Reported Content
Actor	Admin
Summary	Admin reviews content reported by users and may take action on it by allowing it.
Sequence description	
Pre conditions	• Admin is authenticated and flagged content exists in the system.
Post conditions	• Content is retained.
Normal scenario	
	• Admin can view reported activity on the dashboard. • Admin may choose allow the reported content.
Alternative scenario	
	• Reported content contain guidelines violation.

Table 5.5: Use case description "Allow Reported Content"

5.3 Design

In this section, we will present the design part by visually mapping out how different parts of this sprint interact over time and showcasing the order of operations between users, screens, backend components, and databases.

5.3.0.1 Sequence diagram « Report Content »

The figure 5.2 illustrates the scenario of the operation of reporting content.

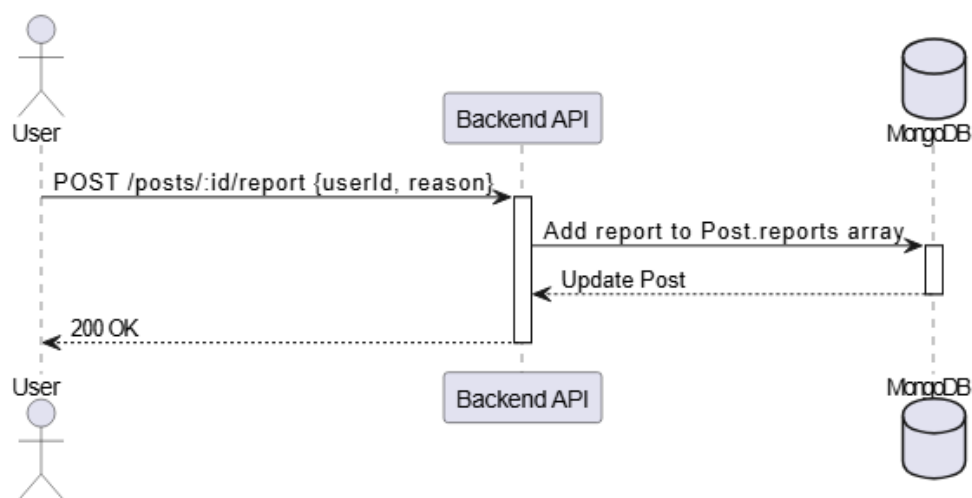


Figure 5.2: Sequence diagram "Report Content"

This sequence diagram illustrates how users report content. These reports will then be displayed on the admin dashboard.

5.3.0.2 Sequence diagram « Delete Reported Content »

The figure 5.3 illustrates how the admin deletes reported content that is confirmed to violate guidelines.

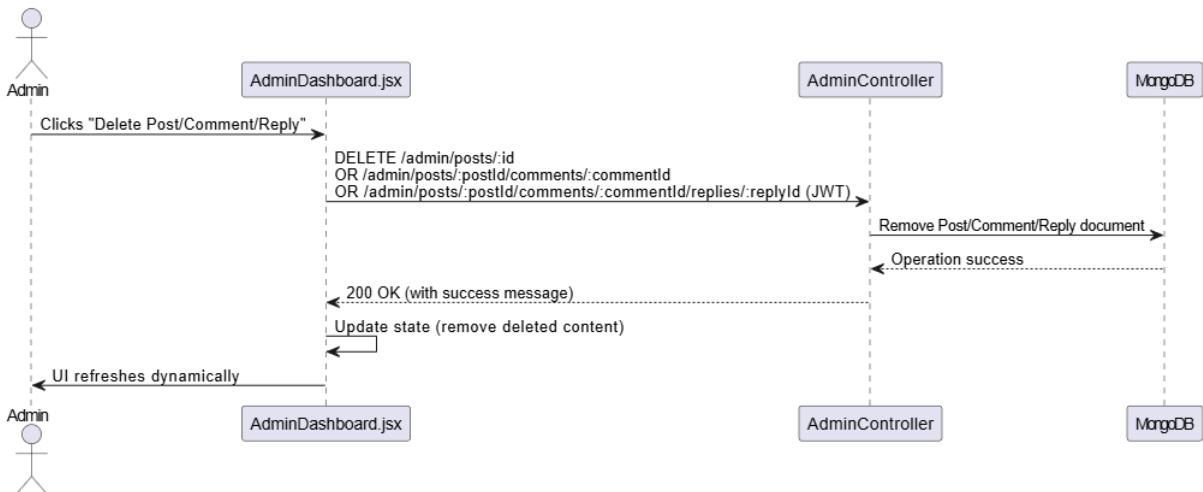


Figure 5.3: Sequence diagram "Delete Reported Content"

This sequence diagram showcases the admin content deletion flow, which includes posts, comments, and replies. When an admin triggers a deletion, the frontend sends a validated request (JWT) to the backend endpoint. The AdminController handles the database removal and returns a success response, prompting the frontend to update the UI state.

5.3.1 Classe Diagram

The figure 5.4 represents the class diagram for the development of this Sprint.

Class	Description
Report	Tracks user-submitted flags on posts/comments/replies. Includes reason, timestamp, and resolution status. Managed by the admin dashboard.

Table 5.6: Description of the participating classes in the first sprint

5.4 Dataset Preparation

The efficacy of any machine learning model, particularly a sophisticated transformer-based architecture for a nuanced task like hate speech detection, is fundamentally dependent on the quality, quantity, and representativeness of the training data. Our AI model is planned to be developed to detect hate speech in English, Arabic, and French. However, one of the biggest challenges we encountered in developing a reliable multilingual hate speech detection system was the lack of large-scale, high-quality annotated datasets. While significant resources exist for the English language, the same cannot be said for Arabic and French, which remain underrepresented in comparison, especially in the context of hate speech. This scarcity posed a significant obstacle for training and evaluating our deep models. As a result, we had to construct a multilingual dataset ourselves by collecting, aggregating, and curating annotated hate speech data from various publicly available and credible sources.

5.4.1 Challenges

As we mentioned, one of the biggest hurdles we encountered in the dataset preparation was the pronounced scarcity of publicly available, large-scale, and meticulously annotated hate speech datasets for French and Arabic compared to English. While numerous English datasets exist, finding comparable resources for French and Arabic proved more difficult and required much research, often yielding smaller or less diverse collections.

Additionally, while hate speech is a critical issue, it typically represents a minority class in overall online content. This class imbalance is made even more acute when sourcing data for under-represented languages, making it harder to collect sufficient positive examples of hate speech in French and Arabic without resorting to oversampling or synthetic data generation techniques, each with its own set of potential biases and limitations.

Finally, what made the data preparation even harder is that we had to focus on not just balancing minority classes and class distribution, but also we had to be mindful of language distribution while doing that.

5.4.2 Datasets Curation

Addressing these challenges mentioned in the previous section required a multi-pronged approach that included:

- **Combining Diverse Sources** : Due to the limitations of individual datasets in terms of size and linguistic coverage, especially in the underrepresented data, it was imperative to collect data from multiple different sources. This process involved identifying and acquiring various publicly available datasets and academic research corpora.
- **Structural and Semantic Inconsistencies** : Aggregating these diverse datasets into a single, cohesive multilingual corpus required
 - **Handling heterogeneous Formats and Structures** : We had to deal with combining different datasets that originated in various formats (e.g., CSV, TSV, plain text) with differing column structures, text field names, and meta-data. Heterogeneous Formats and Structures.
 - **Handling the different annotation schemas** : Another challenge we dealt with was reconciling inconsistent labeling schemes. Different datasets used varied taxonomies for hate speech (e.g., "hate," "offensive," "1," "toxic"). This required a meticulous mapping process to a unified set of class labels, in our case, "HOF" (hateful or offensive) and "NOT". We also standardized and unified column names. Furthermore, some datasets used binary labels, while others had fine-grained categories (sexist, religious, etc.), so we had to review and align these different classes into a broader binary category.
 - **Ensuring Data Integrity During Merging** : The process of merging datasets necessitated careful validation to prevent data corruption, ensure correct alignment of text with labels, and maintain the provenance of data from different sources for potential bias analysis.
- **Balancing Data** : While constructing the multilingual hate speech corpus, we ensured at each step that there was no significant imbalance in language representation or class distribution.

The Table 5.7 below presents the dataset curation and sources.

Language	Number of Samples	Source
English	5,852 sentences	Dataset curated and provided by the HASOC research initiative for the 2019 collaborative research challenge [1].
	3,708 sentences	Dataset curated and provided by the HASOC research initiative for the 2020 collaborative research challenge [1].
French	4,014 sentences	Dataset curated and used in the paper titled *Multilingual and Multi-Aspect Hate Speech Analysis* [30]. The dataset was uploaded to HuggingFace [2].
	3,719 sentences	Dataset curated and used in the paper titled *HateCheck: Functional Tests for Hate Speech Detection Models* [32]. The dataset was uploaded to GitHub [3].
Arabic	3,353 sentences	Dataset curated and used in the paper titled *Multilingual and Multi-Aspect Hate Speech Analysis* [30]. The dataset was uploaded to HuggingFace [2].
	4,646 sentences	Dataset presented in the paper titled *A Multi-Platform Arabic News Comment Dataset for Offensive Language Detection* [16]. The dataset sample was publicly uploaded to GitHub [4].

Table 5.7: Multiple hate speech datasets per language, showing number of samples and sources

In addressing disparities in data availability across languages, we initially considered oversampling and synthetic data generation techniques to augment the representation of under-resourced languages like French and Arabic. However, our experimental evaluations revealed that applying these methods for inter-language balancing introduced undesirable biases and practical limitations. We therefore concluded that a more effective and reliable

approach was to meticulously curate a corpus that was, as much as possible, inherently balanced in its linguistic distribution. Subsequently, techniques such as oversampling were strategically reserved for addressing intra-language class imbalance, specifically, the disparity between instances of hate speech and non-hate speech within each language dataset.

5.4.3 Data Split

For robust evaluation and testing of the model, the train dataset was partitioned into two distinct subsets: training and validation, with 80% of the available dataset allocated for training, while the remaining 20% was reserved for validation. Furthermore, we used a separate dataset entirely of **4,725 sentences**, which were not seen during training or validation, for the testing and evaluation of our fine-tuned models. This approach ensured an unbiased estimation of the model's real-world effectiveness, which is particularly important in a multilingual hate speech detection context.

Meanwhile, the validation set was used during model fine-tuning to monitor performance and prevent overfitting via early stopping and threshold tuning.

5.4.4 Data Balancing, and Augmentation Techniques

Even after preparing a robust multilingual corpus, we still had some imbalance issues in class distribution between "HOF" and "NOT", that is as follows:

Table 5.8: Class Distribution in the Multilingual Hate Speech Corpus

Label	Number of Samples
NOT (Non-hateful)	12,972
HOF (Hate/Offensive)	8,306

Due to this inherent imbalance, where non-hate (or neutral) content significantly outweighs hate speech instances, a series of data balancing techniques were explored in our different attempts and approaches to try and achieve the best results. These included:

- **Synthetic Minority Oversampling Technique (SMOTE)** : a statistical method we used in one of our approaches to address class imbalance by generating synthetic samples for the minority class, where instead of duplicating existing examples, SMOTE creates new synthetic instances by looking at real examples from the minority class, finding other similar examples (called 'nearest neighbors'), and

creating new examples that are somewhere in between them. Additionally, besides helping to balance the class distribution in our training set, this technique also helps to mitigate overfitting that may arise from simple replication.

- **Class Weights** : a cost-sensitive learning strategy that we employed during model training to counteract class imbalance by assigning higher importance to the minority class. This approach modifies the loss function by weighting the contribution of each class inversely proportional to its frequency in the dataset, therefore the model incurs a higher penalty for misclassifying rare instances, thus promoting a more equitable learning process across classes.
- **Synonym Replacement** : Another data augmentation technique used to resolve class distribution imbalance. In this approach, selected words in a sentence are replaced with their semantic equivalents, using synonym dictionaries to increase data samples and variability, and help the model generalize better

In our work, we applied synonym replacement using the WordNet lexical database via the NLTK library. For each selected word in a sentence, a synonym was randomly chosen from its WordNet synsets and substituted, thereby generating semantically similar yet syntactically varied training samples.

Aside from upsampling minority classes, the majority class was slightly downsampled to avoid relying on too much upsampling and synthetic data while removing extreme imbalance and, therefore, avoiding introducing bias.

5.5 Implementation

In this section, we will present the interfaces showcasing the implementation of the manual moderation.

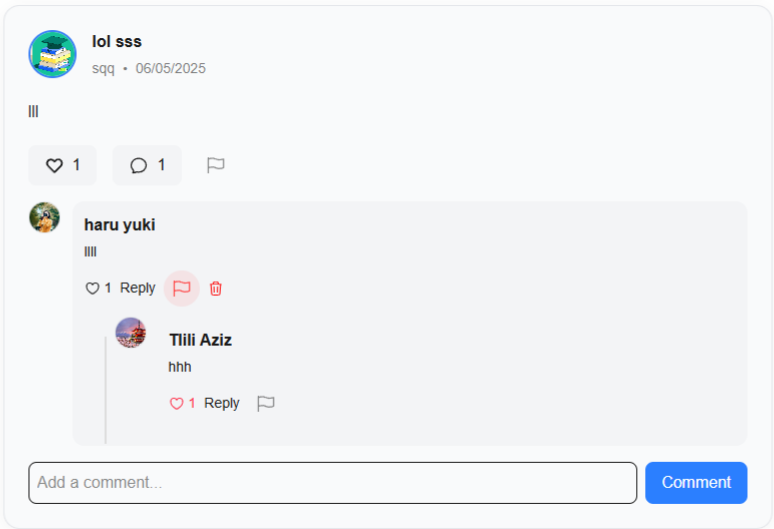


Figure 5.5: Reporting Content, Step 1

To report content, a user must click the flag icon as displayed in the figure above.

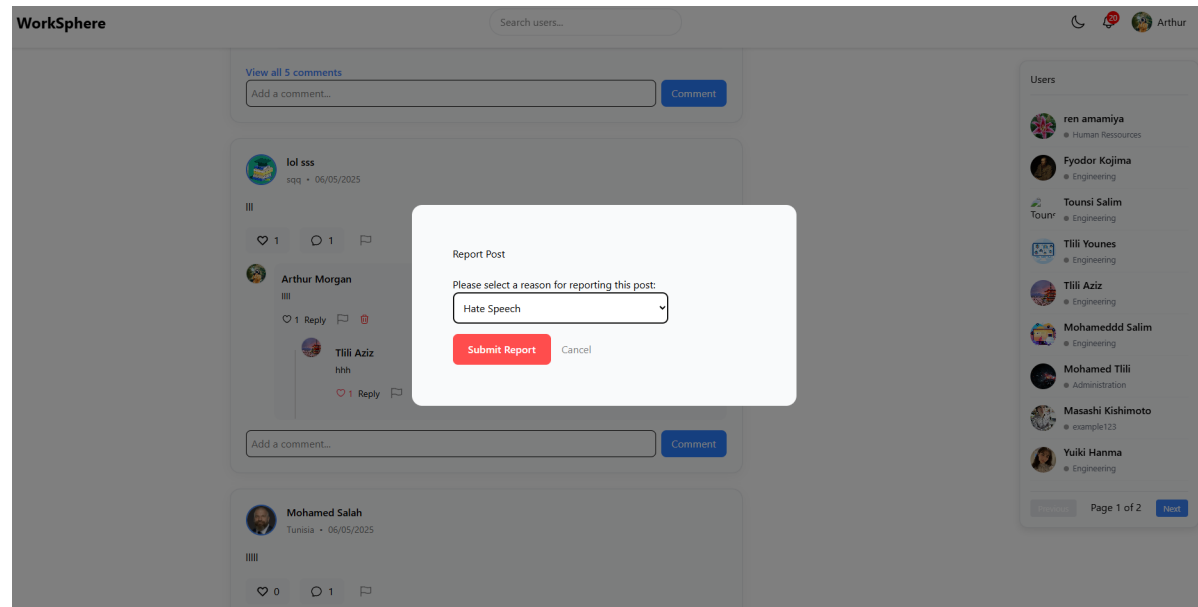


Figure 5.6: Reporting Content, Step 2

Once these icons are pressed, users are prompted to select a reason before submitting a report.

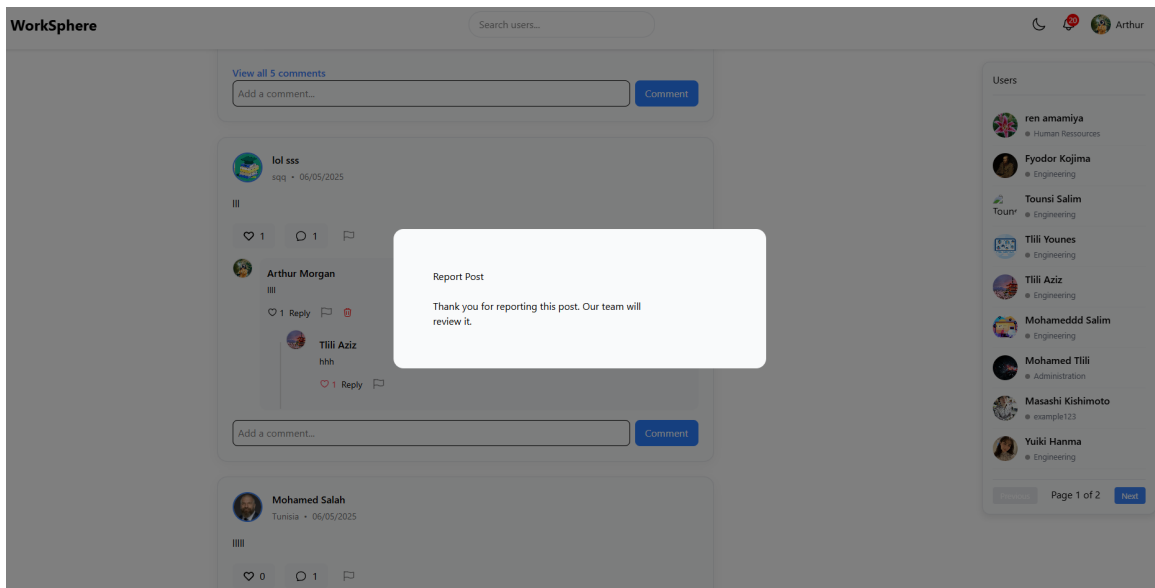


Figure 5.7: Reporting Content, Step 3

After selecting a reason and pressing "Submit Report", the user will receive confirmation.

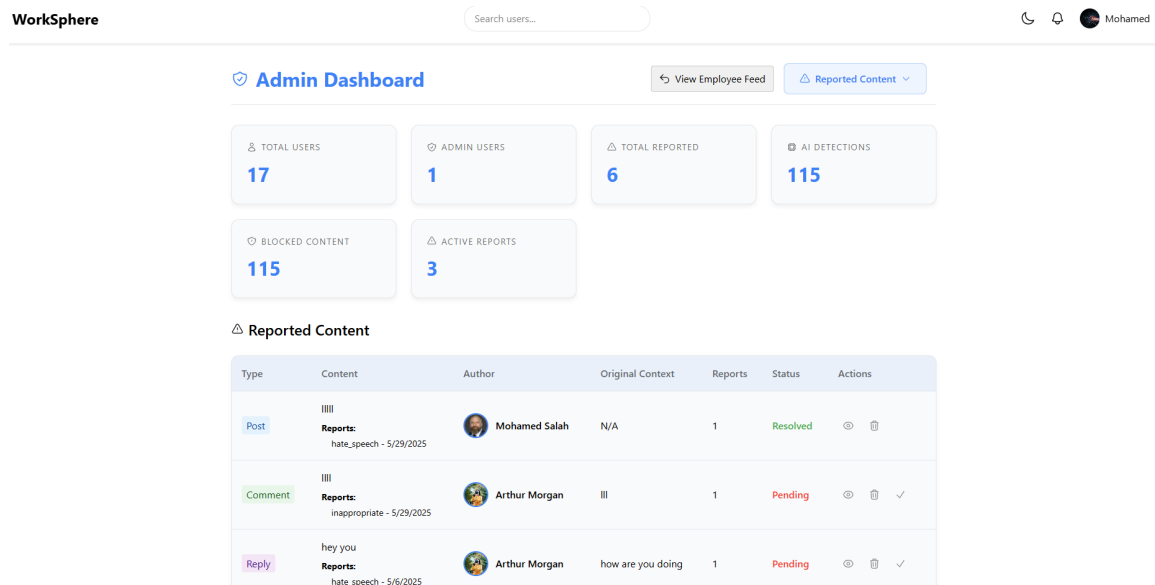


Figure 5.8: Reported Comments, Step 4

The admin will be able to consult reported posts, comments, or replies, and other relevant details, such as how many people reported the content in question. Reported content is displayed in different tables based on its type (post/comment/reply). The admin may delete reported content, in which case they would be prompted to reaffirm their choice, or allow them, and after that, the UI would be updated accordingly.

Conclusion

During this sprint, we implemented manual moderation to complement our automated moderation system. Additionally, we laid the foundation for the next sprint and chapter by collecting and preparing a robust, multilingual dataset that will be used for training and evaluating our AI models.

Chapter

6

Sprint 4. AI Model Development and Integration.

Plan

6.1	Sprint Overview	94
6.1.1	Sprint 4 backlog	94
6.1.2	Functional specification	95
6.2	Design	98
6.2.1	Scenario	98
6.2.2	Class Diagram	99
6.2.3	Description of the classes	100
6.3	Multilingual Hate Speech Detection	100
6.3.1	Challenges in Multilingual Hate Speech Detection	101
6.3.2	Our proposed Multilingual Hate Speech Detection Solution	102
6.4	Model Development and Integration	103
6.4.1	Hate Speech Detection As Binary Classification	103
6.4.2	Model Selection and Fine-tuning	103
6.4.3	Transformer models used	103
6.4.4	Fine-Tuning Approaches	107
6.5	Model Evaluation	108
6.5.1	Chosen Evaluation Metrics	108
6.5.2	Evaluation Results and Model Selection	109
6.6	Model Deployment & API Integration	113

6.6.1	Model Deployment Architecture	113
6.6.2	Model Integration	113
6.7	Implementaion	115

Introduction

In this chapter, we will provide a concise walkthrough of how we developed and integrated our AI model for hate speech detection, from the dataset preprocessing to the fine-tuning and evaluation, and then the integration into our WorkSphere application. Furthermore, this sprint will also include the development of the manual moderation and content flagging functionality, which is implemented as a fail-safe in cases where hate speech may slip past the AI model.

6.1 Sprint Overview

6.1.1 Sprint 4 backlog

The table 6.1 Presenting the backlog for the third sprint

ID Sprint	Theme	Actor	User Story	Priority
N°1	AI Model Development	Developer	As a developer, I need to evaluate multilingual transformer models (e.g., LaBSE, mBERT, XLM-RoBERTa), so I can select the best-performing one.	High
			As a developer, I need to fine-tune the selected model on our dataset, so it accurately detects hate speech in multiple languages.	High
			As a developer, I need to evaluate model performance, so I can validate its effectiveness.	High
	Backend Integration	Developer	As a developer, I need to create an API endpoint for the AI model (FastAPI), so the backend can send content for moderation.	High
			As a developer, I need to integrate the AI API so that all posts, comments or replies are automatically checked.	High

			As a developer, I need to log AI-detected violations in AiDetectionLogs, so admins can review them and determine false positives/negatives as well as users who violated the guidelines.	High
	Frontend Handling	Developer	As a developer, I need to show users a warning when their post is flagged, so they can revise it.	Medium
			As a developer, I need to display reported content in the admin dashboard with great clarity, so admins can moderate it.	High

Table 6.1: Sprint 1 Backlog

6.1.2 Functional specification

In this section, we will outline the requirements specification of our third Sprint, which focuses on enhancing WorkSphere’s content moderation system by:

- Implementing manual user flagging for community-driven moderation.
- Developing and integrating a multilingual hate speech detection AI model to automate this process.
- Ensuring seamless integration of the AI model.

Feature	Functional Requirements	Technical Implementation
AI Content Moderation	- All user-generated text is automatically screened for hate speech.	- FastAPI endpoint (/api/check-content) processes text. - contentVerification.middleware.js intercepts requests to validate text content before it’s saved to the database.
Real-Time User Feedback	- Users receive warnings if content is flagged by AI. - Admins see AI logs with confidence scores.	- PostForm.jsx shows toast alerts for blocked content. - AiDetectionLogs collection stores results for admin review.

Admin Modera- tion Tools	- Admins can view, delete or ignore reported/AI-flagged content.	- AdminDashboard.jsx aggregates data from AiDetectionLog and Post.reports.
--------------------------------	--	--

Table 6.2: Features and Specifications

6.1.2.1 Refined use case diagram « Sprint 3 »

This sprint is generally responsible for moderation and toxicity monitoring on the website, either automatically through the AI model or manually.

The figure 6.1 illustrates the refinement of the use case diagram for Sprint 3.

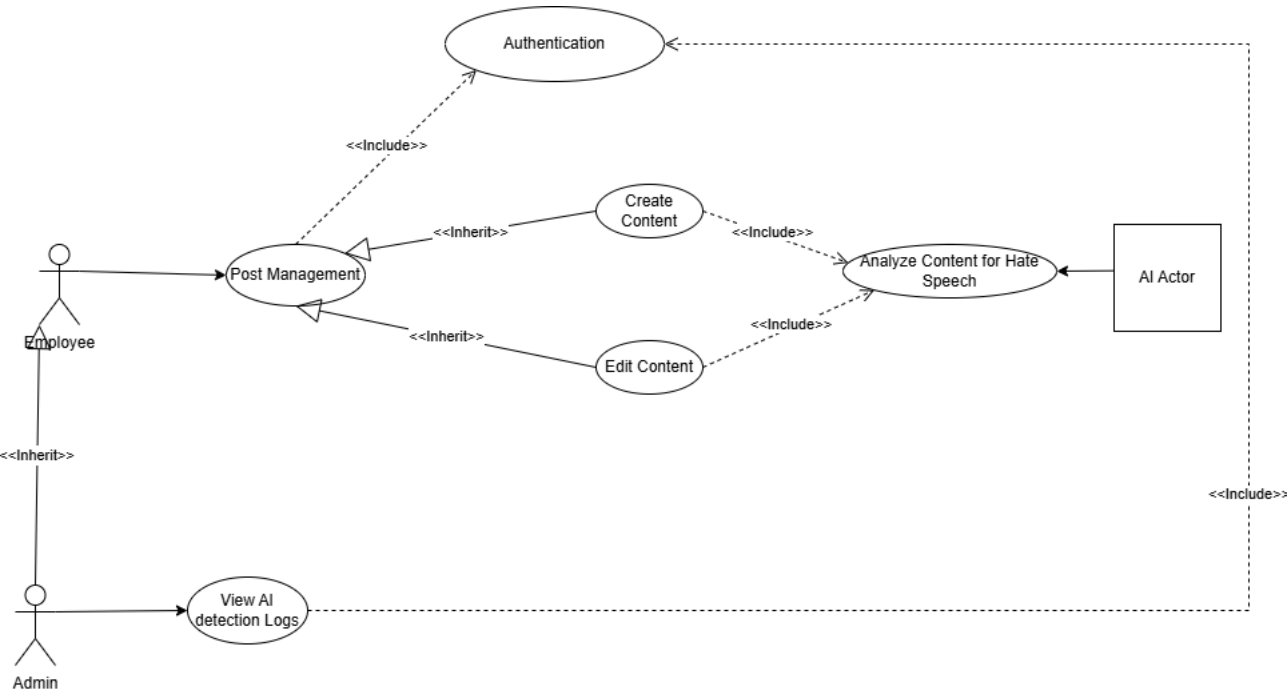


Figure 6.1: Refined use case diagram of "sprint 4"

- This use case diagram illustrates the community-driven and AI-powered content moderation workflows implemented during Sprint 3 of the WorkSphere project.
- The integration of a multilingual hate speech detection model into the platform’s content pipeline is the core of this sprint.
- Users (Employees) interact with the system primarily through creating posts and comments, and each time content is submitted, it is automatically passed through

the AI Detection Model for real-time analysis and hate speech detection. This behavior is modeled as a mandatory dependency («include») between the "Create Post / Comment / Reply" and "Check Content for Hate Speech" use cases.

- The "Analyze Content For Hate Speech" use case checks for hateful content, and if it's detected, it flags it and adds it to the log to ensure content accountability and traceability in the corporate setting.

6.1.2.2 Refined use case description « Sprint 4 »

6.1.2.2.1 Use case description « Analyze Content for Hate Speech » : The table 6.3 provides a description of the use case « **Analyze Content for Hate Speech** »

Title	Analyze Content for Hate Speech
Actor	AI Detection Model
Summary	Automatically analyze submitted content using a multilingual transformer-based model to determine if it contains hate speech.
Sequence description	
Pre conditions	• Model is loaded and API service is running.
Post conditions	• If hate speech is present, flags content and returns probability score.
Normal scenario	
	• System sends content to model. • The model returns a binary label and probability score. • The result is logged in the AI detection log. • The content is either accepted or rejected based on the result.
Alternative scenario	
	• Model API is unavailable or fails. • The system allows posting (fail-open policy) and logs the failure.

Table 6.3: Use case description "Analyze Content for Hate Speech"

6.2 Design

We will present the scenarios to be implemented in this sprint 3.

6.2.1 Scenario

6.2.1.1 Sequence diagram « Analyze Content for Hate Speech »

The figure 6.2 presents the sequence diagram illustrating how the AI model checks submitted content for hate speech and takes action against it.

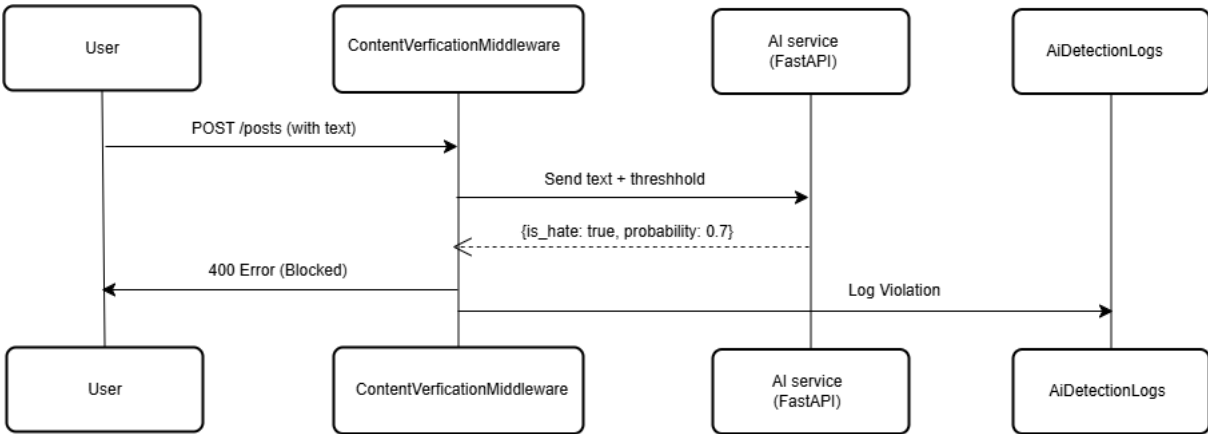


Figure 6.2: Sequence diagram "Check Content for Hate Speech"

This sequence diagram illustrates how the model analyzes posted content for hate speech and, if hate is present, how to block them and create a log of it. The process is initiated when a User submits a POST request containing text to create a new post. This request is first intercepted by the ContentVerificationMiddleware within the Node.js application, an architectural choice that prevents the request from reaching the final createPost controller until it has been vetted. The middleware then makes a synchronous API call to the external Python-based AI service, passing the text content for analysis. The AI service, powered by FastAPI, evaluates the text and returns a payload indicating that the content is hate speech (`is_hate: true`) with a high probability score. Upon receiving this negative verdict, the middleware performs two key actions in parallel:

- It logs a detailed record of the violation, including the user ID, content, and prob-

ability score, in the AiDetectionLogs database collection for administrative review and auditing purposes.

- It terminates the request-response cycle and effectively blocks the harmful content from ever being saved to the database or displayed on the platform.

6.2.2 Class Diagram

The figure below represents the class diagram used for the development of the last Sprint.

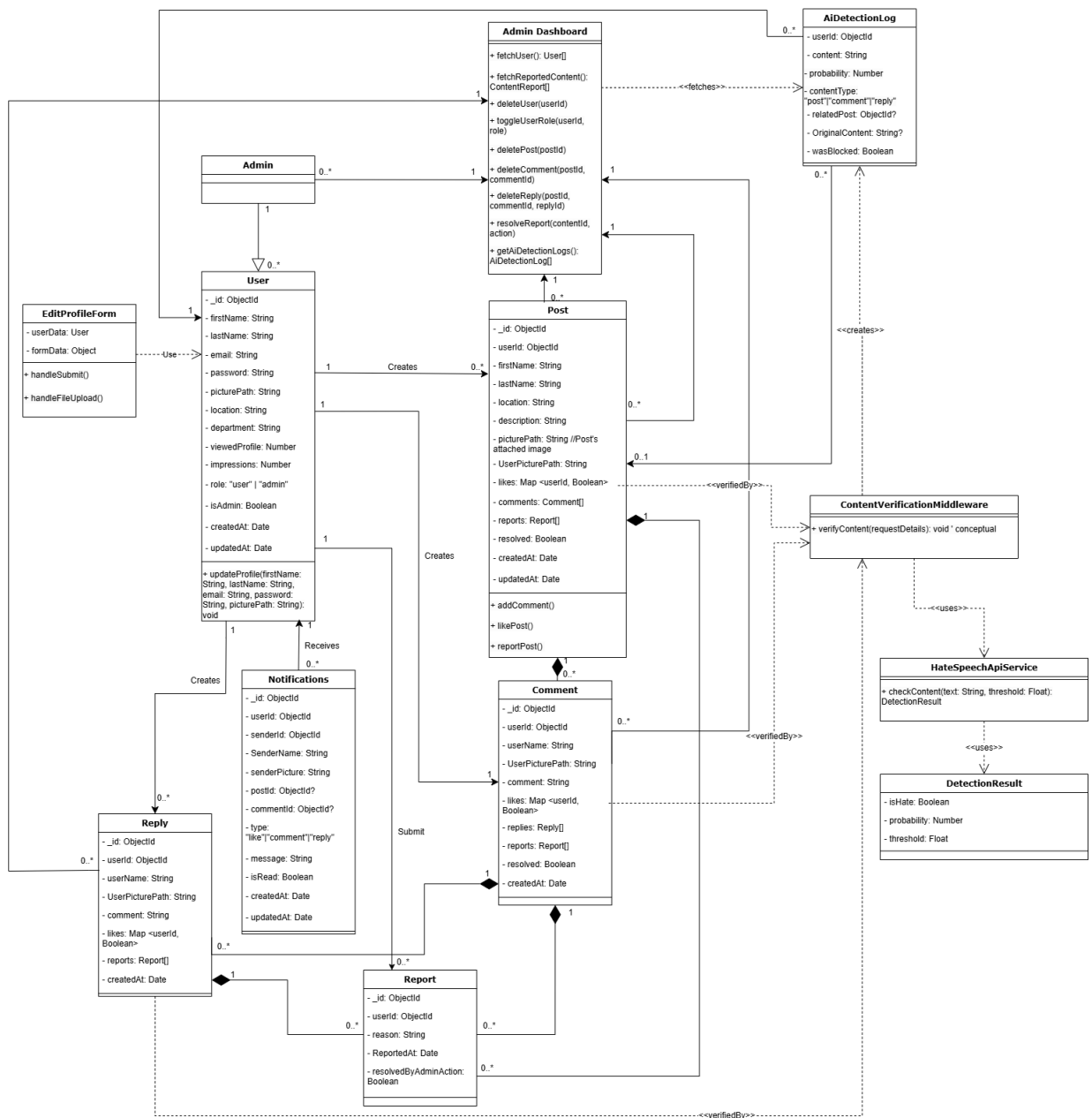


Figure 6.3: Class diagram of Sprint 4

6.2.3 Description of the classes

In this section, we describe the classes involved in this last Sprint (Table 6.4).

Class	Description
AiDetectionLog	Logs AI-detected harmful content (hate speech). Additionally, it tracks probability, content type, and blocked status. These data can be reviewed in the admin dashboard.
ContentVerificationMiddleware	acts as an automatic content checker in the application. When a user tries to post something new, this component steps in and sends the user’s text to the AI service to check for inappropriate language. Based on the AI’s findings, this middleware decides if the content is allowed, should be blocked, and if a record of the check needs to be made.
HateSpeechApiService	A specialized, external service that acts as the core AI engine. It’s responsible for analyzing provided text content against a pre-trained AI model to detect potential hate speech. It receives text and a sensitivity threshold, and returns a probabilistic assessment of whether the content is hateful.
DetectionResult	A data structure that encapsulates the output from the HateSpeechApiService.

Table 6.4: Description of the classes participating in sprint 4

6.3 Multilingual Hate Speech Detection

When it comes to Natural Language Processing, there is a significant disparity between research and development efforts in monolingual and multilingual capabilities, and this constraint is not limited to just hate speech. This limitation can be ascribed to multiple reasons that include linguistic nuances or technical complexities of handling multiple languages, but the most prominent of these reasons is the lack of high-quality, annotated datasets for underrepresented languages. Most hate speech detection research focuses on

the popular English language, contributing to an abundance of quality English datasets. However, this emphasis overlooks other languages, leaving them understudied and underrepresented. Therefore, most available datasets are either English or multilingual but suffer from imbalanced datasets, and often neither as large nor as well-annotated as English datasets.

6.3.1 Challenges in Multilingual Hate Speech Detection

Detecting hate speech in a multilingual context is significantly more complex than in monolingual settings. Amongst the challenges contributing to this complexity, we mention:

- **Linguistic Nuances and Ambiguities:** Different languages present different linguistic structures, idiomatic expressions, and culturally specific connotations, making it difficult to develop models that can encompass multiple languages at once. Moreover, hate speech can be subtle and context-dependent, as sarcasm, humor, or irony can disguise harmful intent, making detection difficult for automated systems, which would struggle between literal meaning vs. intent. As an example, in English, a comment like "Wow, great job taking all our jobs—guess we should just hand you the country too!" might be delivered with sarcasm. A human would recognize the hostility behind it, but an AI model could misinterpret or struggle to classify it.
- **Code-Switching and Mixed Languages :** Code-switching refers to the mixing of different languages within the same text or sentence, which is very prominent on social network platforms. This blending of languages poses a significant challenge since it breaks the assumptions of most NLP models, especially monolingual ones, and confuses word embeddings and the model's understanding of syntax and semantics. Furthermore, most traditional models use tokenizers that are designed for one language at a time. When multiple languages are mixed, especially with irregular spelling, these tokenizers can split words incorrectly or miss their meaning entirely.
- **Low-Resource Languages :** Many languages, especially those spoken in developing regions, are underrepresented in publicly available datasets, as they lack large, high-quality annotated datasets, pre-trained models, and sufficient linguistic resources compared to high-resource languages like English.

As a result, they are insufficient for training reliable and accurate deep learning models, which rely heavily on large volumes of annotated examples. Even popular multilingual models like the transformer-based mBERT or XLM-R are trained on

high-resource languages, meaning they may still perform poorly on low-resource languages.

- **Bias in Pre-Trained Models :** Multilingual models such as XLM-Roberta, mBERT, and LASER, although they have demonstrated impressive capabilities in handling text across a wide range of languages and achieving remarkable success in various tasks, including hate speech detection, they remain, nonetheless, susceptible to biases inherent in the training data which can significantly affect their performance, fairness, and generalization across languages and domains. The reason is that the corpora on which these models are pre-trained are often imbalanced in language representation, introducing bias and impacting the models' ability to perform equally well across languages. These biases, if left unchecked, pose significant challenges to applications like hate speech detection.

While English-based models dominate current research, extending these models to try to support multiple languages and mitigate the challenges often leads to a significant performance drop, making them obsolete.

6.3.2 Our proposed Multilingual Hate Speech Detection Solution

As we briefly mentioned in the first chapter, transformer-based models are currently considered the state-of-the-art (SOTA) in NLP, including hate speech detection, for both monolingual and multilingual settings. This fact can be asserted by multiple performance benchmarks where models like GPT-4, PaLM 2, and XLM-Roberta (for multilingual tasks) consistently top leaderboards [26]. Furthermore, XLM-R and mBERT outperform older models (e.g., LSTMs, CNNs) by 5–15% F1-score on benchmarks like HateXplain [9].

This impressive performance can be attributed to :

- **Superior Handling of Linguistic Nuances and Context :** transformers' superior Handling of Linguistic Nuances and Context is attributed to their self-attention mechanism which allows them to capture long-range dependencies, helping detect sarcasm, irony, and culturally coded hate.
- **Handling code switches :** Transformer-based models like LaBSE and XLM-R are pre-trained on multilingual data that includes natural language variation, including code-switched text, so they handle mixed-language text better than monolingual tokenizers. Furthermore, in the case of LaBSE model for example, it's language

agnostic, meaning it's trained to produce similar embeddings for semantically similar sentences, regardless of the language. Therefore for code-switched text that uses common and grammatically correct mixes (e.g., English-Spanish, Hindi-English), LaBSE can still embed the sentence meaningfully,

- **Handling data scarcity and low resource languages :** Transformers excel in transfer learning, meaning, fine-tuning a pre-trained model like XLM-R or LaBSE on a hate speech dataset allows it to specialize in that task while leveraging its massive multilingual knowledge. This is crucial for low-resource languages, where labeled data is scarce but transformers can still generalize well from related languages.

6.4 Model Development and Integration

6.4.1 Hate Speech Detection As Binary Classification

In its simplest form, hate speech detection is often framed as a binary classification problem: distinguishing between hateful and non-hateful content. In our work, we targeted binary classification problems dealing with: — Hate speech detection a classification distinguishing between hate/offensive (HOF) and non-hate (NOT). This task addresses the broader spectrum of online toxicity, aiming to differentiate explicitly harmful and offensive language from benign or neutral content.

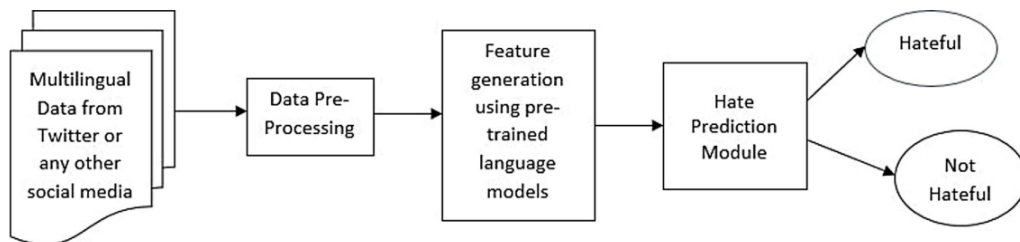


Figure 6.4: Process flow of multilingual hate speech detection [25].

6.4.2 Model Selection and Fine-tuning

6.4.3 Transformer models used

- **Multilingual-BERT (mBERT) :** An extension of the original BERT model and, unlike the original, is designed to handle multiple languages being pre-trained on a corpus of 104 languages and therefore can understand diverse linguistic structures

and semantics and perform various natural language processing (NLP) tasks across languages [31].

mBERT follows the same architectural design as BERT-Base, comprising 12 transformer layers (encoder blocks), each with 12 attention heads and a hidden size of 768, amounting to approximately 110 million parameters.

The model is trained using two unsupervised pretraining objectives:

- Masked Language Modeling (MLM): Predicting randomly masked tokens within a sequence to learn contextual representations[20].
- Next Sentence Prediction (NSP): Determining the relationship between pairs of sentences to model relationships or interactions between them and ensure inter-sentential coherence[20, 27].

In the context of our work on multilingual hate speech detection, mBERT presents an attractive choice due to its capacity to process multiple languages with a unified architecture. This reduces the computational overhead of training separate monolingual models and facilitates zero-shot or few-shot cross-lingual transfer learning, which is beneficial when labeled hate speech data is limited in some languages, such as our case. Furthermore, mBERT retains BERT’s Transformer-based architecture, which includes layers of attention mechanisms and feed-forward neural networks that enable the model to encode linguistic information in a context-aware, leading to effective cross-lingual transfer capabilities.

- **Language Agnostic BERT (LaBSE)** : To address the challenge of creating a unified, multilingual hate speech detection system, this study employs Language-Agnostic BERT (LaBSE) [22]. LaBSE, a multilingual sentence embedding model derived from the BERT architecture, offers robust, language-agnostic sentence representations for over 109 languages. A pivotal characteristic for our work is LaBSE’s capacity to generate high-quality, language-agnostic embeddings. This directly supports our objective because the model, through its dual-encoder architecture and training on extensive monolingual and parallel data using Masked Language Modeling (MLM) and Translation Language Modeling (TLM), learns to map semantically similar sentences from different languages to proximate vectors in a shared embedding space (represented by its normalized [CLS] token outputs). Such a shared semantic space is essential, as it permits a single downstream classifier to be fine-tuned on these embeddings, thereby enabling the identification of hate speech across diverse languages by focusing on underlying meaning rather than language-specific

syntax. This makes LaBSE a strong candidate for building an efficient and effective unified system.

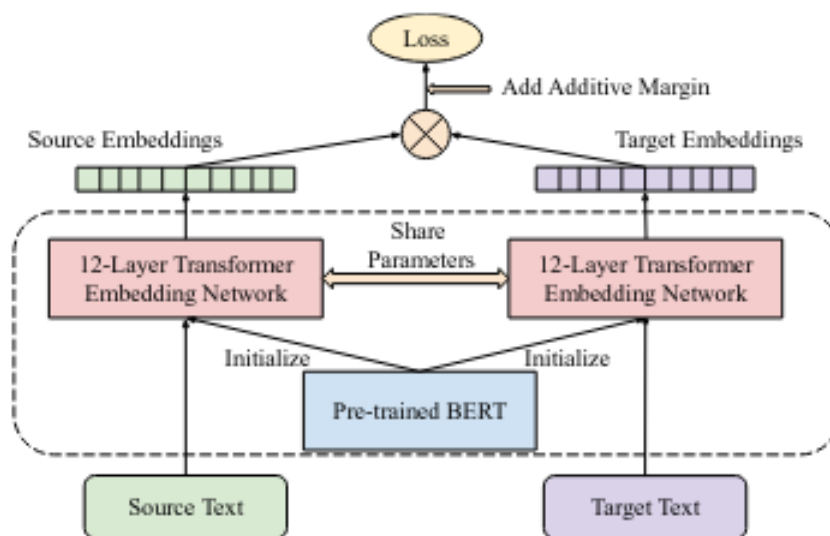


Figure 6.5: Architecture of the Language Agnostic BERT (LaBSE) model, illustrating its dual-encoder structure with shared parameters [22].

This architecture illustrates several critical components of the LaBSE model architecture :

- **Transformer Embedding Network** : This refers to the core processing unit within LaBSE that's based on the state-of-the-art transformer architecture that is exceptionally good at understanding context and relationships in sequential data like text. It uses self-attention to weigh the importance of different words in a sentence when creating its representation. "12-Layer" indicates the depth of this network, meaning the input data passes through 12 distinct processing layers, each progressively refining the sentence's representation.
- **Dual-Encoder Structure** : The diagram shows a "dual-encoder" structure with two identical "12-Layer Transformer Embedding Network" blocks, which crucially share parameters, forcing the model to learn a common representational space for different languages. The model processes a pair of sentences, a "Source Text" and a "Target Text" (typically translation pairs), each through an encoder. This setup, using translation pairs, trains LaBSE to focus on shared semantic meaning rather than language-specific features, thereby achieving language agnosticism. Furthermore, both encoders are initialized from a "Pre-trained BERT" model, giving them a strong linguistic foundation from the start.

- **Embedding Comparison and Loss :** The embeddings generated from the 'Source Text' and 'Target Text' are mathematically compared to measure how alike they are. This similarity score is then used to compute a 'Loss' value, with a high Loss indicating that the translation pair's embeddings were too different. The model uses this error to adjust its shared internal parameters through an optimization process, striving to minimize the Loss. This trains LaBSE to produce similar embeddings for sentences with equivalent meaning, regardless of the original language.
- **XLM-RoBERTa (XLM-R):** XLM-RoBERTa builds upon the XLM (Cross-lingual Language Model) and RoBERTa (Robustly Optimized BERT Pretraining Approach) architectures, aiming to enhance cross-lingual transfer by leveraging their respective strengths. Unlike mBERT, XLM-R is pretrained on a significantly larger and more diverse dataset (CommonCrawl), comprising 2.5TB of filtered text across 100 languages. This extensive pretraining enables XLM-R to capture deeper cross-lingual and semantic representations.

Furthermore, XLM-R adopts RoBERTa's optimized pretraining strategy, which notably excludes the Next Sentence Prediction (NSP) objective. This decision aligns with findings suggesting that the NSP task offers limited to no benefit for downstream language understanding tasks; indeed, its removal can even improve performance by allowing the model to more effectively leverage a robust Masked Language Modeling (MLM) pretraining strategy [28].

Architecturally, XLM-RoBERTa is fundamentally a RoBERTa model—an optimized BERT variant—that undergoes this extensive multilingual pre-training. Consequently, its core structure is a Transformer encoder. As depicted in Figure 1.2 (Chapter 1), which illustrates the general Transformer architecture, this encoder mechanism is representative of XLM-RoBERTa's design. The Masked Language Modeling (MLM) pre-training task, pivotal for XLM-RoBERTa's ability to learn effective cross-lingual representations, is a key training objective executed using this encoder framework.

These models' extensive use as a baseline in prior multilingual hate speech detection studies, as well as their relatively moderate size and computational efficiency, further support their use in our work and settings where resources are limited.

6.4.4 Fine-Tuning Approaches

Following the selection of LaBSE, mBERT, and XLM-RoBERTa as our base pre-trained multilingual models, a series of fine-tuning experiments were conducted to adapt these models specifically for the task of multilingual hate speech detection. The objective of this fine-tuning phase was to optimize the performance of each base model by exploring various architectural modifications and regularization techniques. The approaches detailed below were systematically applied, in combination and otherwise, to identify the most effective configurations for our specific dataset and task. The primary fine-tuning strategies investigated included:

- **Custom Classification Head Architecture :** Beyond the standard single dense layer often used for classification, we investigated the impact of more complex classification heads. we experimented with adding up to 4 more dense (fully connected) layers on top of the transformer's output. The number of dense layers and the number of units within each layer were varied to explore different capacities for the classification head. Activation functions "ReLU" were used in these hidden dense layers. Furthermore, to prevent overfitting in these newly added dense layers, dropout was applied almost after each dense layer. We experimented with different dropout rates to assess their impact on generalization.
- **Applying Weight Regularization :** To further enhance model generalization and prevent overfitting during the fine-tuning process, an additional regularization method was employed, specifically, L2 regularization was applied to the weights of the dense layers in the classification head. This penalizes large weight values, encouraging the model to learn simpler patterns and reducing its sensitivity to noise in the training data.
- **Focal Loss :** To further address the class imbalance issue, we tried using focal loss, which is an improved version of cross-entropy loss that reduces the influence of easy-to-classify examples and puts more focus on hard ones, in our case, rare hate speech cases. This helps the model learn better from difficult examples and not just from the majority class.
- **Layer Freezing and Unfreezing Strategies :** To leverage the rich linguistic knowledge learned during pre-training and to mitigate the risk of catastrophic forgetting, we experimented with layer freezing/unfreezing.
- **Early Stopping :** A form of regularization that we used during the model training process for all our experiments. It works by monitoring the model's performance on

a validation set. If the performance on this validation set stops improving, or starts getting worse, for a certain number of epochs, the training process is automatically stopped and the weights are restored to the best epoch. Early stopping is a great tool to prevent overfitting and save computational resources.

The application of these fine-tuning approaches was an iterative process. For each base model (LaBSE, mBERT, XLM-R), different combinations of these strategies were systematically explored. The performance of each configuration was monitored on a validation set to guide the selection of optimal hyperparameters and architectural choices. The specific configurations that yielded the best performance for each base model will be detailed and discussed in the subsequent Evaluations subsection.

6.5 Model Evaluation

6.5.1 Chosen Evaluation Metrics

Generally, for the evaluation of deep learning NLP models, no single metric can truly fully capture a model's performance, characteristics, and suitability for the task it's developed for. Therefore, for the evaluation of a multilingual hate speech detection model, we chose a combination of metrics to provide a comprehensive understanding of the model's strengths and weaknesses, especially considering the class imbalance and the differing costs associated with misclassifications. The following selected metrics are as follows :

- **Accuracy** : Accuracy measures the proportion of all instances (both hate speech and non-hate speech) that the model correctly classified. It is calculated as:

$$\frac{\text{True Positives} + \text{True Negatives}}{\text{Total Instances}}$$

This metric provides us with a general overview of the model's overall correctness, but alone it's not sufficient as it can be misleading in scenarios with significant class imbalance and needs to be considered in conjunction with other metrics.

- **Precision (Positive Predictive Value)** : Precision for the "hate speech" class measures the proportion of instances flagged as hate speech by the model that are genuinely hate speech. It is calculated as:

$$\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

In the context of social media moderation, high precision is crucial for minimizing false positives that may lead to unwarranted censorship.

- **Recall** : Recall for the "hate speech" class measures the proportion of all actual hate speech instances that the model successfully identified. It is calculated as:

$$\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

High recall is equally critical for a hate speech detection system as the other metrics. A false negative—where the model fails to identify actual hate speech, allowing it to remain on the platform—undermines moderation efforts. Thus, maximizing recall ensures the model captures as much harmful content as possible.

- **Macro F1 Score** : The Macro F1 score calculates the F1 score for each class independently ("hate speech" and "non-hate speech") and then takes the unweighted average. This metric was chosen because it gives equal weight to all classes, making it ideal for hate speech detection, where "hate speech" cases are rarer but critical. Furthermore, it prevents strong performance on the common "non-hate" class from masking poor detection of actual hate speech. This provides a truer measure of a model's ability to identify hate speech, regardless of its scarcity.
- **Micro F1 Score** : The Micro F1 score calculates precision and recall globally by summing up the individual true positives, false positives, and false negatives across all classes before calculating the F1 score. This was selected because it gives equal weight to each individual sample, thus reflecting the overall performance on a per-instance basis.

This suite of evaluation metrics was chosen specifically to provide a holistic view of the model's performance. Accuracy offers a general sense, while precision and recall target the specific types of errors critical to avoid in content moderation. Additionally, the comparison between Micro F1 (reflecting overall instance-level correctness) and Macro F1 (reflecting average per-class performance) offers insights into how class imbalance affects the model's performance.

6.5.2 Evaluation Results and Model Selection

Given resource constraints, we initially evaluated the three base models using basic uniform fine-tuning for the hate detection task. This approach is a baseline aimed at identifying the highest-performing model among the three selected, allowing us to subsequently

concentrate our efforts and, more importantly, resources on it for more extensive evaluation with varied preprocessing techniques and fine-tuning configurations.

The table 6.5 shows the evaluation results for the different models with minimal fine-tuning and data balancing.

Model	Test Acc. (%)	Macro F1 (%)	Micro F1 (%)	Precision (%)	Recall (%)
LaBSE	67.14	66.84	67.14	74	67
XLN-RoBERTa	34.92	25.88	34.92	12	35
mBERT	56.72	56.64	56.72	71	57

Table 6.5: Performance Summary of Base Models (Basic)

Next, selecting LaBSE as the better-performing model of the three, we moved on to further fine-tuning the model and evaluating performance with the purpose of achieving the best performance.

The table 6.7 summarizes the different techniques and approaches we adopted as well as the performance they yielded. Note that the experiments are not in order of implementation. Note that the experiment ID does not reflect any specific order.

Exp. ID	NotablePreprocessing & Balancing Techniques	Layer Freezing Strategy	Classification Head (Dense Layers: Units, Activation)	Dropout Rate(s)	L2 Reg.	Loss Function (Gamma if Focal)	Epoch Count
1	-Augmented and upsampled the minority class (HOF) through Synonym Replacement to match majority class. -For each HOF instance, two augmented samples were generated with up to three random words replaced by their synonyms.	All Layers Trainable	1: (400, ReLU), 1: (230, ReLU), 1: (50, ReLU)	2: 0.4	3: 0.01	Focal Loss	50

Exp. ID	Notable Preprocessing Technique(s)	Layer Freezing Strategy	Classification Head (Dense Layers: Units, Activation)	Dropout Rate(s)	L2 Reg.	Loss Function (Gamma if Focal)	Epoch Count
2	-Balanced Class Weights using <code>compute_class_weight</code> from <code>sklearn.utils.class_weight</code>	All Layers Trainable.	1: (250, ReLU), 1: (50, ReLU)	1: 0.4	1: 0.01	Focal Loss	50
3	-Balanced Class Weights using <code>compute_class_weight</code> from <code>sklearn.utils.class_weight</code>	All Layers Trainable.	1: (250, ReLU), 1: (50, ReLU)	1: 0.4	2: 0.01	Focal Loss	69
4	- Balanced class distributions within each language by upsampling minority class instances using resample function from the Scikit-learn library. - Applied WordNet-based synonym replacement augmentation to the newly created upsampled text instances to increase data diversity.	All layers trainable	1: (512, ReLU), 1: (128, ReLU)	1: 0.5	1: 0.02, 1: 0.01	Focal Loss	50
5	-Relied entirely on focal loss for handling imbalance.	Unfroze top 8 of 12 LaBSE encoder layers.	1: (128, ReLU), 1: (50, ReLU)	1: 0.4	2: 0.01	Focal Loss ($\gamma=2.0$ $\alpha=0.65$)	56

Exp. ID	Notable Preprocessing Technique(s)	Layer Freezing Strategy	Classification Head (Dense Layers: Units, Activation)	Dropout Rate(s)	L2 Reg.	Loss Function (Gamma if Focal)	Epoch Count
6	-Upsampled the initial dataset to 29817, making "HOF" the majority class, with 16,881 HOF samples and 12,936 samples "NOT" samples.	Unfroze the top 10 of 12 LaBSE encoder layers.	1: (256, ReLU), 1: (128, ReLu), 1(50, ReLU)	2: 0.4	3: 0.01	Focal Loss ($\gamma=2.0$ $\alpha=0.43$)	70
7	-Used the original dataset. We implemented SMOTE to address the class imbalance in the training data by creating new, synthetic examples of the minority 'HOF' (hate speech) class from the LaBSE embeddings.	All layers trainable.	1: (250, ReLU), 1: (50, ReLu)	1: 0.4	2: 0.01	binary cross-entropy	21

Table 6.7: Overview of Experiments On The Selected LaBSE Model

The next table 6.8 showcases the evaluation results yielded by each of the experiments.

Exp. ID	Test Acc. (%)	Macro F1 (%)	Micro F1 (%)	Precision (%)	Recall (%)
1	83.17	81.5	83.17	83	83
2	79.32	78.5	79.32	83	79
3	82.68	81.86	82.68	85	83
4	79.07	78.46	79.07	83	79
5	34.92	25.88	34.92	12	35
6	55.23	35.58	55.23	31	55
7	82.98	81.90	82.98	84	83

Table 6.8: Evaluation Results for The Different Experiments

Based on the results, we chose the 3rd experiment's model as the best model for our platform. Based on the results for each class, it is more effective at the primary goal: finding hate speech. It correctly identifies 88% of hate (HOF specific Recall), compared to only 76% for the second runner-up, Experiment 1. This means it misses far less harmful content. Same thing for experiment 7, which correctly identifies 84% of hate content.

6.6 Model Deployment & API Integration

The successful operation of the multilingual hate speech detection model within the WorkSphere platform is paramount to achieving the project's prime goal of fostering a safe and respectful digital workplace. After selecting our model, this section will detail the architectural decisions and implementation strategies employed for deploying said model and integrating its inference capabilities into the MERN stack application.

6.6.1 Model Deployment Architecture

To serve the hate speech detection model effectively, a dedicated microservice architecture was adopted, decoupling the machine learning inference logic from the main application backend. This approach was chosen as it offered several advantages, including independent scalability, maintainability, and the ability to utilize Python's rich ecosystem for machine learning tasks. Additionally, the model's inference service was developed using the Python-based FastAPI framework. This choice was driven by FastAPI's high-performance characteristics, essential for low-latency model inference; its asynchronous capabilities, which enable efficient handling of concurrent API requests, and its automatic data validation via Pydantic schemas, ensuring robust input/output handling.

6.6.2 Model Integration

The overall system architecture, illustrating how these components interact, is depicted in Figure 6.6.

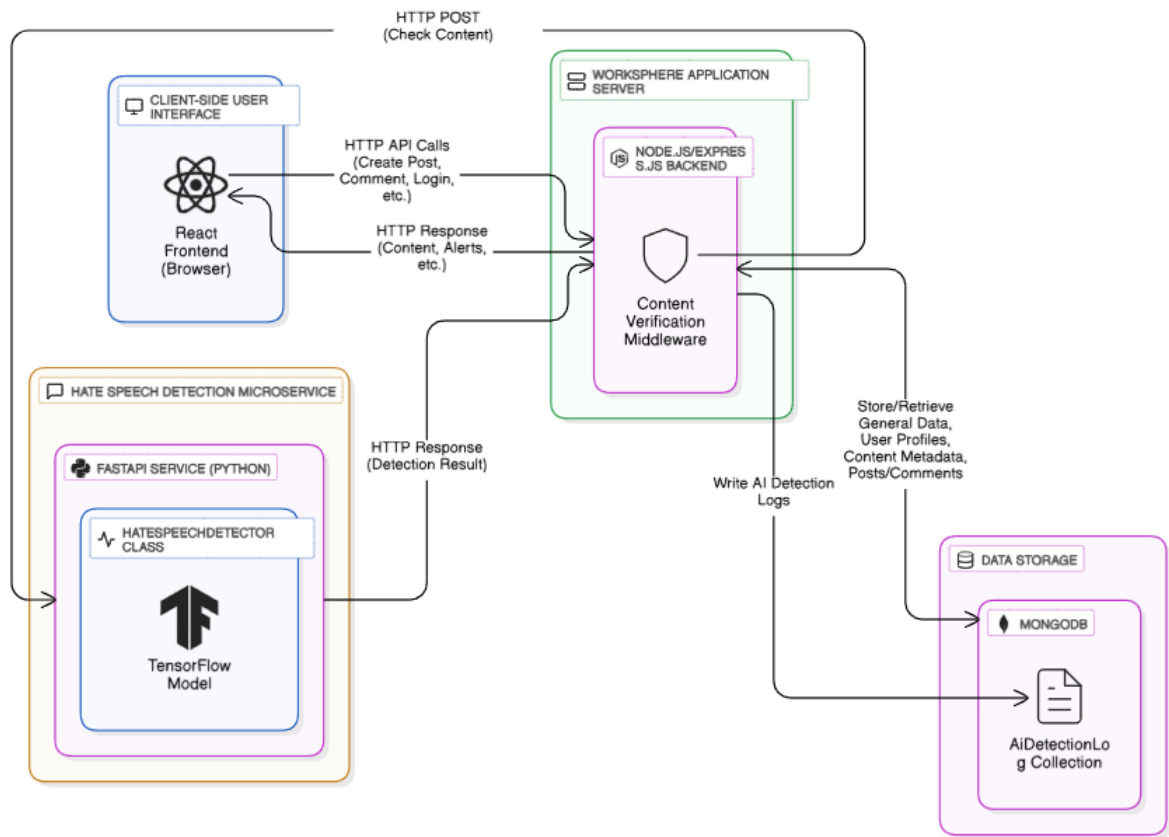


Figure 6.6: WorkSphere System Architecture with Integrated Hate Speech Detection

A core component within this service, a `HateSpeechDetector` class, manages the loading of the pre-trained TensorFlow SavedModel and its associated Hugging Face AutoTokenizer. The service exposes a primary HTTP POST endpoint for content analysis. This endpoint accepts a JSON payload containing the text to be analyzed and an optional classification threshold, structured according to a predefined request schema. Upon processing, it returns a JSON response indicating whether the content is classified as hate speech, the model's confidence score, and the threshold employed. Standard API features such as Cross-Origin Resource Sharing (CORS) were enabled for frontend communication, and a dedicated health endpoint was implemented to monitor service status and model readiness.

The WorkSphere application server (Node.js/Express.js) integrates with the inference service via a custom `verifyContent` middleware. This middleware intercepts content creation requests (for posts, comments, replies) and asynchronously calls the FastAPI service. If content is flagged as hate speech, `verifyContent` returns an HTTP 400 error to the client and logs detailed detection event data (user, content, probability, context) to an `AiDetectionLog` collection in MongoDB for auditing and model assessment. This middleware is applied to relevant content creation routes. A "fail-open" strategy ensures application

availability during inference service outages by logging the error and allowing submission, while still capturing disruption data.

6.7 Implementaion

The figure 6.7 illustrates the system’s response when an employee tries to post content containing hate speech.

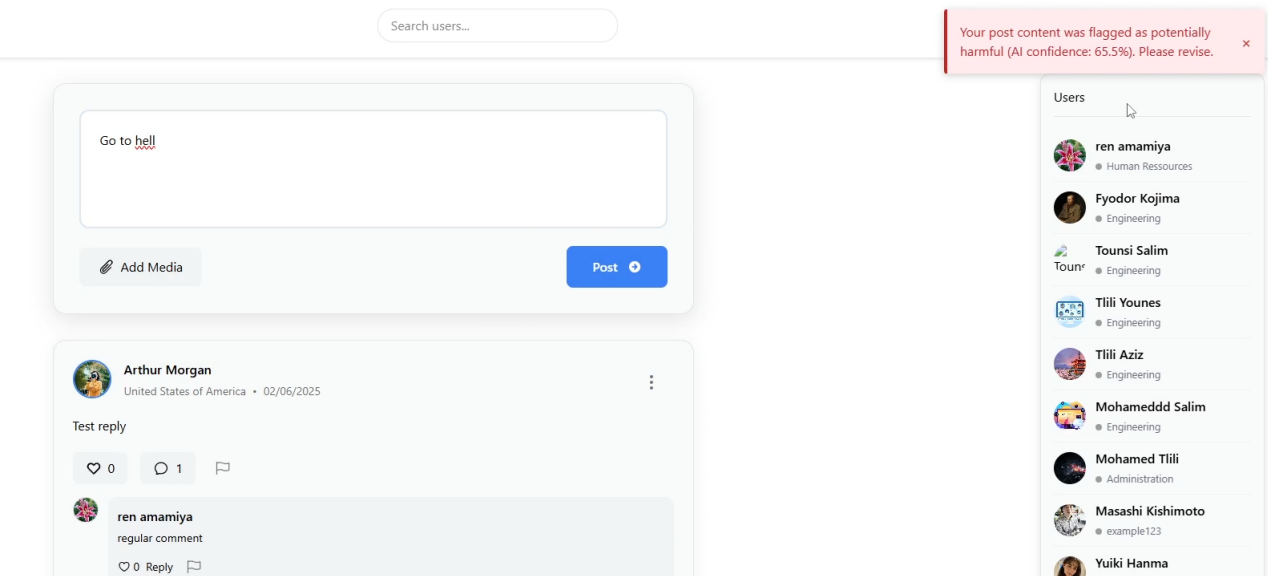


Figure 6.7: AI model in action

Upon Hate detection, a log is created and reported to the admin for transparency and future accountability, as shown in 6.8.

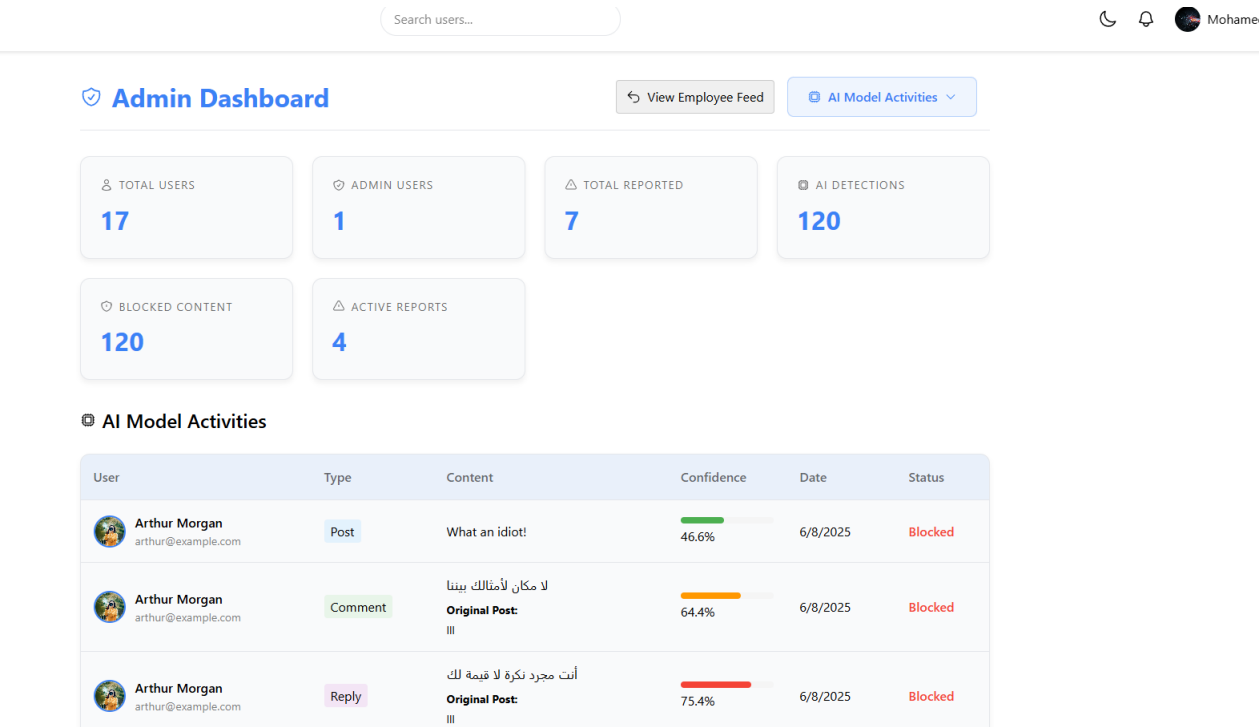


Figure 6.8: AI logs and Activity table

Conclusion

In conclusion, this chapter detailed the execution of Sprint 4, which constituted the central innovation of the WorkSphere project. We successfully navigated the complexities of multilingual hate speech by adopting a state-of-the-art approach based on fine-tuning Transformer models. Through a rigorous empirical evaluation using metrics such as F1-Score, precision, and recall, we identified and selected the most performant model for our specific use case. The culmination of this sprint was the successful deployment of this model as a microservice and its seamless integration into the WorkSphere application's backend. This crucial step transformed the platform from a standard social hub into an intelligent system capable of automatically analyzing and flagging potentially harmful content, thereby fulfilling the primary objective of creating a proactively safe digital workplace.

General conclusion

This thesis has meticulously documented the conception, design, and implementation of WorkSphere, an AI-powered corporate social hub. The project successfully achieved its primary objective: to create a secure, engaging, and modern social network for enterprise environments, where employee communication is fostered and protected by an intelligent content moderation system. The development journey, structured across four distinct sprints, demonstrates a progressive build from a foundational social platform to a sophisticated, AI-integrated application.

In the initial sprint, we laid the essential groundwork by developing the core social architecture. This included a robust user authentication system, the central post feed for content sharing, and a real-time notification system. These features established the fundamental user interactions necessary for a dynamic social platform.

The second sprint enriched the user experience and established administrative control. We introduced detailed user profiles, the ability for users to edit their account information, and, most critically, a comprehensive Admin Dashboard. This dashboard equips administrators with the tools to manage the user base, including viewing user lists and modifying user roles, thereby setting the stage for effective governance.

The third sprint focused on building a robust moderation framework. We implemented a manual reporting system, allowing users to flag inappropriate content (posts, comments, and replies). The Admin Dashboard was enhanced to display this reported content, giving human moderators the authority to review and take action. This sprint was also pivotal in preparing for the AI component, involving the curation and preparation of a multilingual dataset for training the hate speech detection model.

The final and most innovative sprint represented the culmination of the project's core vision: the development and integration of the AI-powered hate speech detection model. Leveraging a fine-tuned Transformer-based model, we created a system capable of analyzing multilingual content in real-time. This was deployed as a dedicated Python

microservice using FastAPI and seamlessly integrated into the MERN stack application. The integration operates on two levels:

- **Proactive Blocking:** A content verification middleware that intercepts new posts and comments, preventing content classified as hate speech from ever appearing on the platform.
- **Monitoring and Logging:** All AI detection events, both blocked and allowed, are logged in a dedicated AiDetectionLog collection, providing administrators with a transparent and detailed view of the model's activity and the platform's overall health.

From a technical standpoint, the project successfully integrated two distinct technology stacks: a primary MERN (MongoDB, Express.js, React, Node.js) stack for the web application and a Python/TensorFlow/FastAPI stack for the deep-learning microservice. This architecture ensures a separation of concerns, allowing the AI service to be scaled and updated independently of the main application, a critical consideration for real-world deployment. The development process was not without its challenges. Key hurdles included finding and collecting sufficient quality data in multiple languages for model training, acquiring access to very high-end resources to accommodate the implementation, fine-tuning, and training of a complex transformer-based model, dealing with the complexities of integrating two different technology ecosystems, ensuring low-latency responses from the AI model to maintain a smooth user experience, and the inherent difficulty of creating a nuanced hate speech model that understands context across multiple languages. The solutions, such as using a lightweight FastAPI server and carefully designing the data flow between services, represent significant learning outcomes.

While WorkSphere stands as a complete and functional application, there are several avenues for future work. The AI model could be continuously improved by retraining it on data gathered from manual moderation decisions on the platform. Its capabilities could be expanded to include image and video content analysis for a more comprehensive moderation solution. Furthermore, the platform could be enhanced with features like sentiment analysis to gauge employee morale or integration with other enterprise tools like calendars and project management software.

Finally, WorkSphere is more than a proof-of-concept; it is a comprehensive blueprint for the next generation of corporate communication tools. It successfully demonstrates that a feature-rich social platform can be powerfully combined with cutting-edge artificial intelligence to create a digital workplace that is not only collaborative and efficient but also fundamentally safe, respectful, and inclusive. This work serves as a testament to

the practical and impactful application of modern software engineering and deep-learning principles to solve tangible business challenges.

Bibliography

- [1] <https://hasocfire.github.io/hasoc/2020/dataset.html>.
- [2] https://huggingface.co/datasets/nedjmaou/MLMA_hate_speech.
- [3] <https://github.com/rewire-online/multilingual-hatecheck>.
- [4] https://github.com/shammur/arabic_offensive_language_detection.
- [5] Overleaf. <https://fr.wikipedia.org/wiki/Overleaf>. Accessed on 03/25/2024.
- [6] Postman. [https://fr.wikipedia.org/wiki/Postman_\(logiciel\)](https://fr.wikipedia.org/wiki/Postman_(logiciel)). Accessed on 02/25/2024.
- [7] Scrum. <https://www.scrum.org/resources/what-scrum-module>. (Accessed on 05/02/2024).
- [8] Visual Studio Code. https://fr.wikipedia.org/wiki/Visual_Studio_Code. Accessed on 03/02/2024.
- [9] Hatexplain: A benchmark dataset for explainable hate speech detection. 35:14867–14875, May 2021.
- [10] Express - node.js web application framework. <https://expressjs.com/>, 2025. Accessed on 21/04/2025.
- [11] Google colaboratory – colab pro. <https://colab.research.google.com/signup>, 2025. Accessed on 21/04/2025.
- [12] MongoDB atlas – multi-cloud developer data platform. <https://www.mongodb.com/atlas>, 2025. Accessed on 21/04/2025.
- [13] Node.js. <https://nodejs.org/en/about/>, 2025. Accessed on 21/04/2025.

- [14] Vite documentation – scaffolding your first vite project. <https://vitejs.dev/guide/#scaffolding-your-first-vite-project>, 2025. Accessed on 21/04/2025.
- [15] P. Badjatiya, S. Gupta, M. Gupta, and V. Varma. Deep learning for hate speech detection in tweets. In *Proceedings of the 26th international conference on World Wide Web companion*, pages 759–760, 2017.
- [16] S. A. Chowdhury, H. Mubarak, A. Abdelali, S.-g. Jung, B. J. Jansen, and J. Salminen. A multi-platform Arabic news comment dataset for offensive language detection. In N. Calzolari, F. Béchet, P. Blache, K. Choukri, C. Cieri, T. Declerck, S. Goggi, H. Isahara, B. Maegaard, J. Mariani, H. Mazo, A. Moreno, J. Odijk, and S. Piperidis, editors, *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 6203–6212, Marseille, France, May 2020. European Language Resources Association.
- [17] H. M. M. G. Y. Y. S. S. M. J. Clarke, C. Rule by example: Harnessing logical rules for explainable hate speech detection. 1:364–376, 2023.
- [18] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8440–8451. Association for Computational Linguistics, 2020.
- [19] W. D. M. M. . W. I. Davidson, T. Automated hate speech detection and the problem of offensive language. pages 512–515, 2017.
- [20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, 2019. Association for Computational Linguistics.
- [21] EVERFI. Preventing harassment and discrimination in a remote or hybrid workplace. *EVERFI Blog*, 2023. Accessed: April 2, 2025.
- [22] F. Feng, Y. Yang, D. Cer, N. Arivazhagan, and W. Wang. Language-agnostic bert sentence embedding, 2022.
- [23] . N. S. Fortuna, P. A survey on automatic detection of hate speech in text. *ACM Computing Surveys.*, 51(4):1–30, 2019.

- [24] Gallup. State of the global workplace 2023 report, 2023. Accessed: April 2, 2025.
- [25] A. Gandhi, P. Ahir, K. Adhvaryu, P. Shah, R. Lohiya, E. Cambria, S. Poria, and A. Hussain. Hate speech detection: A comprehensive review of recent works. *Expert Systems*, page 13562, 2024.
- [26] J. Hu, S. Ruder, A. Siddhant, G. Neubig, O. Firat, and M. Johnson. Xtreme: A massively multilingual multi-task benchmark for evaluating cross-lingual generalization, 2020.
- [27] K. K. Z. Wang, S. Mayhew, and D. Roth. Cross-lingual ability of multilingual bert: An empirical study, 2020.
- [28] Y. Liu. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 364, 2019.
- [29] J. S. Malik, H. Qiao, G. Pang, and A. van den Hengel. Deep learning for hate speech detection: a comparative study. *International Journal of Data Science and Analytics*, pages 1–16, 2024.
- [30] N. Ousidhoum, Z. Lin, H. Zhang, Y. Song, and D.-Y. Yeung. Multilingual and multi-aspect hate speech analysis, 2019.
- [31] T. Pires, E. Schlinger, and D. Garrette. How multilingual is multilingual bert?, 2019.
- [32] P. Röttger, B. Vidgen, D. Nguyen, Z. Waseem, H. Margetts, and J. Pierrehumbert. HateCheck: Functional tests for hate speech detection models. In C. Zong, F. Xia, W. Li, and R. Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 41–58, Online, Aug. 2021. Association for Computational Linguistics.
- [33] V. Sanh. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [34] ThriveSparrow. The importance of open communication in the workplace. *ThriveSparrow Blog*, 2023. Accessed: April 2, 2025.
- [35] A. Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [36] T. W. R. . W. I. Waseem, Z. Davidson. Understanding abuse: A typology of abusive language detection. *ACL Anthology*.

- [37] Z. Waseem and D. Hovy. Hateful symbols or hateful people? predictive features for hate speech detection on twitter. In *Proceedings of the NAACL-HLT 2016 Workshop on Abusive Language Online*, pages 88–93, 2016.
- [38] P. J. Zak. The neuroscience of trust. *Harvard Business Review*, 2017. Accessed: April 2, 2025.
- [39] L. Zhang, S. Wang, and B. Liu. Deep learning for sentiment analysis: A survey. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 8(4):e1253, 2018.